ECMA-367-2026-with-Style-Guide

ECMA-367:2026 Edition with Eiffel Style Guide

Eiffel: Analysis, Design and Programming Language

2026 Edition

> This is the complete ECMA-367 standard for the Eiffel programming language, incorporating all language features through 2026, supplemented with the official Eiffel Style Guide covering naming conventions, code organization, and best practices.

Table of Contents

Sections 1-7: Fundamentals

- Section 1: Scope
- Section 2: Conformance
- Section 3: Normative references
- Section 4: Definitions
- Section 5: Notational conventions
- Section 6: Acronyms and abbreviations
- Section 7: General description

Section 8: Language Specification

- 8.1: General organization
- 8.2: Syntax, validity and semantics
- 8.3: The architecture of Eiffel software
- 8.4: Classes
- 8.5: Features
- 8.6: The inheritance relation
- 8.7: Clients and exports
- 8.8: Routines
- 8.9: Correctness and contracts
- 8.10: Feature adaptation
- 8.11: Types
- 8.12: Genericity
- 8.13: Tuples
- 8.14: Conformance
- 8.15: Convertibility
- 8.16: Repeated inheritance
- 8.17: Control structures
- 8.18: Attributes
- 8.19: Objects, values and entities
- 8.20: Creating objects
- 8.21: Comparing and duplicating objects
- 8.22: Attaching values to entities

- 8.23: Feature call
- 8.24: Eradicating void calls
- 8.25: Typing-related properties
- 8.26: Exception handling
- 8.27: Agents
- 8.28: Expressions
- 8.29: Constants
- 8.30: Basic types
- 8.31: Interfacing with C, C++ and other environments
- 8.32: Lexical components
- 8.33: **Void**-safety system
- 8.34: Concurrency with SCOOP
- 8.35: ACROSS iteration
- 8.36: Modern type system
- 8.37: Enhanced agents
- 8.38: Feature declaration enhancements
- 8.39: Modern expression forms
- 8.40: Non-conforming inheritance
- 8.41: Multiple generic constraints
- 8.42: Annotation and metadata

Sections 9-10

- Section 9: External Interfaces
- Section 10: Standard Library

Appendices

- Appendix A: BNF-E Notation Reference
- Appendix B: Backward Compatibility Matrix
- Appendix C: Feature Adoption Timeline
- Appendix D: **Void**-Safety Migration Guide
- Appendix E: SCOOP Best Practices
- Appendix F: Cross-References to ECMA-367:2006
- Appendix G: Modernization Checklist

Glossary

Part II: Eiffel Style Guide

- 11: Style guide overview
- 12: Class naming
- 13: Feature naming
- 14: Argument naming
- 15: Local variable naming
- 16: Attachment local naming
- 17: Other variable binding contexts
- 18: Constants and enumerations
- 19: Attribute naming

- 20: Model query naming
- 21: Design by Contract patterns
- 22: Command-query separation patterns
- 23: Visibility and **feature** organization
- 24: Type annotations and genericity
- 25: Anti-patterns
- 26: Alias patterns
- 27: Testing and verification naming
- 28: Complete example

---

8 Language specification

8.1 General organization

Informative text

End

8.2 Syntax, validity and semantics

8.2.1 Definition: Syntax, BNF-E

Syntax is the set of rules describing the structure of software texts. The notation used to define Eiffel's syntax is called BNF-E.

Informative text

End

8.2.2 Definition: Component, construct, specimen

Any **class** text, or syntactically meaningful part of it, such as an instruction, an expression or an identifier, is called a component. The structure of any kind of components is described by a construct. A component of a kind described by a certain construct is called a specimen of that construct.

Informative text

class
class
class
class
class

End

8.2.3 Construct Specimen convention

The phrase ‘‘an X’’, where X is the name of a construct, serves as a shorthand for ‘‘a specimen of X’’.

Informative text

End

8.2.4 Construct Nameconvention

Every construct has a name starting with an upper-case letter and continuing with lower-case letters, possibly with underscores (to **separate** parts of the name if it uses several English words).

Informative text

Result

End

8.2.5 Definition: Terminal, non-terminal, token

Specimens of a terminal construct have no further syntactical structure. Examples include: • Reserved words such as if and **Result**. • Manifest constants such as the integer 234; symbols such as ; (semicolon) and + (plus sign) .

• Identifiers (used to denote classes, features, entities) such as LINKED_LIST and put . The specimens of terminal constructs are called tokens. In contrast, the specimens of a non-terminal construct are defined in terms of other constructs.

Informative text

End

8.2.6 Definition: Production

A production is a formal description of the structure of all specimens of a non-terminal construct. It has the form Construct

≜ right-side where right-side describes how to obtain specimens of the Construct.

Informative text

≜ may be read aloud **as** “is defined **as**”. BNF-E uses exactly one production for each non-terminal. The reason for this convention is explained below.

End

8.2.7 Kinds of production

A production is of one of the following three kinds, distinguished by the form of the right-side: • Aggregate, describing a construct whose specimens are made of a fixed sequence of parts, some of which may be optional. • Choice, describing a construct having a set of given variants. • Repetition, describing a construct whose specimens are made of a variable number of parts, all specimens of a given construct.

8.2.8 Definition: Aggregate production

An aggregate right side is of the formC 1 C 2 ... C n (n > 0), where every one of the C iisa construct and any contiguous subsequence may appear in square brackets as [C i ... C j] for 1 ≤ i ≤ j ≤ n. Every specimen of the corresponding construct consists of a specimen of C 1 , followed by a specimen of C 2 , ..., followed by a specimen of C n , with the provision that for any subsequence in brackets the corresponding specimens may be absent.

8.2.9 Definition: Choice production

A choice right side is of the formC 1

```
| C
```

2

```
| ... | C
```

n (n > 1), where every one of the C iisa construct. Every specimen of the corresponding construct consists of exactly one specimen of one of the C i .

8.2.10 Definition: Repetition production, separator

A repetition right side is of one of the two forms {C § ...} *{C § ...} + where C and § (the separator) are constructs. Every specimen of the corresponding construct consists of zero or more (one or more in the second form) specimens of C, each separated from the next, if any, by a specimen of §. The following abbreviations may be used if the separator is empty: C* C +

Informative text

End

8.2.11 Basic syntax description rule

Every non-terminal construct is defined by exactly one production.

Informative text

End

8.2.12 Definition: Non-production syntax rule

A non-production syntax rule, marked “(non-production)”, is a syntax property expressed outside of the BNF-E formalism.

Informative text

End

8.2.13 Textual conventions

The syntax (BNF-E) productions and other rules of the Standard apply the following conventions:

1 Symbols of BNF-E itself, such as the vertical bars | signaling a choice production, appear

in black (non-bold, non-italic).

2 Any construct name appears in dark green (non-bold, non-italic), with a first letter in

upper case, as Class.

3 Any component (Eiffel text element) appears in blue.

4 The double quote, one of Eiffel’s special symbols, appears in productions as ' " ': a double

quote character (blue like other Eiffel text) enclosed in two single quote characters (black since they belong to BNF-E, not Eiffel).

5 All other special symbols appear in double quotes, for example a comma as ",", an

assignment symbol as " :=", a single quote as "'" (double quotes black, single quote blue).

6 Keywords and other reserved words, such as **class** and **Result**, appear in bold (blue like

other Eiffel text). They do not **require** quotes since the conventions avoid ambiguity with construct names: Class is the name of a construct, **class** a keyword.

7 Examples of Eiffel comment text appear in non-bold, non-italic (and in blue), as -- A

comment .

8 Other elements of Eiffel text, such as entities and **feature** names (including in comments)

appear in non-bold italic (blue). The color-related parts of these conventions do not affect the language definition, which remains unambiguous under black-and-white printing (thanks to the letter-case and font parts of the conventions). Color printing is recommended for readability.

Informative text

class
variant
feature
class

End

8.2.14 Definition: Validity constraint

A validity constraint on a construct is a requirement that every syntactically well-formed specimen of the construct must satisfy to be acceptable as part of a software text.

8.2.15 Definition: Valid

A construct specimen, built according to the syntax structure defined by the construct's production, is said to be valid, and will be accepted by the language processing tools of any Eiffel environment, if and only if it satisfies the validity constraints, if any, applying to the construct.

8.2.16 Validity: General Validity rule

Validity code: **VBGV** Every validity constraint relative to a construct is considered to include an implicit supplementary condition stating that every component of the construct satisfies every validity constraint applicable to the component.

8.2.17 Definition: Semantics

The semantics of a construct specimen that is syntactically legal and valid is the construct's effect on the execution of a system that includes the specimen.

8.2.18 Definition: Execution terminology

• Run time is the period during which a system is executed. • The machine is the combination of hardware (one or more computers) and operating system through which you can execute systems. • The machine type, that is to say a certain combination of computer type and operating system, is called a platform. • Language processing tools serve to build, manipulate, explore and execute the text of an Eiffel system on a machine.

Informative text

End

8.2.19 Semantics: Case Insensitivity principle

In writing the letters of an Identifier serving as name for a **class**, **feature** or entity, or a reserved word, using the upper-case or lower-case versions has no effect on the semantics.

Informative text

class
feature

End

8.2.20 Definition: Upper name, lower name

The upper name of an Identifier or Operator iisi written with all letters in upper case; its lower name , i with all letters in lower case.

Informative text

variant
class

End

8.2.21 Syntax (non-production): Semicolon Optionality rule

In writing specimens of any construct defined by a Repetition production specifying the semicolon ";" as separator, it is permitted, without any effect on the syntax structure, validity and semantics of the software, to omit any of the semicolons, or to add a semicolon after the last element.

Informative text

The rule also guarantees that an extra semicolon at the end, as in a; b; instead of just a; b is harmless. The style guidelines suggest omitting semicolons (which would only obscure reading) for successive elements appearing on **separate** lines, as is usually the case for instructions and declarations, and including them to **separate** elements on a given line. Because the semicolon is still formally in the grammar, programmers used to languages where the semicolon is an instruction terminator, who may then out of habit add a semicolon after every instruction, will not suffer any adverse effect, and will get the expected meaning.

End

8.3 The architecture of Eiffel software

Informative text

End

8.3.1 Definition: Cluster, subcluster, contains directly, contains

A cluster is a collection of classes, (recursively) other clusters called its subclusters, or both. The cluster is said to contain directly these classes and subclusters. A cluster contains a **class** C if it contains directly either C or a cluster that (recursively) contains C.

Informative text

class

End

8.3.2 Definition: Terminal cluster, internal cluster

A cluster is terminal if it contains directly at least one **class**. A cluster is internal if it contains at least one subcluster.

Informative text

End

8.3.3 Definition: Universe

A universe is a set of classes.

Informative text

End

8.3.4 Syntax: Class names

Class_name

≙ Identifier

8.3.5 Validity: Class Name rule

Validity code: **VSCN** It is valid for a universe to include a **class** if and only if no other **class** of the universe has the same upper name.

Informative text

class
class

End

8.3.6 Semantics: Class name semantics

A Class_name C appearing in the text of a **class** D denotes the **class** called C in the enclosing universe.

8.3.7 Definition: System, root type name, root procedure name

A system is defined by the combination of:

1 A universe.

2 A type name, called the root type name.

3 A **feature** name, called the root procedure name.

8.3.8 Definition: Type dependency

A type T depends on a type R if any of the following holds:

1 R is a parent of the base **class** C of T.

2 T is a client of R.

3 (Recursively) there is a type S such that T depends on S and S depends on R.

Informative text

1 relies on the property that every type derives from a **class**, called its “base **class**”; for

example a generically derived type such as LIST [INTEGER] has base **class** LIST. Case 3 gives us indirect forms of dependency, derived from the other cases.

End

8.3.9 Validity: Root Type rule

Validity code: `VSRT` It is valid to designate a type TN as root type of a system of universe U if and only if it satisfies the following conditions:

1 TN is the name of a stand-alone type T.

2 T only involves classes in U.

3 T's base **class** is not **deferred**.

4 The base **class** of any type on which T depends is in U.

Informative text

End

8.3.10 Validity: Root Procedure rule

Validity code: `VSRP` It is valid to specify a name pn as root procedure name for a system S if and only if it satisfies the following conditions:

1 pn is the name of a creation procedure p of S's root type.

2 p has no formal argument.

3 p is precondition-free.

Informative text

Regarding condition 1, notethat a non-**deferred class** that doesn't explicitly list any creation procedures is understood to have a single one, procedure default_create, which does nothing by default but may be redefined in any **class** to carry out specific initializations.

End

8.3.11 Definition: Root type, root procedure, root class

In a system S of root type name TN and root procedure name pn, the root type is the type of name TN, the root **class** is the base **class** of that root type, and the root procedure is the procedure of name pn in that **class**.

8.3.12 Semantics: System execution

To execute a system on a machine means to cause the machine to apply a creation instruction to the system's root type.

Informative text

End

8.4 Classes

Informative text

End

8.4.1 Definition: Current class

The current **class** of a construct specimen is the **class** in which it appears.

Informative text

feature
class
class

End

8.4.2 Syntax: Class declarations

Class_declaration ≙ [Notes]

Class_header
[Formal_generics]
[Obsolete]
[Inheritance]
[Creators]
[Converters]
[Features]
[Invariant]
[Notes]
end

8.4.3 Syntax: Notes

Notes ≙ **note** Note_list

Note_list ≙ {Note_entry ";" …}*

Note_entry ≙ Note_name Note_values

Note_name ≙ Identifier ":"

Note_values ≙ {Note_item ","…}$^{+}$

Note_item ≙ Identifier | Manifest_constant

Informative text

Notes parts (there may be up to two, one at the beginning and one at the end) have no effect on the execution semantics of the **class**. They serve to associate information with the **class**, for use in particular by tools for configuration management, documentation, cataloging, archival, and for retrieving classes based on their properties.

End

8.4.4 Semantics: Notes semantics

A Notes part has no effect on system execution.

8.4.5 Syntax: Class headers

Class_header ≙ [Header_mark] **class** Class_name

Header_mark ≜ **deferred** | **expanded** | **frozen**

Informative text

The Class_name part gives the name of the **class**. The recommended convention (here and in any context where a **class** text refers to a **class** name) is the upper name. The keyword **class** may optionally be preceded by one of the keywords **deferred**, **expanded** and **frozen**, corresponding to variants of the basic notion of **class**:

- A **deferred class** describes an incompletely implemented abstraction, which descendants will use as a basis for further refinement.
- Declaring a **class** as **expanded** indicates that entities declared of the corresponding type will denote objects rather than references to objects.
- Making a **class frozen** prohibits it from serving as "conforming parent" to other classes.

End

8.4.6 Validity: Class Header rule

Validity code: **VCCH** A Class_header appearing in the text of a **class** C is valid if and only if has either no **deferred feature** or a Header_mark of the **deferred** form.

Informative text

class
deferred
feature
deferred
class
deferred
deferred
class
deferred
class
expanded
frozen
attached

End

8.4.7 Definition: Expanded, frozen, deferred, effective class

A **class** is: • Expanded if its Class_header is of the **expanded** form. • Frozen if its Class_header is of the **frozen** or **expanded** form.

• Deferred if its Class_header is of the **deferred** form. • Effective if it is not **deferred**.

Informative text

frozen
expanded
expanded
class
frozen
expanded

class
frozen
class
deferred
deferred
feature
class
deferred
deferred
class
deferred
deferred
class

End

8.4.8 Syntax: Obsolete marks

Obsolete

obsolete Message Message

Manifest_string

8.4.9 Semantics: Obsolete semantics

Specifying an Obsolete mark for a **class** or **feature** has no run-time effect. When encountering such a mark, language processing tools may issue a report, citing the obsolescence Message and advising software authors to replace the **class** or **feature** by a newer version.

8.5 Features

Informative text

End

8.5.1 Definition: Inherited, immediate; origin; redeclaration; introduce

Any **feature** f of a **class** C is of one of the following two kinds:

1 If C obtains f from one of its parents, f is an inherited **feature** of C. In this case any

declaration of f in C (adapting the original properties of f for C) is a redeclaration.

2 If a declaration appearing in C applies to a **feature** that is not inherited, the **feature** is said

to be immediate in C. Then C is the origin (short for "**class** of origin") of f, and is said to introduce f.

Informative text

End

8.5.2 Syntax: Feature parts

Features

≙ Feature_clause + Feature_clause

≙ **feature** [Clients] [Header_comment] Feature_declaration_list Feature_declaration_list

≙ {Feature_declaration ";" …}* Header_comment

≙ Comment

Informative text

and

End

8.5.3 Feature categories: overview

Every **feature** of a **class** is either an attribute or a routine. An attribute is either constant or variable. A routine is either a procedure or a function.

Informative text

feature
feature

End

8.5.4 Syntax: Feature declarations

Feature_declaration

≙ New_feature_list Declaration_body Declaration_body

≙ [Formal_arguments] [Query_mark] [Feature_value] Query_mark

≙ Type_mark [Assigner_mark] Type_mark

≙ ":" Type Feature_value

≙ [Explicit_value] [Obsolete] [Header_comment] [Attribute_or_routine] Explicit_value

≙ "=" Manifest_constant

Informative text

all
and
and
from
and
all

End

8.5.5 Syntax: New feature lists

New_feature_list

≜ {New_feature "," …} + New_feature

≜ [**frozen**] Extended_feature_name

Informative text

or

End

8.5.6 Syntax: Feature bodies

Attribute_or_routine

≜ [Precondition] [Local_declarations] Feature_body [Postcondition] [Rescue] **end** Feature_body

≜ Deferred | Effective_routine | Attribute

8.5.7 Validity: Feature Body rule

Validity code: **VFFB** A Feature_value is valid if and only if it satisfies one of the following conditions:

1 It has an Explicit_value and no Attribute_or_routine.

2 It has an Attribute_or_routine with a Feature_body of the Attribute kind.

3 It has no Explicit_value and has an Attribute_or_routine with a Feature_body of the

Effective_routine kind, itself of the Internal kind (beginning with do or once).

4 It has no Explicit_value and has an Attribute_or_routine with neither a

Local_declarations nor a Rescue part, and a Feature_body that is either Deferred or an Effective_routine of the External kind.

Informative text

3 and 4 exclude this possibility.

The Local_declarations and Rescue parts only make sense (case 4) for a **feature** with an associated algorithm present in the **class** text itself; this means a routine that is neither **deferred** nor external, or an attribute with explicit initialization. In both cases 1 and 2 the **feature** will be an attribute. Case 1 is a constant attribute declaration such as n: INTEGER = 100, with no further details. Case 2 is the long form, explicitly using the keyword attribute and making it possible, as with routines, to have a Precondition, a Postcondition, and even an implementation (including a Rescue clauseif desired) which will be used, for “self-initializing” types, on first use of an uninitialized field. The Feature Body rule is the basic validity condition on **feature** declarations. But for a full view of the constraints we must take into account a set of definitions appearing next, which say what it takes for a **feature** declaration — already satisfying the Feature Body rule — to belong to one of the relevant categories: variable attribute, constant attribute, function, procedure. Another fundamental constraint, the Feature Declaration rule (**VFFD**), will then **require** that the **feature** described by any declaration match one of these categories. So the definitions below are a little more than definitions: they collectively yield a validity requirement complementing the Feature Body rule.

End

8.5.8 Definition: Variable attribute

A Feature_declaration is a variable attribute declaration if and only if it satisfies the following conditions:

1 There is no Formal_arguments part.

2 There is a Query_mark part.

3 There is no Explicit_value part.

4 If there is a Feature_value part, it has an Attribute_or_routine with a Feature_body of the

Attribute kind.

8.5.9 Definition: Constant attribute

A Feature_declaration is a constant attribute declaration if and only if it satisfies the following conditions:

1 It has no Formal_arguments part.

2 It has a Query_mark part.

3 There is a Feature_value part including an Explicit_value.

8.5.10 Definition: Routine, function, procedure

A Feature_declaration is a routine declaration if and only if it satisfies the following condition: • There is a Feature_value including an Attribute_or_routine, whose Feature_body is of the Deferred or Effective_routine kind. If a Query_mark is present, the routine is a function; otherwise it is a procedure.

Informative text

deferred
feature

End

8.5.11 Definition: Command, query

A command is a procedure. A query is an attribute or function.

Informative text

End

8.5.12 Definition: Signature, argument signature of a feature

The signature of a **feature** f is a pair argument_types, result_type where argument_types and result_type are the following sequences of types: • For argument_types: if f is a routine, the possibly empty sequence of its formal argument types, in the order of the arguments; if f is an attribute, an empty sequence. • For result_type: if f is a query, a one-element sequence, whose element is the type of f; if f is a procedure, an empty sequence. The argument_types part is the **feature**'s argument signature.

Informative text

End

8.5.13 Feature principle

Every **feature** has an associated identifier. Any valid call (qualified or unqualified) to the **feature** can be expressed through this identifier.

Informative text

feature
class
feature

End

8.5.14 Syntax: Feature names

Extended_feature_name

≙ Feature_name [Alias] Feature_name

≙ Identifier Alias

≙ **alias** "" Alias_name "" [**convert**] Alias_name

≙ Operator | Bracket Bracket

≙ "[]"

Informative text

convert
feature
as
from
not

End

8.5.15 Syntax (non-production): Alias Syntax rule

The Alias_name of an Alias must immediately follow and precede the enclosing double quote symbols, with no intervening characters (in particular no breaks). When appearing in such an Alias_name, the two-word operators and then and or else must be written with exactly one space (but no other characters) between the two words.

Informative text

End

8.5.16 Definition: Operator feature, bracket feature, identifier-only

A **feature** is an operator **feature** if its Extended_feature_name fn includes an Operator alias, a bracket **feature** if fn includes a Bracket alias. It is identifier-only if neither of these cases applies.

Informative text

feature
feature
class
feature
feature

class
feature
feature

End

8.5.17 Definition: Identifier of a feature name

The Identifier that starts a Extended_feature_name is called the identifier of that Extended_feature_name and, by extension, of the associated **feature**.

8.5.18 Validity: Feature Identifier principle

Given a **class** C and an identifierf, C contains at most one **feature** of identifierf.

Informative text

feature
class

End

8.5.19 Definition: Same feature name, same operator, same alias

Two **feature** names are considered to be “the same **feature** name” if and only if their identifiers have identical lower names. Two operators are “the same operator” if they have identical lower names. An Alias in an Extended_feature_name is “the same alias” as another if and only if they satisfy the following conditions: • They are either the same Operator or both Bracket. • If either has a **convert** mark, so does the other.

Informative text

feature
feature
feature

This notion is useful in particular to enforce the rule that, in any **class**, there can be only one **feature** of a given name (no “overloading”), and to determine what constitutes a “name clash” under multiple inheritance. In such cases the language rules simply ignore letter case.

End

8.5.20 Syntax: Operators

Operator

≜ Unary | Binary Unary

≜ **not** | "+" | "–" | Free_unary Binary

≜ "+" | "–" | "*" | "/" | "//" | "" | "^" | ".." | "<" | ">" | "<=" | ">=" | **and** | **or** | **xor** | **and then** | **or else** | **implies** | Free_binary

Informative text

as
or
as

as
alias
or
as

End

8.5.21 Syntax: Assigner marks

Assigner_mark

assign Feature_name

Informative text

attribute
as
assign
not
as
all
and
class
class
as
as
not
as
only
alias
as
not
all
as
class
from

End

8.5.22 Validity: Assigner Command rule

Validity code: **VFAC** An Assigner_mark appearing in the declaration of a query q with n arguments (n ≥ 0) and listing a Feature_name fn, called the assigner command for q, is valid if and only if it satisfies the following conditions:

1 fn is the identifier of a command c of the **class**.

2 c has n + 1 arguments.

3 The type of c's first argument and the result type of q have the same deanchored form.

4 For every i in 1..n, the type of the i+1-st argument of c and the type of the i-th argument

of q have the same deanchored form.

Informative text

End

8.5.23 Definition: Synonym

A synonym of a **feature** of a **class** C is a **feature** with a different Extended_feature_name such that both names appear in the same New_feature_list of a Feature_declaration of C.

8.5.24 Definition: Unfolded form of a possibly multiple declaration

The unfolded form of a Feature_declaration listing one or more **feature** names, as in: f 1 , f 2 , … , f n declaration_body (n ≥ 1) where each f iisa New_feature, is the corresponding sequence of declarations naming only one **feature** each, and with identical declaration bodies, as in: f 1 declaration_body f 2 declaration_body ... f n declaration_body

Informative text

feature
feature
class
feature
class
class
rename
frozen

End

8.5.25 Validity: Feature Declaration rule

Validity code: **VFFD** A Feature_declaration appearing in a **class** C is valid if and only if it satisfies all of the following conditions for every declaration of a **feature** f in its unfolded form:

1 The Declaration_body describes a **feature** which, according to the rules given earlier, is

one of: variable attribute, constant attribute, procedure, function.

2 f does not have the same **feature** name as any other **feature** introduced in C (in

particular, any other **feature** of the unfolded form).

3 If f has the same **feature** name as the final name of any inherited **feature**, the

Declaration_body satisfies the Redeclaration rule.

4 If the Declaration_body describes a **deferred feature**, then the Extended_feature_name

of f is not preceded by **frozen**.

5 If the Declaration_body describes a once function, the result type is stand-alone.

6 Any anchored type for an argument is **detachable**.

7 The Alias clause, if present, is alias-valid for f.

Informative text

End

8.5.26 Validity: Alias Validity rule

Validity code: **VFAV** An Alias clause is alias-valid for a **feature** f of a **class** C if and only if it satisfies the following conditions:

1 If it lists an Operator op: f is a query; no other query of C has an Operator alias using the

same operator and the same number of arguments; and either: op is a Unary and f has no argument, or op is a Binary and f has one argument.

2 If it lists a Bracket alias: f is a query with at least one argument, and no other **feature** of

C has a Bracket alias.

3 If it includes a **convert** mark: it lists an Operator and f has one argument.

Informative text

2 tells us that there may be at most one bracket **feature** per **class**.

Condition 3 indicates that a **convert** mark, specifying “target conversion” as in your_integer + your_real , makes sense only for features with one argument, with an Operator which (from condition

1. must be a Binary.

End

8.6 The inheritance relation

Informative text

End

8.6.1 Syntax: Inheritance parts

Inheritance

- Inherit_clause + Inherit_clause
- **inherit** [Non_conformance] Parent_list Non_conformance
- "{" NONE "}" Parent_list
- {Parent ";" …} + Parent
- Class_type [Feature_adaptation] Feature_adaptation
- [Undefine] [Redefine] [Rename] [New_exports] [Select] **end**

Informative text

all
as

End

8.6.2 Syntax (non-production): Feature adaptation rule

A Feature_adaptation part must include at least one of the optional components.

Informative text

class
inherit
class

End

8.6.3 Definition: Parent part for a type, for a class

If a Parent partp of an Inheritance part lists a Class_type T, p is said to be a Parent part for T, and also for the base **class** of T.

Informative text

inherit
class

Informative text

class
class
class

End

8.6.4 Validity: Class ANY rule

Validity code: `VHCA` Every system must include a non-generic **class** called ANY.

Informative text

separate

End

8.6.5 Validity: Universal Conformance principle

Validity code: `VHUC` Every type conforms to ANY.

Informative text

class
class

End

8.6.6 Definition: Unfolded inheritance part of a class

Any **class** C has an unfolded inheritance part defined as follows:

1 If C has an Inheritance part: that part.

2 Otherwise: an Inheritance part of the form **inherit** ANY.

8.6.7 Definition: Multiple, single inheritance

A **class** has multiple inheritance if it has an unfolded inheritance part with two or more Parent parts. It has single inheritance otherwise.

Informative text

class
class
class

End

8.6.8 Definition: Inherit, heir, parent

A **class** C inherits from a type or **class** B if and only if C's unfolded inheritance part contains a Parent part for B. B is then a parent of C ("parent type" or "parent **class**" if there is any ambiguity), and C an heir (or "heir **class**") of B. Any type of base **class** C is also an heir of B ("heir type" in case of ambiguity).

8.6.9 Definition: Conforming, non-conforming parent

A parent B in an Inheritance part is non-conforming if and only if every Parent part for B in the clause appears in an Inherit_clause with a Non_conformance marker. It is conforming otherwise.

8.6.10 Definition: Ancestor types of a type, of a class

The ancestor types of a type CT of base **class** C include:

1 CT itself.

2 (Recursively) The result of applying CT's generic substitution to the ancestor types of

every parent type for C. The ancestor types of a **class** are the ancestor types of its current type.

Informative text

class
class
inherit
class

End

8.6.11 Definition: Ancestor, descendant

Class A is an ancestor of **class** B if and only if A is the base **class** of an ancestor type of B. Class B is a descendant of **class** A if and only if A is an ancestor of B.

Informative text

End

8.6.12 Definition: Proper ancestor, proper descendant

The proper ancestors of a **class** C are its ancestors other than C itself. The proper descendants of a **class** B are its descendants other than B itself.

8.6.13 Validity: Parent rule

Validity code: **VHPR** The unfolded inheritance part of a **class** D is valid if and only if it satisfies the following conditions:

1 In every Parent part for a **class** B, B is not a descendant of D.

2 No conforming parent is a **frozen class**.

3 If two or more Parent parts are for classes which have a common ancestor A, D meets

the conditions of the Repeated Inheritance Consistency constraint for A.

4 At least one of the Parent parts is conforming.

5 No two ancestor types of D are different generic derivations of the same **class**.

6 Every Parent is generic-creation-ready.

Informative text

End

8.6.14 Syntax: Rename clauses

Rename

rename Rename_list Rename_list

{Rename_pair "," …} + Rename_pair

Feature_name **as** Extended_feature_name

Informative text

feature
alias
feature
feature
or
alias
or
alias
if

Forms of **feature** adaptation other than renaming, in particular effecting and redefinition, do not affect the Alias, if any, associated with a Feature_name.

End

8.6.15 Validity: Rename Clause rule

Validity code: **VHRC** A Rename_pair of the formold_name as new_name, appearing in the Rename subclause of the Parent part for B in a **class** C, is valid if and only if it satisfies the following conditions:

1 old_name is the final name of a **feature** f of B.

2 old_name does not appear as the first element of any other Rename_pair in the same

Rename subclause.

3 new_name satisfies the Feature Name rule for C.

4 The Alias of new_name, if present, is alias-valid for the version of f in C.

Informative text

End

8.6.16 Semantics: Renaming principle

Renaming does not affect the semantics of an inherited **feature**.

Informative text

feature

End

8.6.17 Definition: Final name, extended final name, final name set

Every **feature** f of a **class** C has an extended final name in C, an Extended_feature_name, and a final name, a Feature_name, defined as follows:

1 The final name is the identifier of the extended final name.

2 If f is immediate in C, its extended final name is the Extended_feature_name under

which C declares it.

3 If f is inherited, f is obtained from a **feature** of a parent B of C. Let

extended_parent_name be (recursively) the extended final name of that **feature** in B, and parent_name its final name of f in B. Then the extended final name of f in C is: • If the Parent part for B in C contains a Rename_pair of the form **rename** parent_name as new_name : new_name. • Otherwise: extended_parent_name. The final names of all the features of a **class** constitute the final name set of a **class**.

Informative text

feature
feature

End

8.6.18 Definition: Inherited name

The inherited name of a **feature** obtained from a **feature** f of a parent B is the final name of f in B.

Informative text

feature

End

8.6.19 Definition: Declaration for a feature

A Feature_declaration in a **class** C, listing a Feature_name fn, is a declaration for a **feature** f if and only if fn is the final name of f in C.

Informative text

feature

End

8.7 Clients and exports

Informative text

End

8.7.1 Definition: Client relation between classes and types

A **class** C is a client of a type S if some ancestor of C is a simple client, an **expanded** client or a generic client of S.

Informative text

8.7.2 Definition: Client relation between classes

A **class** C is a client of a **class** B if and only if C is a client of a type whose base **class** is B. The same convention applies to the simple client, **expanded** client and generic client relations.

8.7.3 Definition: Supplier

A type or **class** S is a supplier of a **class** C if C is a client of S, with corresponding variants: simple, **expanded**, generic, indirect.

8.7.4 Definition: Simple client

A **class** C is a simple client of a type S if, in C, S is the type of some entity or expression or the Explicit_creation_type of a Creation_instruction, or is one of the Constraining_types of a formal generic parameter of C, or is involved in the Type of a Non_object_call or of a Manifest_type.

Informative text

class
class

End

8.7.5 Definition: Expanded client

A **class** C is an **expanded** client of a type S if S is an **expanded** type and some attribute of C is of type S.

8.7.6 Definition: Generic client, generic supplier

A **class** C is a generic client of a type S if for some generically derived type T of the formB […, S, …] one of the following holds:

1 C is a client of T.

2 T is a parent type of an ancestor of C.

Informative text

2 covers C inheriting directly or indirectly (remember that C is one of its own

ancestors) from B [S].

End

8.7.7 Definition: Indirect client

A **class** A is an indirect client of a type S of base **class** B if there is a sequence of classes C 1

A, C 2 , …, C n = B such that n > 2 and every C iisa client of C i+1 for 1 ≤ i < n. The indirect forms of the simple client, **expanded** client and generic client relations are defined similarly.

8.7.8 Definition: Client set of a Clients part

The client set of a Clients part is the set of descendants of every **class** of the universe whose name it lists. By convention, the client set of an absent Clients part includes all classes of the system.

Informative text

class
class
class
class

This important convention is in line with the reuse focus of Eiffel and its application to component-based development. You may develop a **class** C in a certain system, where it lists some **class** S in a Clients part, to give S access to some of its features; then you reuse C in another system that does not include S. You should not have to change C since no bad consequence can result from listing a **class** not present in the system, as long as C does not itself use S as its supplier or ancestor. Even in a single system, this policy means that you can remove S — if you find it is no longer needed — without causing compilation errors in the classes that listit in their Clients parts. With a stricter rule, you would have to remove S from every such Clients part. But then if you later change your mind — as part of the normal hesitations of an incremental design process — you would have to put it back in each of these places. This process is tedious, and it wouldn't take many iterations **until** programmers start making many features public just in case — hardly an improvement for information hiding, the purpose of all this.

End

8.7.9 Syntax: Clients

Clients

≙ "{" Class_list "}" Class_list

≙ {Class_name "," …} +

Informative text

class
not
class

End

8.7.10 Syntax: Export adaptation

New_exports

≙ **export** New_export_list New_export_list

≙ {New_export_item ";" …} + New_export_item

≙ Clients [Header_comment] Feature_set Feature_set

≙ Feature_list | **all** Feature_list

≙ {Feature_name "," …} +

8.7.11 Validity: Export List rule

Validity code: VLEL A New_exports clause appearing in class C in a Parent part for a parent B, of the form export {class_list 1 } feature_set 1 … {class_list n } feature_set n is valid if and only if for every feature_set i (for i in the interval 1..n) that is a Feature_list (rather than all):

1 Every element of the list is the final name of a feature of C inherited from B.

2 No feature name appears more than once in any such list.

Informative text

The following definitions and rules express these properties. They start by extending the notion of “client set” from entire Clients parts to individual features.

End

8.7.12 Definition: Client set of a feature

The client set of a feature f of a class C, of final name f name, includes the following classes (for all cases that match):

1 If f is introduced or redeclared in C: the client set of the Feature_clause of the declaration

for f in C.

2 If f is inherited: the union of the client sets (recursively) of all its precursors from

conforming parents.

3 If the Feature_set of one or more New_exports clauses of C includes f name or all, the

union of the client sets of their Clients parts.

Informative text

export
feature
feature

1. is also “inherited” (case 2) and the applicable Parent part may have a New_exports (case 3). • As a result of case 2, the client set can never diminish under conforming inheritance: features can win new clients, but never lose one. This is necessary under polymorphism and dynamic binding to avoid certain type of “catcalls” leading to run-time crashes.

   End

8.7.13 Definition: Available for call, available

A feature f is available for call, or just available for short, to a class C or to a type based on C, if and only if C belongs to the client set of f.

Informative text

export

End

8.7.14 Definition: Exported, selectively available, secret

The export status of a feature of a class is one of the following:

1 The **feature** may be available to all classes. It is said to be exported, or generally

available.

2 The **feature** may be available to specific classes (other than NONE and ANY) only. In

that case it is also available to the descendants of all these classes. Such a **feature** is said to be selectively available to the given classes and their descendants.

3 Otherwise the **feature** is available only to NONE. It is then said to be secret.

Informative text

• A **feature** introduced by **feature** {NONE} (case 3) is available to no useful classes. • A **feature** introduced by **feature** {ANY}, or just **feature**, is available to all classes and so will be considered to fall under case 1. • A **feature** introduced by **feature** {A, B, C}, where none of {A, B, C} is ANY, falls under case 2. A **feature** available to a **class** is also available to all the proper descendants of that **class**. As a consequence, selective **export** does not restrict reuse as much as it may seem at first: while the features will only be available to certain classes, these may be classes written much later, as long as they are descendants of one of the listed Clients.

End

8.7.15 Definition: Secret, public property

A property of a **class** text is secret if and only if it involves any of the following, describing information on which client classes cannot rely to establish their correctness:

1 Any **feature** that is not available to the given client, unless this is overridden by the next

case.

2 Any **feature** that is not available for creation to the given client, unless this is overridden

by the previous case.

3 The body and **rescue** clause of any **feature**, except for the information that the **feature**

is external or Once and, in the last case, its once keys if any.

4 For a query without formal arguments, whether it is implemented as an attribute or a

function, except for the information that it is a constant attribute.

5 Any Assertion_clause that (recursively) includes secret information.

6 Any parent part for a non-conforming parent (and as a consequence the very presence

of that parent).

7 The information that a **feature** is **frozen**.

Any property of a **class** text that is not secret is public.

Informative text

class
feature
feature
feature

End

8.7.16 Definition: Incremental contract view, short form

The incremental contract view of a **class**, also called its short form, is a text with the same structure as the **class** but retaining only public properties.

Informative text

class
class

End

8.7.17 Definition: Contract view, flat-short form

The contract view of a **class**, also called its flat-short form, is a text following the same conventions as the incremental contract view form but extended to include information about inherited as well as immediate features, the resulting combined preconditions and postconditions and the unfolded form of the **class invariant** including inherited clauses.

Informative text

class
require
ensure
class
invariant
require

End

8.8 Routines

Informative text

End

8.8.1 Definition: Formal argument, actual argument

Entities declared in a routine to represent information passed by callers are the routine's formal arguments . The corresponding expressions in a particular call to the routine are the call's actual arguments.

Informative text

convert

A noteon terminology: Eiffel always uses the term argument to refer to the arguments of a routine. The word "parameter" is never used in this context, because it could create confusion with the types that can parameterize classes, called generic parameters.

End

8.8.2 Syntax: Formal argument and entity declarations

Formal_arguments

≙ "(" Entity_declaration_list ")" Entity_declaration_list

≜ {Entity_declaration_group ";" …} + Entity_declaration_group

≜ Identifier_list Type_mark Identifier_list

≜ {Identifier "," …} +

Informative text

from
as
as
local

End

8.8.3 Validity: Formal Argument rule

Validity code: **VRFA** Let fa be the Formal_arguments part of a routine r in a **class** C. Let formals be the concatenation of every Identifier_list of every Entity_declaration_group in fa. Then fa is valid if and only if no Identifier e appearing in formals is the final name of a **feature** of C.

Informative text

feature
feature

End

8.8.4 Validity: Entity Declaration rule

Validity code: **VRED** Let el be an Entity_declaration_list. Let identifiers be the concatenation of every Identifier_list of every Entity_declaration_group in el. Then el is valid if and only if no Identifier appears more than once in the list identifiers.

8.8.5 Syntax: Routinebodies

Deferred

≜ **deferred** Effective_routine

≜ Internal | External Internal

≜ Routine_mark Compound Routine_mark

≜ **do** | Once Once

≜ **once** ["("Key_list ")"] Key_list

≜ {Manifest_string "," …} +

8.8.6 Definition: Once routine, once procedure, once function

A once routine is an Internal routine r with a Routine_mark of the Once form. If r is a procedure it is also a once procedure; if r is a function, it is also a once function.

8.8.7 Syntax: Local variable declarations

Local_declarations

≙ **local** [Entity_declaration_list]

8.8.8 Validity: Local Variable rule

Validity code: **VRLV** Let ld be the Local_declarations part of a routine r in a **class** C. Let locals be the concatenation of every Identifier_list of every Entity_declaration_group in ld. Then ld is valid if and only if every Identifier e in locals satisfies the following conditions:

1 No **feature** of C has e as its final name.

2 No formal argument of r has e as its Identifier.

Informative text

End

8.8.9 Definition: Local variable

The local variables of a routine include all entities declared in its Local_declarations part, if any, and, if it is a query, the predefined entity **Result**.

Informative text

Result
Result

End

8.8.10 Syntax: Instructions

Compound

≙ {Instruction ";" …}* Instruction

≙ Creation_instruction | Call | Assignment | Assigner_call | Conditional | Multi_branch

```
| Loop | Debug | Precursor | Check | Retry
```

Informative text

Retry is used in conjunction with the exception handling mechanism.

End

8.9 Correctness and contracts

Informative text

End

8.9.1 Syntax: Assertions

Precondition

require [else] Assertion Postcondition

ensure [then] Assertion [Only] Invariant

invariant Assertion Assertion

{Assertion_clause ";" …}* Assertion_clause

[Tag_mark] Unlabeled_assertion_clause Unlabeled_assertion_clause

Boolean_expression | Comment Tag_mark

Tag ":" Tag

Identifier

8.9.2 Syntax (non-production): Assertion Syntax rule

An Assertion without a Tag_mark may not begin with any of the following:

1 An opening parenthesis "(".

2 An opening bracket "[".

3 A non-keyword Unary operator that is also Binary.

Informative text

1. and also (case 3) if what follows a is –b, which could express a subtraction from a in the same clause, or start a new clause about the negated value of b. The Assertion Syntax rule avoids this. The rule does significantly restrict expressiveness, since violations are rare and will be flagged clearly in reference to the rule, and it is recommended practice anyway to use a Tag_mark, which removes any ambiguity.

End

8.9.3 Definition: Precondition, postcondition, invariant

The precondition and postcondition of a **feature**, or the **invariant** of a **class**, is the Assertion of, respectively, the corresponding Precondition, Postcondition or Invariant clauseif present and non-empty, and otherwise the assertion **True**.

Informative text

End

8.9.4 Definition: Contract, subcontract

Let pre and post be the precondition and postcondition of a **feature** f. The contract of f is the pair of assertions [pre, post]. A contract [pre', post'] is said to be a subcontract of [pre, post] if and only if pre **implies** pre' and post' **implies** post.

8.9.5 Validity: Precondition Export rule

Validity code: **VAPE** A Precondition of a **feature** r of a **class** S is valid if and only if every **feature** f appearing in every Assertion_clause of its unfolded formu satisfies the following two conditions for every **class** C to which r is available:

1 If f appears as **feature** of a call in u or any of its subexpressions, f is available to C.

2 If u or any of its subexpressions uses f as creation procedure of a Creation_expression,

f is available for creation to C.

Informative text

class
feature
feature
feature
feature
feature
feature

End

8.9.6 Definition: Availability of an assertion clause

An Assertion_clause aofa routine Precondition or Postcondition is available to a **class** B if and only if all the features involved in the Equivalent Dot Form of a are available to B.

Informative text

class

End

8.9.7 Syntax: “Old” postcondition expressions

Old

≙ **old** Expression

8.9.8 Validity: Old Expression rule

Validity code: **VAOX** An Old expression oe of the form old e is valid if and only if it satisfies the following conditions:

1 It appears in a Postcondition part post of a **feature**.

2 It does not involve **Result**.

3 Replacing oe by e in post yields a valid Postcondition.

Informative text

3 simply states that old e is valid in a postcondition

if e itself is. The expression e may not, for example, involve any local variables (although it might include **Result** were it not for condition 2), but may refer to features of the **class** and formal arguments of the routine.

End

8.9.9 Semantics: Old Expression Semantics, associated variable, associated exception marker

The effect of including an Old expression oe in a Postcondition of an effective **feature** f is equivalent to replacing the semantics of its Feature_body by the effect of a call to a fictitious routine possessing a local variable av, called the

associated variable of oe, and semantics defined by the following succession of steps:

1 Evaluate oe.

2 If this evaluation triggers an exception, record this event in an associated exception

marker for oe.

3 Otherwise, **assign** the value of oe to av.

4 Proceed with the original semantics.

Informative text

End

8.9.10 Semantics: Associated Variable Semantics

As part of the evaluation of a postcondition clause, the evaluation of the associated variable of an Old expression:

1 Triggers an exception of type OLD_EXCEPTION if an associated exception marker has

been recorded.

2 Otherwise, yields the value to which the variable has been set.

8.9.11 Syntax: “Only” postcondition clauses

Only

≙ **only** [Feature_list]

Informative text

only
feature
as
feature
as
not
do
old
not
only
all
some
not
feature
not
feature
not
only
feature
as
class
all
then

as

End

8.9.12 Validity: Only Clause rule

Validity code: **VAON** An Only clause appearing in a Postcondition of a **feature** of a **class** C is valid if and only if every Feature_name qn appearing its Feature_list if any satisfies the following conditions:

1 There is no other occurrence of qn in that Feature_list.

2 qn is the final name of a query q of C, with no arguments.

3 If C redeclares f from a parent B, q is not a **feature** of B.

Informative text

End

8.9.13 Definition: Unfolded feature list of an Only clause

The unfolded **feature** list of an Only clause appearing in a Postcondition of a **feature** f in a **class** C is the Feature_list containing:

1 All the **feature** names appearing in its Feature_list if any.

2 If f is the redeclaration of one or more features, the final names in C of all the features

whose names appear (recursively) in their unfolded Only clauses.

Informative text

End

8.9.14 Definition: Unfolded Only clause

The unfolded Only clause of a **feature** f of a **class** C is a sequence of Assertion_clause components of the following form, one for every argument-less query q of C that does not appear in the unfolded **feature** list of the Only clause of its Postcondition if any: q = (old q)

Informative text

feature
feature
attached

End

8.9.15 Definition: Hoare triple notation (total correctness)

In definitions of correctness notions for Eiffel constructs, the notation {P} A {Q} (a mathematical convention, not a part of Eiffel) expresses that any execution of the Instruction or Compound A started in a state of the computation satisfying the assertion P will terminate in a state satisfying the assertion Q.

8.9.16 Semantics: Class consistency

A **class** C is consistent if and only if it satisfies the following conditions:

1 For every creation procedure p of C:

{pre p } do p {INV C and then post p }

2 For every **feature** f of C exported generally or selectively:

{INV C and then pre f } do f {INV C and then post f } where INV C is the **invariant** of C and, for any **feature** f, pre f is the unfolded form of the precondition of f, post f the unfolded form of its postcondition, and do f its body.

Informative text

End

8.9.17 Syntax: Check instructions

Check

check Assertion [Notes] **end**

Informative text

or

End

8.9.18 Definition: Check-correct

An effective routine r is check-correct if, for every Check instruction cinr, any execution of c (as part of an execution of r) satisfies its Assertion.

8.9.19 Syntax: Variants

Variant

variant [Tag_mark] Expression

8.9.20 Validity: Variant Expression rule

Validity code: **VAVE** A Variant is valid if and only if its **variant** expression is of type INTEGER or one of its sized variants.

8.9.21 Definition: Loopinvariant and variant

The Assertion introduced by the Invariant clause of a **loop** is called its **loop invariant**. The Expression introduced by the Variant clause is called its **loop variant**.

8.9.22 Definition: Loop-correct

A routine is **loop**-correct if every loopit contains, with **loop invariant** INV, **loop variant** **VAR**, Initialization INIT, Exit condition EXIT and body (Compound part of the Loop_body) BODY, satisfies the following conditions:

1 {**True**} INIT {INV}

2 {**True**} INIT {**VAR** ≥ 0}

3 {INV and then not EXIT} BODY {INV}

4 {INV and then not EXIT and then (**VAR** = v)} BODY {0 ≤ **VAR** < v}

Informative text

3 and 4 express that the body, when executed in a

state where the **invariant** is satisfied but not the exit condition, will preserve the **invariant** and decrease the **variant**, while keeping it non-negative. (v is an auxiliary variable used to refer to the value of **VAR** before BODY's execution.)

End

8.9.23 Definition: Correctness (class)

A **class** is correct if and only if it is consistent and every routine of the **class** is check-correct, **loop**-correct and exception-correct.

8.9.24 Definition: Local unfolded form of an assertion

The local unfolded form of an assertion a — a Boolean_expression — is the Equivalent Dot Form of the expression that would be obtained by applying the following transformations to a in order:

1 Replace any Only clauseby the corresponding unfolded Only clause.

2 Replace any Old expression by its associated variable.

3 Replace any clause of the Comment formby **True**.

Informative text

End

8.9.25 Semantics: Evaluation of an assertion

To evaluate an assertion consists of computing the value of its unfolded form.

Informative text

End

8.9.26 Semantics: Assertion monitoring

The execution of an Eiffel system may evaluate, or monitor, specific kinds of assertion, and **loop** variants, at specific stages:

1 Precondition of a routine r: on starting a call to r, after argument evaluation and prior to

executing any of the instructions in r's body.

2 Postcondition of a routine r: on successful (not interrupted by an exception) completion

of a call to r, after executing any applicable instructions of r.

3 Invariant of a **class** C: on both start and termination of a qualified call to a routine of C.

4 Invariant of a **loop**: after execution of the Initialization, and after every execution (if any)

of the Loop_body.

5 Assertion in a Check instruction: on any execution of that instruction.

6 Variant of a **loop**: as with the **loop invariant**.

8.9.27 Semantics: Assertion violation

An assertion violation is the occurrence at run time, as a result of assertion monitoring, of any of the following: • An assertion (in the strict sense of the term) evaluating to false. • A **loop variant** found to be negative.

• A **loop variant** found, after the execution of a Loop_body, to be no less than in its previous evaluation.

Informative text

variant

End

8.9.28 Semantics: Assertion semantics

In the absence of assertion violations, assertions have no effect on system execution other than through their evaluation as a result of assertion monitoring. An assertion violation causes an exception of type ASSERTION_VIOLATION or one of its descendants.

8.9.29 Semantics: Assertion monitoring levels

An Eiffel implementation must provide facilities to enable or disable assertion monitoring according to some combinations of the following criteria: • Statically (at compile time) or dynamically (at run time). • Through control information specified within the Eiffel text or through outside elements such as a user interface or configuration files. • For specific kinds as listed in the definition of assertion monitoring: routine preconditions, routine postconditions, **class** invariants, **loop** invariants, Check instructions, **loop** variants. • For specific classes, specific clusters, or the entire system. The following combinations must be supported:

1 Statically disable all monitoring for the entire system.

2 Statically enable precondition monitoring for an entire system.

3 Statically enable precondition monitoring for specified classes.

4 Statically enable all assertion monitoring for an entire system.

8.10 Feature adaptation

Informative text

End

8.10.1 Definition: Redeclare, redeclaration

A **class** redeclares an inherited **feature** if it redefines or effects it. A declaration for a **feature** f is a redeclaration of f if it is either a redefinition or an effecting of f.

Informative text

feature
deferred
feature
feature

End

8.10.2 Definition: Unfolded form of an assertion

The unfolded form of an assertion a of local unfolded formua in a **class** C is the following Boolean_expression:

1 If a is the **invariant** of C and C has n parents for some n ≥ 1: up

1 and … and up n and then ua , where up 1 , … up n are (recursively) the unfolded forms of the invariants of these parents, after application of any **feature** renaming specified by C’s corresponding Parent clauses.

2 If a is the precondition of a redeclared **feature** f: the combined precondition for a.

3 If a is the postcondition of a redeclared **feature** f: the combined postcondition for a.

4 In all other cases: ua.

Informative text

1 can mention “the invariants of” the parents of C.

End

8.10.3 Definition: Assertion extensions

The Assertion of a Precondition starting with **require** else is a precondition extension. The Assertion of a Postcondition starting with **ensure** then is a postcondition extension.

Informative text

require
ensure

End

8.10.4 Definition: Covariance-aware form of an inherited assertion

The covariance-aware form of an inherited assertion a is:

1 If the enclosing routine has one or more arguments x

1 , … x n redefined covariantly to types U 1 , … U n : the assertion ({y 1 : U 1 } x 1 and … and {y n : U n } x n) and then a’ where y 1 , … y n are fresh names and a’ is the result of substituting y i for each corresponding x iina.

2 Otherwise: a.

Informative text

End

8.10.5 Definition: Combined precondition, postcondition

Consider a **feature** f redeclared in a **class** C. Let f 1 , … f n (n ≥ 1) be its versions in parents, pre 1 , … pre n the covariance-aware forms of (recursively) the combined preconditions of these versions, and post 1 , … post n the covariance-aware forms of (recursively) their combined postconditions. Let pre be the precondition extension of f if defined and not empty, otherwise **False**. Let post be the postcondition extension of f if defined and not empty, otherwise **True**. The combined precondition of f is the Assertion (pre 1 or… or pre n) or else pre The combined postcondition of f is the Assertion (old pre 1 **implies** post 1) and … and (old pre n **implies** post n) and then post

Informative text

End

8.10.6 Definition: Inherited as effective, inherited as deferred

An inherited **feature** is inherited as effective if it has at least one precursor that is an effective **feature**, and the corresponding Parent part does not **undefine** it. Otherwise the **feature** is inherited as **deferred**.

8.10.7 Definition: Effect, effecting

A **class** effects an inherited **feature** f if and only if it inherits f as **deferred** and contains a declaration for f that defines an effective **feature**.

Informative text

feature
feature
deferred
redefine
deferred

End

8.10.8 Definition: Redefine, redefinition

A **class** redefines an inherited **feature** f if and only if it contains a declaration for f that is not an effecting of f. Such a declaration is then known as a redefinition of f

Informative text

feature
redefine
class
deferred
deferred
feature
feature
deferred

As defined earlier, the two cases, effecting and redefinition, are together called redeclaration.

End

8.10.9 Definition: Nameclash

A **class** has a name clash if it inherits two or more features from different parents under the same final name.

Informative text

class
class
feature

End

8.10.10 Syntax: Precursor

Precursor

Precursor [Parent_qualification] [Actuals] Parent_qualification

"{" Class_name "}"

8.10.11 Definition: Relative unfolded form of a Precursor

In a **class** C, consider a **Precursor** specimen p appearing in the redefinition of a routine r inherited from a parent **class** B. Its unfolded form relative to B is an Unqualified_call of the formr' if p has no Actuals, or r' (args) if p has actual arguments args, where r' is a fictitious **feature** name added, with a **frozen** mark, as synonym for r in B.

8.10.12 Validity: Precursor rule

Validity code: **VDPR** A **Precursor** is valid if and only if it satisfies the following conditions:

1 It appears in the Feature_body of a Feature_declaration of a **feature** f.

2 If the Parent_qualification part is present, its Class_name is the name of a parent **class**

P of C.

3 Among the features of C's parents, limited to features of P if condition 2 applies, exactly

one is an effective **feature** redefined by C into f. (The **class** to which this **feature** belongs is called the applicable parent of the **Precursor**.)

4 The unfolded form relative to the applicable parent is, as an Unqualified_call,

argument-valid. In addition:

5 It is valid as an Instruction if and only if f is a command, and as an Expression if and only

if f is a query.

Informative text

4 relies on this notion.

Condition 1 states that the **Precursor** construct is only valid in a routine redefinition. In general the language definition treats functions and attributes equally (Uniform Access principle), but here an attribute would not be permissible, even with an Attribute body. Because of our interpretation of a multiple declaration as a set of **separate** declarations, this means that if **Precursor** appears in the body of a multiple declaration it applies separately to every **feature** being redeclared. This is an unlikely case, and this rule makes it unlikely to be valid.

Condition 2 states that if you include a **class** name, as in **Precursor** {B}, then B must be the name of one of the parents of the current **class**. The following condition makes this qualified form compulsory in case of potential ambiguity, but even in the absence of ambiguity you may use it to state the parent explicitly if you think this improves readability. Condition 3 specifies when this explicit parent qualification is required. This is whenever an ambiguity could arise because the redefinition applies to more than one effective parent version. The phrasing takes care of all the cases in which this could happen, for example as a result of a join. Condition 4 simply expresses that we understand the **Precursor** specimen as a call to a **frozen** version of the original routine; we must make sure that such a call would be valid, more precisely "argument-valid", the requirement applicable to such an Unqualified_call. A **Precursor** will be used as either an Instruction or an Expression, in the same way as a call to (respectively) a procedure or a function; indeed **Precursor** appears as one of the syntax variants for both of these constructs. So in addition to being valid on its own, it must be valid in the appropriate role. Condition

5 takes care of this.

End

8.10.13 Definition: Unfolded form of a Precursor

The unfolded form (absolute) of a valid **Precursor** is its unfolded form relative to its applicable parent.

8.10.14 Semantics: Precursor semantics

The effect of a **Precursor** is the effect of its unfolded form.

8.10.15 Syntax: Redefinition

Redefine

redefine Feature_list

8.10.16 Validity: Redefine Subclause rule

Validity code: VDRS A Redefine subclause appearing in a Parent part for a class B in a class C is valid if and only if every Feature_name f name that it lists (in its Feature_list) satisfies the following conditions:

1 f name is the final name of a feature f of B.

2 f was not frozen in B, and was not a constant attribute.

3 f name appears only once in the Feature_list.

4 The Features part of C contains one Feature_declaration that is a redeclaration but not

an effecting of f.

5 If that redeclaration specifies a deferred feature, C inherits f as deferred.

8.10.17 Semantics: Redefinition semantics

The effect in a class C of redefining a feature f in a Parent part for A is that the version of f in C is, rather than its version in A, the feature described by the applicable declaration in C.

Informative text

feature
class

End

8.10.18 Syntax: Undefine clauses

Undefine

undefine Feature_list

8.10.19 Validity: Undefine Subclause rule

Validity code: VDUS An Undefine subclause appearing in a Parent part for a class B in a class C is valid if and only if every Feature_name f name that it lists (in its Feature_list) satisfies the following conditions:

1 f name is the final name of a feature f of B.

2 f was not frozen in B, and was not an attribute.

3 f was effective in B.

4 f name appears only once in the Feature_list.

5 Any redeclaration of f in C specifies a deferred feature.

8.10.20 Semantics: Undefinition semantics

The effect in a class C of undefining a feature f in a Parent part for A is to cause C to inherit from A, rather than the version of f in A, a deferred form of that version.

8.10.21 Definition: Effective, deferred feature

A **feature** f of a **class** C is an effective **feature** of C if and only if it satisfies either of the following conditions:

1 C contains a declaration for f whose Feature_body is not of the Deferred form.

2 f is an inherited **feature**, coming from a parent B of C where it is (recursively) effective,

and C does not **undefine** it. f is **deferred** if and only if it is not effective.

Informative text

feature
deferred
deferred
deferred
feature
deferred
deferred
feature
deferred

2 the **feature** is inherited; it was already effective in the parent, and C doesn't

change that status.

End

8.10.22 Definition: Effecting

A redeclaration into an effective **feature** of a **feature** inherited as **deferred** is said to effect that **feature**.

8.10.23 Deferred Class property

A **class** that has at least one **deferred feature** must have a Class_header starting with the keyword **deferred**. The **class** is then said to be **deferred**.

8.10.24 Effective Class property

A **class** whose features, if any, are all effective, is effective unless its Class_header starts with the keyword **deferred**.

Informative text

class
deferred
deferred
class
class
class
deferred

End

8.10.25 Definition: Origin, seed

Every **feature** f of a **class** C has one or more features known as its seeds and one or more classes known as its origins, as follows:

1 If f is immediate in C: f itself as seed; C as a origin.

2 If f is inherited: (recursively) all the seeds and origins of its precursors.

Informative text

2 is about.

End

8.10.26 Validity: Redeclaration rule

Validity code: VDRD Let C be a **class** and g a **feature** of C. It is valid for g to be a redeclaration of a **feature** f inherited from a parent B of C if and only if the following conditions are satisfied.

1 No effective **feature** of C other than f and g has the same final name.

2 The signature of g conforms to the signature of f.

3 The Precondition of g, if any, begins with **require** else (not just **require**), and its

Postcondition, if any, begins with **ensure** then (not just **ensure**).

4 If the redeclaration is a redefinition (rather than an effecting) the Redefine subclause of

the Parent part for B lists in its Feature_list the final name of f in B.

5 If f is inherited as effective, then g is also effective.

6 If f is an attribute, g is an attribute, f and g are both variable, and their types are either

both **expanded** or both non-**expanded**.

7 f and g have either both no alias or the same alias.

8 If both features are queries with associated assigner commands fp and gp, then gp is

the version of fp in C.

Informative text

feature
class
deferred
deferred
deferred
feature
deferred
detachable
feature
deferred
deferred
feature
deferred
feature
deferred

Condition 6 prohibits redeclaring a constant attribute, or redeclaring a variable attribute into a function or constant attribute. It also precludes redeclaring a (variable) attribute of an **expanded** type into one of reference type or conversely. You may, however, redeclare a function into an attribute — variable or constant. Condition 7 requires the features, if they have aliases, to have the same ones. If you want to introduce an alias for an inherited **feature**, change an inherited alias, or remove it, redeclaration is not the appropriate technique: you must **rename** the **feature**. Of course you can still redeclare it as well. Condition 8 applies to assigner commands. It is valid for a redeclaration to include an

assigner command if the precursor did not include one, or conversely; but if both versions of the query have assigner commands, they must, for obvious reasons of consistency, be the same procedure in C.

End

8.10.27 Definition: Precursor (joined features)

A precursor of an inherited **feature** is a version of the **feature** in the parent from which it is inherited.

8.10.28 Validity: Join rule

Validity code: `VDJR` It is valid for a **class** C to **inherit** two different features under the same final name under and only under the following conditions:

1 After possible redeclaration in C, their signatures are identical.

2 They either have both no aliases or have the same alias.

3 If they both have assigner commands, the associated procedures have the same final

name in C.

4 If both are inherited as effective, C redefines both into a common version.

Informative text

2 of the Redeclaration rule) must have a signature conforming to all of the

precursors' signatures. If the redeclaration describes an effective **feature**, this is the case of both joining and effecting a set of inherited features. If the redeclaration describes a **feature** that is still **deferred**, it is a redefinition, used to adapt the signature and possibly the specification. In this case, point

4 of the

Redeclaration rule requires every one of the precursors to appear in the Redefine subclause for the corresponding parent. In either case, nothing requires the precursors' signatures to conform to each other, as long as the signature of the redeclared version conforms to all of them. This means you may write a **class** inheriting two **deferred** features of the form f (p: P): T … f (t: Q): U … and redeclare them with f (x: ? R): V … provided R conforms to both P and Q and V to both T and U. No conformance is required between the types appearing in the precursors' signatures (P and Q, T and U). The assumption that the features are "different" is important: they could in fact be the same **feature**, appearing in two parents of C that have inherited it from a common ancestor, without any intervening redeclaration. This would be a valid case of repeated inheritance; here the rule that

determines validity is the Repeated Inheritance Consistency constraint. The semantic specification (sharing under the Repeated Inheritance rule) indicates that C will have just one version of the **feature**. Conditions 2 and 3 of the Join rule are consistency requirements on aliases and on assigner commands. The condition on aliases is consistent with condition

7 of the Redeclaration rule, which

requires a redeclaration to keep the alias if any; it was noted in the comment to that rule that redeclaration is not the appropriate way to add, change or remove an alias (you should use renaming for that purpose); neither is join. The condition on assigner commands ensures that any Assigner_call has the expected effect, even under dynamic binding on a target declared of a parent type.

End

8.10.29 Semantics: Join Semantics rule

Joining two or more inherited features with the same final name, under the terms of the Join rule, yields the **feature** resulting from their redeclaration if any, and otherwise defined as follows:

1 Its name is the final name of all its precursors.

2 Its signature is the precursors' signature, which the Join rule requires to be the same for

all precursors after possible redeclaration.

3 Its precondition is the or of all the precursors' combined preconditions.

4 Its postcondition is the and of all the precursors' combined postconditions.

5 Its Header_comment is the concatenation of those of all precursors.

6 Its body is **deferred** if all the precursors are inherited as **deferred**, otherwise is the body

of the single effective precursor. It is not **obsolete** (even if some of the precursors are **obsolete**).

Informative text

class
feature
obsolete

End

8.11 Types

Informative text

End

8.11.1 Syntax: Types

Type

Class_or_tuple_type | Formal_generic_name | Anchored Class_or_tuple_type

Class_type | Tuple_type Class_type

[Attachment_mark] Class_name [Actual_generics] Attachment_mark

"?" | "!" Anchored

[Attachment_mark] **like** Anchor Anchor

Feature_name | **Current**

Informative text

End

8.11.2 Semantics: Direct instances and values of a type

The direct instances of a type T are the run-time objects resulting from: representing a manifest constant, manifest tuple, Manifest_type, agent or Address expression of type T; applying a creation operation to a target of type T; (recursively)

cloning an existing direct instance of T. The values of a type T are the possible run-time values of an entity or expression of type T.

8.11.3 Semantics: Instances of a type

The instances of a type TX are the direct instances of any type conforming to TX.

Informative text

End

8.11.4 Semantics: Instance principle

Any value of a type T is: • If T is reference, either a reference to an instance of T or (unless T is **attached**) a void reference. • If T is **expanded**, an instance of T.

8.11.5 Definition: Instances, direct instances of a class

An instance of a **class** C is an instance of any type T based on C. A direct instance of C is a direct instance of any type T based on C.

Informative text

class
class

End

8.11.6 Base principle; base class, base type, based

Any type T proceeds, directly or indirectly, from a Class_or_tuple_type called its base type, and an underlying **class** called its base **class**. The base **class** of a type is also the base **class** of its base type. A type is said to be based on its base type, and also on its base **class**.

Informative text

class

The base **class** is the **class** providing the features applicable to instances of the type. If T is a Class_type the connection to a **class** is direct: T is either the name of a non-generic **class**, such as PARAGRAPH, or the name of a generic **class** followed by Actual_generics, such as LIST [WINDOW] . In both cases the base **class** of T is the **class** whose name is used to obtain T, with any Actual_generics removed: PARAGRAPH and LIST in the examples. For a Tuple_type, the base **class** is a fictitious **class** TUPLE, providing the features applicable to all tuples. For types not immediately obtained from a **class** we obtain the base **class** by going through base type: for example T is an Anchored type of the form like anchor, and anchor is of type LIST [WINDOW] , then the base **class** of that type, LIST, is also the base **class** of T.

End

8.11.7 Base rule

The base type of any type is a Class_or_tuple_type, with no Attachment_mark. The base **class** of any type other than a Class_or_tuple_type is (recursively) the base **class** of its base type. The direct instances of a type are those of its base type.

Informative text

class

class

End

8.11.8 Validity: Class Type rule

Validity code: VTCT A Class_type is valid if and only if it satisfies the following two conditions:

1 Its Class_name is the name of a **class** in the surrounding universe.

2 If it has a “?” Attachment_mark, that **class** is not **expanded**.

Informative text

End

8.11.9 Semantics: Type Semantics rule

To define the semantics of a type T it suffices to specify:

1 Whether T is **expanded** or reference.

2 Whether T, if reference, is **attached** or **detachable**.

3 What is T’s base type.

4 If T is a Class_or_tuple_type, what are its base **class** and its type parameters if any.

8.11.10 Definition: Base class and base type of an expression

Any expression e has a base type and a base **class**, defined as the base type and base **class** of the type of e.

8.11.11 Semantics: Non-generic class type semantics

A non-generic **class** C used as a type (of the Class_type category) has the same expansion status as C (i.e. it is **expanded** if C is an **expanded class**, reference otherwise). It is its own base type (after removal of any Attachment_mark) and base **class**.

8.11.12 Definition: Expanded type, reference type

A type T is **expanded** if and only if it is not a Formal_generic_name and the base **class** of its deanchored form is an **expanded class**.

T is a reference typeif it is neither a Formal_generic_name nor **expanded**.

Informative text

expanded
expanded

End

8.11.13 Definition: Basic type

The basic types are BOOLEAN, CHARACTER and its sized variants, INTEGER and its sized variants, REAL and its sized variants and POINTER.

Informative text

class
class

End

8.11.14 Definition: Anchor, anchored type, anchored entity

The anchor of an anchored type like anchor is the entity anchor. A declaration of an entity with such a type is an anchored declaration, and the entity itself is an anchored entity.

Informative text

Current
feature
class

Informative text

End

8.11.15 Definition: Anchor set; cyclic anchor

The anchor set of a type T is the set of entities containing, for every anchored type like anchor involved in T: • anchor. • (Recursively) the anchor set of the type of anchor. An entity a of type T is a cyclic anchor if the anchor set of T includes a itself.

Informative text

Because of genericity, the cycles that make an anchor “cyclic” might occur not directly through the anchors but through the types they involve, as with a of type LIST [like b] where b is of type like a. Here we say that a type “involves” all the types appearing in its definition, as captured by the following definition.

End

8.11.16 Definition: Types and classes involved in a type

The types involved in a type T are the following: • T itself. • If T is of the form a T’ where a is an Attachment_mark: (recursively) the types involved in T’. • If T is a generically derived Class_type or a Tuple_type: all the types (recursively) involved in any of its actual parameters. The classes involved in T are the base classes of the types involved in T.

Informative text

End

8.11.17 Definition: Deanchored form of a type

The deanchored form of a type T in a **class** C is the type (Class_or_tuple_type or Formal_generic) defined as follows:

1 If T is like **Current**: the current type of C.

2 If T is like anchor where the type AT of anchor is not anchored: (recursively) the

deanchored form of AT.

3 If T is like anchor where the type AT of anchor is anchored but anchor is not a cyclic

anchor: (recursively) the deanchored form of AT in C.

4 If T is a AT, where a is an Attachment_mark: a DT, where DT is (recursively) the

deanchored form of AT deprived of its Attachment_mark if any.

5 If none of the previous cases applies: T after replacement of any actual parameter by

(recursively) its deanchored form.

Informative text

detachable
attached
detachable

End

8.11.18 Validity: Anchored Type rule

Validity code: `VTAT` It is valid to use an anchored type AT of the form like anchor in a **class** C if and only if it satisfies the following conditions:

1 anchor is either **Current** or the final name of a query of C.

2 anchor is not a cyclic anchor.

3 The deanchored form UT of AT is valid in C.

The base **class** and base type of AT are those of UT.

Informative text

End

8.11.19 Definition: Attached, detachable

A type is **detachable** if its deanchored form is a Class_type declared with the ? Attachment_mark. A type is **attached** if it is not **detachable**.

Informative text

attached
detachable
attached
expanded
attached
attached

End

8.11.20 Semantics: Attached type semantics

Every run-time value of an **attached** type is non-void (**attached** to an object).

Informative text

detachable
attached
attached
detachable
attached
attached
attached

ensure
attached
attached
ensure
detachable
attached
class
class

End

8.11.21 Definition: Stand-alone type

A Type is stand-alone if and only if it involves neither any Anchored type nor any Formal_generic_name.

Informative text

End

8.12 Genericity

Informative text

End

8.12.1 Syntax: Actual generic parameters

Actual_generics

≙ "[" Type_list "]" Type_list

≙ {Type "," …} +

8.12.2 Syntax: Formal generic parameters

Formal_generics

≙ "[" Formal_generic_list "]" Formal_generic_list

≙ {Formal_generic ","…} + Formal_generic

≙ [**frozen**] Formal_generic_name [Constraint] Formal_generic_name

≙ [?] Identifier

8.12.3 Validity: Formal Generic rule

Validity code: `VCFG` A Formal_generics part of a Class_declaration is valid if and only if every Formal_generic_name G in its Formal_generic_list satisfies the following conditions:

1 G is different from the name of any **class** in the universe.

2 G is different from any other Formal_generic_name appearing in the same

Formal_generics part.

Informative text

frozen
frozen

Adding the ? mark to a Formal_generic_name, as in ? G, means that the **class** may declare self-initializing variables (variables that will be initialized automatically on first use) of type G; this requires that any actual generic parameter that is an **attached** type must also be self-initializing, that is to say, make default_create from ANY available for creation.

End

8.12.4 Definition: Generic class; constrained, unconstrained

Any **class** declared with a Formal_generics part (constrained or not) is a generic **class**. If a formal generic parameter of a generic **class** is declared with a Constraint, the parameter is constrained; if not, it is unconstrained. A generic **class** is itself constrained if it has at least one constrained parameter, unconstrained otherwise.

Informative text

class

End

8.12.5 Definition: Generic derivation, generic type, non-generic type

The process of producing a type from a generic **class** by providing actual generic parameters is generic derivation. A type resulting from a generic derivation is a generically derived type, or just generic type. A type that is not generically derived is a non-generic type.

Informative text

class

End

8.12.6 Definition: Self-initializing formal parameter

A Formal_generic_parameter is self-initializing if and only if its declaration includes the optional ? mark.

Informative text

attached

End

8.12.7 Definition: Constraint, constraining types of a Formal_generic

The constraint of a formal generic parameter is its Constraint partif present, and otherwise ANY. Its constraining types are all the types listed in its Constraining_types if present, and otherwise just ANY.

8.12.8 Syntax: Generic constraints

Constraint

≙ "–>" Constraining_types [Constraint_creators] Constraining_types

≙ Single_constraint | Multiple_constraint Single_constraint

≙ Type [Renaming] Renaming

≙ Rename **end**

Multiple_constraint

≙ "{" Constraint_list "}" Constraint_list

≙ {Single_constraint "," …} + Constraint_creators

≙ **create** Feature_list **end**

8.12.9 Validity: Generic Constraint rule

Validity code: VTGC A Constraint part appearing in the Formal_generics part of a **class** C is valid if and only if it satisfies the following conditions for every Single_constraint listing a type T in its Constraining_types:

1 T involves no anchored type.

2 If a Renaming clause **rename** rename_list end is present, a **class** definition of the form

class NEW **inherit** BT **rename** rename_list end (preceded by **deferred** if the base **class** of T is **deferred**), where BT is the base **class** of T, would be valid.

3 Every name listed in the Constraint_creators is the generically constrained name of a

procedure p of one of the corresponding constraining types.

Informative text

implies
separate
deferred
class
class
deferred
deferred
frozen
class
class
frozen

End

8.12.10 Definition: Constraining creation features

If G is a formal generic parameter of a **class**, the constraining creators of G are the features of G's Constraining_types, if any, corresponding after possible Renaming to the **feature** names listed in the Constraining_creators if present.

Informative text

End

8.12.11 Validity: Generic Derivation rule

Validity code: VTGD Let C be a generic **class**. A Class_type CT having C as base **class** is valid if and only if it satisfies the following conditions for every actual generic parameter T and every Single_constraint U appearing in the constraint for the corresponding formal generic parameter G:

1 The number of Type components in CT's Actual_generics list is the same as the number

of Formal_generic parameters in the Formal_generic_list of C's declaration.

2 T conforms to the type obtained by applying to U the generic substitution of CT.

3 If C is **expanded**, CT is generic-creation-ready.

4 If G is a self-initializing formal parameter and T is **attached**, then T is a self-initializing

type.

Informative text

End

8.12.12 Definition: Generic-creation-ready type

A type of base **class** C is generic-creation-ready if and only if every actual generic parameter T of its deanchored form satisfies the following conditions:

1 If the specification of the corresponding formal generic parameter includes a

Constraint_creators, the versions in T of the constraining creators for the corresponding formal parameter are creation procedures, available for creation to C, and T is (recursively) generic-creation-ready.

2 If T is **expanded**, it is (recursively) generic-creation-ready.

Informative text

3 of the Creation Expression rule.

• For a Parent in an Inheritance part: condition 6 of the Parent rule. • For an **expanded** type: condition 3 of the just seen Generic Derivation rule.

End

8.12.13 Semantics: Generically derived class type semantics

A generically derived Class_type of the formC […], where C is a generic **class**, is **expanded** if C is an **expanded class**, reference otherwise. It is its own base type, and its base **class** is C.

Informative text

class

End

8.12.14 Definition: Base type of a single-constrained formal generic

The base type of a constrained Formal_generic_name G having as its constraining types a Single_constraint listing a type T is:

1 If T is a Class_or_tuple_type: T.

2 Otherwise (T is a Formal_generic_name): the base type of T if it can be determined by

(recursively) case 1, otherwise ANY.

Informative text

2 to make sure that this definition is not cyclic, because we permit cases such as C [G,

H–> D [G]] , and as a consequence cases such as C [G –> H, H–> G] or even C [G –> G] even though they are not useful; both of these examples yield ANY as base types for the parameters. As a result of the definition of “constraining types”, the base type of an unconstrained formal generic, such as G in C [G], is also ANY.

End

8.12.15 Definition: Base type of an unconstrained formal generic

The base type of an unconstrained Formal_generic_name type is ANY.

8.12.16 Definition: Reference or expanded status of a formal generic

A Formal_generic_name represents a reference type or **expanded** type depending on the corresponding status of the associated actual generic parameter in a particular generic derivation.

8.12.17 Definition: Current type

Within a **class** text, the current type is the type obtained from the current **class** by providing as actual generic parameters, if required, the **class**’s own formal generic parameters.

Informative text

class
class

End

8.12.18 Definition: Features of a type

The features of a type are the features of its base **class**.

Informative text

class

End

8.12.19 Definition: Generic substitution

Every type T defines a mapping σ from names to types known as its generic substitution:

1 If T is generically derived, σ associates to every Formal_generic_name the

corresponding actual parameter.

2 Otherwise, σ is the identity substitution.

8.12.20 Generic Type Adaptation rule

The signature of an entity or **feature** f of a type T of base **class** C is the result of applying T’s generic substitution to the signature of f in C.

Informative text

End

8.12.21 Definition: Generically constrained feature name

Consider a generic **class** C, a constrained Formal_generic_name G of C, a type T appearing as one of the Constraining_types for G, and a **feature** f of name f name in the base **class** of T. The generically constrained names of f for G in C are:

1 If one or more Single_constraint clauses for T include a Rename part with a clause

f name as e name, where the Feature_name part of e name (an Extended_feature_name) is g name: all such g name.

2 Otherwise: just f name.

8.12.22 Validity: Multiple Constraints rule

Validity code: `VTMC` A **feature** of name f name is applicable in a **class** C to a targetx whose type is a Formal_generic_name G constrained by two or more types CONST1, CONST2,…, if and only if it satisfies the following conditions:

1 At least one of the CONST

i has a **feature** available to C whose generically constrained name for G in C is f name.

2 If this is the case for two or more of the CONST

i , all the corresponding features are the same.

8.12.23 Definition: Base type of a multi-constraint formal generic type

The base type of a multiply constrained Formal_generic_name type is a type generically derived, with the same actual parameters as the current **class**, from a fictitious **class** with none of the optional parts except for Formal_generics and an Inheritance clause that lists all the constraining types as parents, with the given Renaming clauseif any, and resolves any conflicts between potentially ambiguous features by further renaming them to new names not available to develope rs.

8.13 Tuples

Informative text

End

8.13.1 Syntax: Tuple types

Tuple_type

≙ TUPLE [Tuple_parameter_list] Tuple_parameter_list

≙ "[" Tuple_parameters "]"

Tuple_parameters

≙ Type_list | Entity_declaration_list

8.13.2 Syntax: Manifest tuples

Manifest_tuple

≙ "[" Expression_list "]" Expression_list

≙ {Expression "," …}*

8.13.3 Definition: Type sequence of a tuple type

The type sequence of a tuple type is the sequence of types obtained by listing its arguments, if any, in the order in which they appear, every labeled argument being listed as many times as it has labels.

Informative text

End

8.13.4 Definition: Valuesequences associated with a tuple type

The value sequences associated with a tuple type T are sequences of values, each of the type appearing at the corresponding position in T's type sequence.

Informative text

End

8.14 Conformance

Informative text

VNC

End

8.14.1 Definition: Compatibility between types

A type is compatible with another if it either conforms or converts to it.

8.14.2 Definition: Compatibility between expressions

An expression b is compatible with an expression a if and only if b either conforms or converts to a.

8.14.3 Definition: Expression conformance

An expression exp of type SOURCE conforms to an expression ent of type TARGET if and only if they satisfy the following conditions:

1 SOURCE conforms to TARGET.

2 If TARGET is **attached**, so is SOURCE.

3 If SOURCE is **expanded**, its version of the function cloned from ANY is available to the

current **class**.

Informative text

attached

3 allows us, in the semantics of attachment, to

use a cloning operation when attaching an object with "copy semantics", without causing inconsistencies. A later definition will state what it means for an expression b to **convert** to another a. As a special case these properties also apply to entities. Conformance and convertibility are exclusive of each other, so we study the two mechanisms separately. The rest of the present discussion is devoted to conformance.

End

8.14.4 Validity: Signature conformance

Validity code: **VNCS** A signaturet = [B 1 , … B n], [S] conforms to a signatures = [A 1 , … A n], [R] if and only if it satisfies the following conditions:

1 Each of the two components of t has the same number of elements as the corresponding

component of s.

2 Each type in each of the two components of t conforms to the corresponding type in the

corresponding component of s.

3 Any B

i not identical to the corresponding A i is **detachable**.

Informative text

End

8.14.5 Definition: Covariant argument

In a redeclaration of a routine, a formal argument is covariant if its type differs from the type of the corresponding argument in at least one of the parents' versions.

Informative text

End

8.14.6 Validity: General conformance

Validity code: **VNCC** Let T and V be two types. V conforms to T if and only if one of the following conditions holds:

1 V and T are identical.

2 V conforms directly to T.

3 V is NONE and T is a **detachable** reference type.

4 V is B [Y

1 ,… Y n] where B is a generic **class**, T is B [X 1 ,… X n], and for every X i the corresponding Y i is identical to X i or, if the corresponding formal parameter does not specify **frozen**, conforms (recursively) to X i .

5 For some type U (recursively), V conforms to U and U conforms to T.

6 T or V or both are anchored types appearing in the same **class** C, and the deanchored

form of V in C (recursively) conforms to the deanchored form of T.

Informative text

class
assign
attached
expanded
class
class

frozen

End

8.14.7 Definition: Conformance path

A conformance path from a type U to a type T is a sequence of types T 0 , T 1 , … T n (n ≥ 1) such that T 0 is U, T n is T, and every T i (for 0 ≤ i < n) conforms to T i+1 . This notion also applies to classes by considering the associated base classes.

8.14.8 Validity: Direct conformance: reference types

Validity code: **VNCN** A Class_type CT of base **class** C conforms directly to a reference type BT if and only if it satisfies the following conditions:

1 Applying CT's generic substitution to one of the conforming parents of C yields BT.

2 If BT is **attached**, so is CT.

Informative text

End

8.14.9 Validity: Direct conformance: formal generic

Validity code: **VNCF** Let G be a formal generic parameter of a **class** C, which in the text of C may be used as a Formal_generic_name type. Then:

1 No type conforms directly to G.

2 G conforms directly to every type listed in its constraint, and to no other type.

8.14.10 Validity: Direct conformance: expanded types

Validity code: **VNCE** No type conforms directly to an **expanded** type.

Informative text

expanded
VNCN

End

8.14.11 Validity: Direct conformance: tuple types

Validity code: **VNCT** A Tuple_type U, of type sequence us, conforms directly to a type T if and only if T satisfies the following conditions:

1 T is a tuple type, of type sequence ts.

2 The length of us is greater than or equal to the length of ts.

3 For every element X of ts, the corresponding element of us conforms to X.

No type conforms directly to a tuple type except as implied by these conditions.

Informative text

End

8.15 Convertibility

Informative text

End

8.15.1 Definition: Conversion procedure, conversion type

A procedure whose name appears in a Converters clause is a conversion procedure. A type listed in a Converters clause is a conversion type.

8.15.2 Definition: Conversion query, conversion feature

A query whose name appears in a Converters clause is a conversion query. A **feature** that is either a conversion procedure or a conversion query is a conversion **feature**.

8.15.3 Validity: Conversion principle

No type may both conform and **convert** to another.

8.15.4 Validity: Conversion Asymmetry principle

No type T may **convert** to another through both a conversion procedure and a conversion query.

8.15.5 Validity: Conversion Non-Transitivity principle

That V converts to U and U to T does not imply that V converts to T.

8.15.6 Syntax: Converter clauses

Converters

convert Converter_list Converter_list

{Converter ","…} + Converter

Conversion_procedure | Conversion_query Conversion_procedure

Feature_name "(" "{" Type_list "}" ")" Conversion_query

Feature_name ":" "{" Type_list "}"

8.15.7 Validity: Conversion Procedure rule

Validity code: **VYCP** A Conversion_procedure listing a Feature_name fn and appearing in a **class** C with current type CT is valid if and only if it satisfies the following conditions, applicable to every type SOURCE listed in its Type_list:

1 fn is the name of a creation procedure cp of C.

2 If C is not generic, SOURCE does not conform to CT.

3 If C is generic, SOURCE does not conform to the type obtained from CT by replacing

every formal generic parameter by its constraint.

4 SOURCE's base **class** is different from the base **class** of any other conversion type listed

for a Conversion_procedure in the Converters clause of C.

5 The specification of the base **class** of SOURCE does not list a conversion query

specifying a type of base **class** C.

6 cp has exactly one formal argument, of a type ARG.

7 SOURCE conforms to ARG.

8 SOURCE involves no anchored type.

Informative text

2 states is that the conversion types must not conform to it (more precisely, to the current

type).

End

8.15.8 Validity: Conversion Query rule

Validity code: VYCQ A Conversion_query listing a Feature_name fn and appearing in a **class** C with current type CT is valid if and only if it satisfies the following conditions, applicable to every type TARGET listed in its Type_list:

1 fn is the name of a query f of C.

2 If C is not generic, CT does not conform to TARGET.

3 If C is generic, the type obtained from CT by replacing every formal generic parameter

by its constraint does not conform to TARGET.

4 TARGET's base **class** is different from the base **class** of any other conversion type listed

for a Conversion_query in the Converters clause of C.

5 The specification of the base **class** of TARGET does not list a conversion procedure

specifying a type of base **class** C.

6 f has no formal argument.

7 The result type of f conforms to TARGET.

8 TARGET involves no anchored type.

Informative text

End

8.15.9 Definition: Converting to a class

A type T of base **class** CT converts to a **class** C if either: • The deanchored form of T appears as conversion type for a procedure in the Converters clause of C. • A type based on C appears as conversion type for a query in the Converters clause of CT.

8.15.10 Definition: Converting to and from a type

A type U of base **class** D converts to a Class_type T of base **class** C if and only if either:

1 The deanchored form of U is the result of applying the generic substitution of the

deanchored form of T to a conversion type for a procedure cp appearing in the Converters clause of C.

2 The deanchored form of T is the result of applying the generic substitution of the

deanchored form of U to a conversion type for a query cq appearing in the Converters clause of D. A Class_type T converts from a type U if and only if U converts to T.

8.15.11 Definition: Converting “through”

A type U that converts to a type T:

1 Converts to T through a procedure cp if case 1 of the definition of “converting to a type”

applies.

2 Converts to T through a query cq if case 2 of the definition applies.

These terms also apply to “converting from” specifications.

Informative text

End

8.15.12 Semantics: Conversion semantics

Given an expression e of type U and a variable x of type T, where U converts to T, the effect of a conversion attachment of source e and targetx is the same as the effect of either:

1 If U converts to T through a procedure cp: the creation instruction create x.cp (e).

2 If U converts to T through a query cq: the assignment x := e.cq.

Informative text

End

8.15.13 Definition: Explicit conversion

The Kernel Library **class** TYPE [G] provides a function adapted alias "[ ]" (x: G): G which can be used for any type T and any expression exp of a type U compatible with T to produce a T version of exp, written {T} [exp] If U converts to T, this expression denotes the result of converting exp to T, and is called an explicit conversion.

Informative text

feature
class
20, "April", 2007
class

End

8.15.14 Validity: Expression convertibility

Validity code: **VYEC** An expression exp of type U converts to an entity ent of type T if and only if U converts to T through a conversion **feature** conv satisfying either of the following two conditions:

1 conv is precondition-free.

2 exp statically satisfies the precondition.

8.15.15 Definition: Statically satisfied precondition

A **feature** precondition is statically satisfied if it satisfies any of the following conditions:

1 It applies to a boolean, character, integer or real expression involving only constants,

states that the expression equals a specific constant value or (in the last three cases) belongs to a specified interval, and holds for that value or interval.

2 It applies to the type of an expression, states that it must be one of a specified set of

types, and holds for that type.

Informative text

class

1 is as a matter of fact acceptable as a NATURAL_8. So the definition of

expression convertibility permits a “statically satisfied” precondition, making such cases valid. It would be possible to generalize the definition by making permissible any precondition that can be assessed statically. But this would leave too much initiative to individual compilers: a “smarter” compiler might accept a precondition that another rejects, leading to incompatibilities. It was judged preferable to limit the rule to the two cases known to be important in practice; if others appear in the future, the rule will be extended.

End

8.15.16 Validity: Precondition-free routine

Validity code: **VYPF** A **feature** r of a **class** C is precondition-free if it is either:

1 Immediate in C, with either no Precondition clause or one consisting of a single

Assertion_clause (introduced by **require**) whose Boolean_expression is the constant **True**.

2 Inherited, and such that every precursor of r is (recursively) precondition-free, or r is

redeclared in C with a Precondition consisting of a single Assertion_clause (introduced by **require** else) whose Boolean_expression is the constant **True**.

Informative text

feature
class
feature
feature
class
require
True
require
feature

End

Informative text

End

8.16 Repeated inheritance

Informative text

End

8.16.1 Definition: Repeated inheritance, ancestor, descendant

Repeated inheritance occurs whenever (as a result of multiple inheritance) two or more of the ancestors of a **class** D have a common parent A. D is then called a repeated descendant of A, and A a repeated ancestor of D.

8.16.2 Semantics: Repeated Inheritance rule

Let D be a **class** and B 1 , … B n (n ≥ 2) be parents of D based on classes having a common ancestor A. Let f 1 , … f n be features of these respective parents, all having as one of their seeds the same **feature** f of A. Then:

1 Any subset of these features inherited by D under the same final name in D yields a

single **feature** of D.

2 Any two of these features inherited under a different name yield two features of D.

Informative text

End

8.16.3 Definition: Sharing, replication

A repeatedly inherited **feature** is shared if case 1 of the Repeated Inheritance rule applies, and replicated if case 2 applies.

8.16.4 Validity: Call Sharing rule

Validity code: `VMCS` It is valid for a **feature** f repeatedly inherited by a **class** D from an ancestor A, such that f is shared under repeated inheritance and not redeclared, to involve a **feature** g of A other than as the **feature** of a qualified callif and only if g is, along the corresponding inheritance paths, also shared.

Informative text

End

8.16.5 Semantics: Replication Semantics rule

Let f and g be two features both repeatedly inherited by a **class** A and both replicated under the Repeated Inheritance rule, with two respective sets of different names: f1 and f2, g1 and g2. If the version of f in D is the original version from A and either contains an unqualified call to g or (if f is an attribute) is the target of an assignment whose source involves g, the f1 version will use g1 for that call or assignment, and the f2 version will use g2.

Informative text

separate

End

8.16.6 Syntax: Select clauses

Select

select Feature_list

Informative text

when
feature
from

End

8.16.7 Validity: Select Subclause rule

Validity code: **VMSS** A Select subclause appearing in the parent part for a **class** B in a **class** D is valid if and only if, for every Feature_name f name in its Feature_list, f name is the final name in D of a **feature** that has two or more potential versions in D, and f name appears only once in the Feature_list.

Informative text

End

8.16.8 Definition: Version

A **feature** g from a **class** D is a version of a **feature** f from an ancestor of D if f and g have a seed in common.

8.16.9 Definition: Multiple versions

A **class** D has n versions (n ≥ 2) of a **feature** f of an ancestor A if and only if n of its features, all with different final names in D, are all versions of f.

8.16.10 Validity: Repeated Inheritance Consistency constraint

Validity code: **VMRC** It is valid for a **class** D to have two or more versions of a **feature** f of a proper ancestor A if and only if it satisfies one of the following conditions:

1 There is at most one conformance path from D to A.

2 There are two or more conformance paths, and the Parent clause for exactly one of them

in D has a Select clause listing the name of the version of f from the corresponding parent.

8.16.11 Definition: Dynamic binding version

For any **feature** f of a type T and any type U conforming to T, the dynamic binding version of f in U is the **feature** g of U defined as follows:

1 If f has only one version in U, then g is that **feature**.

2 If f has two or more versions in U, then the Repeated Inheritance Consistency constraint

ensures that either exactly one conformance path exists from U to T, in which case g is the version of f in U obtained along that path, or that a Select subclause name a version of f, in which case g is that version.

8.16.12 Definition: Inherited features

Let D be a **class**. Let precursors be the list obtained by concatenating the lists of features of every parent of D; this list may contain duplicates in the case of repeated inheritance. The list inherited of inherited features of D is obtained from precursors as follows:

1 In the list precursors, for any set of two or more elements representing features that are

repeatedly inherited in D under the same name, so that the Repeated Inheritance rule yields sharing, keep only one of these elements. The Repeated Inheritance Consistency constraint (sharing case) indicates that these elements must all represent the same **feature**, so that it does not matter which one is kept.

2 For every **feature** f in the resulting list, if D undefines f, replace fbya **deferred feature**

with the same signature, specification and header comment.

3 In the resulting list, for any set of **deferred** features with the same final name in D, keep

only one of these features, with assertions and header comment joined as per the Join Semantics rule. (Keep the signature, which the Join rule requires to be the same for all the features involved after possible redeclaration.)

4 In the resulting list, remove any **deferred feature** such that the list contains an effective

feature with the same final name. (This is the case in which a **feature** f, inherited as effective, effects one or more **deferred** features: of the whole group, only f remains.)

5 All the features of the resulting list have different names; they are the inherited features

of D in their parent forms. From this list, produce a new one by replacing any **feature** that D redeclares (through redefinition or effecting) with the result of the redeclaration, and retaining any other **feature** as it is.

6 The result is the list inherited of inherited features of D.

8.16.13 Semantics: Join-Sharing Reconciliation rule

If a **class** inherits two or more features satisfying both the conditions of sharing under the Repeated Inheritance rule and those of the Join rule, the applicable semantics is the Repeated Inheritance rule.

8.16.14 Definition: Precursor

A precursor of an inherited **feature** of final name f name is any parent **feature** — appearing in the list precursors obtained through case

1 of the definition of “Inherited features” — that the **feature**

mergings resulting from the subsequent cases reduce into a **feature** of name f name.

8.16.15 Validity: Feature Name rule

Validity code: `VMFN` It is valid for a **feature** f of a **class** C to have a certain final name if and only if it satisfies the following conditions:

1 No other **feature** of C has that same **feature** name.

2 If f is shared under repeated inheritance, its precursors all have either no Alias or the

same alias.

Informative text

separate
feature
feature
feature
feature
deferred
feature
feature
feature
feature
feature
class
feature

End

8.16.16 Validity: Name Clash rule

Validity code: VMNC The following properties govern the names of the features of a **class** C:

1 It is invalid for C to introduce two different features with the same name.

2 If C introduces a **feature** with the same name as a **feature** it inherits as effective, it must

rename the inherited **feature**.

3 If C inherits two features as effective from different parents and they have the same

name, the **class** must also (except under sharing for repeated inheritance) remove the name clash through renaming.

Informative text

End

8.17 Control structures

Informative text

End

8.17.1 Semantics: Compound (non-exception) semantics

The effect of executing a Compound is: • If it has zero instructions: to leave the state of the computation unchanged. • If it has one or more instructions: to execute the first instruction of the Compound, then (recursively) to execute the Compound obtained by removing the first instruction. This specification, the non-exception semantics of Compound, assumes that no exception is triggered. If the execution of any of the instructions triggers an exception, the Exception Semantics rule takes effect for the rest of the Compound's instructions.

Informative text

End

8.17.2 Syntax: Conditionals

Conditional

≙ **if** Then_part_list [Else_part] **end** Then_part_list = ≙ {Then_part **elseif** …} + Then_part

≙ Boolean_expression **then** Compound Else_part

≙ **else** Compound

8.17.3 Definition: Secondary part

The secondary part of a Conditional possessing at least one **elseif** is the Conditional obtained by removing the initial "if Then_part_list" and replacing the first **elseif** of the remainder by if.

8.17.4 Definition: Prevailing immediately

The execution of a Conditional starting with if condition 1 is said to prevail immediately if condition 1 has value true.

8.17.5 Semantics: Conditional semantics

The effect of a Conditional is: • If it prevails immediately: the effect of the first Compound in its Then_part_list. • Otherwise, if it has at least one **elseif**: the effect (recursively) of its secondary part. • Otherwise, if it has an Else part: the effect of the Compound in that Else part. • Otherwise: no effect.

Informative text

elseif
until

End

8.17.6 Definition: Inspect expression

The **inspect** expression of a Multi_branch is the expression appearing after the keyword **inspect**.

8.17.7 Syntax: Multi-branch instructions

Multi_branch

≙ **inspect** Expression [When_part_list] [Else_part] **end** When_part_list

≙ When_part + When_part

≙ **when** Choices **then** Compound Choices

≙ {Choice "," …} + Choice

≙ Constant | Manifest_type | Constant_interval | Type_interval Constant_interval

≙ Constant ".." Constant Type_interval

≙ Manifest_type ".." Manifest_type

8.17.8 Definition: Interval

An interval is a Constant_interval or Type_interval.

8.17.9 Definition: Unfolded form of a multi-branch

To obtain the unfolded form of a Multi_branch instruction, apply the following transformations in the order given:

1 Replace every constant **inspect** valueby its manifest value.

2 If the type T of the **inspect** expression is any sized **variant** of CHARACTER, STRING or

INTEGER, replace every **inspect** valuev by {T} v.

3 Replace every interval by its unfolded form.

Informative text

1000 here, will cause a validity error.

End

8.17.10 Definition: Unfolded form of an interval

The unfolded form of an interval a..b is the following (possibly empty) list:

1 If a and b are constants, both of either a character type, a string type or an integer type,

and of manifest values va and vb: the list made up of all values i, if any, such that va ≤ i ≤ vb, using character, integer or lexicographical order respectively.

2 If a and b are both of type TYPE [T] for some T, and have manifest values va and vb:

the list containing every Manifest_type of the system conforming to vb and to which va conforms.

3 If neither of the previous two cases apply: an empty list.

Informative text

End

8.17.11 Validity: Interval rule

Validity code: **VOIN** An Interval is valid if and only if its unfolded form is not empty.

8.17.12 Definition: Inspect values of a multi-branch

The **inspect** values of a Multi_branch instruction are all the values listed in the Choices parts of the instruction's unfolded form.

Informative text

inspect

End

8.17.13 Validity: Multi-branch rule

Validity code: **VOMB** A Multi_branch instruction is valid if and only if its unfolded form satisfies the following conditions.

1 Inspect values are all valid.

2 Inspect values are all constants.

3 The manifest values of any two **inspect** values are different.

4 If the **inspect** expression is of type TYPE [T] for some type T, all **inspect** values are types.

5 If case 4 does not apply, the **inspect** expression is one of the sized variants of INTEGER,

CHARACTER or STRING.

8.17.14 Semantics: Matching branch

During execution, a matching branch of a Multi_branch is a When_part wp of its unfolded form, satisfying either of the following for the value val of its **inspect** expression:

1 val ~ i, where i is one of the non-Manifest_type **inspect** values listed in wp.

2 val denotes a Manifest_type listed among the choices of wp.

Informative text

End

8.17.15 Semantics: Multi-Branch semantics

Executing a Multi_branch with a matching branch consists of executing the Compound following the then in that branch. In the absence of matching branch:

1 If the Else_part is present, the effect of the Multi_branch is that of the Compound

appearing in its Else_part.

2 Otherwise the execution triggers an exception of type BAD_INSPECT_VALUE.

8.17.16 Syntax: Loops

Loop

Initialization [Invariant] Exit_condition Loop_body [Variant] **end** Initialization

from Compound Exit_condition

until Boolean_expression Loop_body

loopCompound

8.17.17 Semantics: Loop semantics

The effect of a Loop is the effect of executing the Compound of its Initialization, then its Loop_body. The effect of executing a Loop_body is: • If the Boolean_expression of the Exit_condition evaluates to true: no effect (leave the state of the computation unchanged). • Otherwise: the effect of executing the Compound clause, followed (recursively) by the effect of executing the Loop_body again in the resulting state.

8.17.18 Syntax: Debug instructions

Debug

debug ["("Key_list ")"] Compound **end**

8.17.19 Semantics: Debug semantics

A language processing tool must provide an option that makes its possible to enable or disable Debug instructions, both globally and for individual keys of a Key_list. Such an option may be settable for an entire system, or for individual classes, or both. Letter case is not significant for a debug key. The effect of a Debug instruction depends on the mode that has been set for the current **class**: • If the Debug option is on generally, or if the instruction includes a Key_list and the option is on for at least one of the keys in the list, the effect of the Debug instruction is that of its Compound. • Otherwise the effect is that of a null instruction.

8.18 Attributes

Informative text

End

8.18.1 Syntax: Attribute bodies

Attribute

attribute Compound

Informative text

attribute
attached
expanded
not
as

then
if
assign
Result
once

End

8.18.2 Validity: Manifest Constant rule

Validity code: VQMC A declaration of a **feature** f introducing a manifest constant is valid if and only if the Manifest_constant m used in the declaration matches the type T declared for f in one of the following ways:

1 misa Boolean_constant and T is BOOLEAN.

2 misa Character_constant and T is one of the sized variants of CHARACTER for which

misa valid value.

3 m is an Integer_constant and T is one of the sized variants of INTEGER for which m is

a valid value.

4 misa Real_constant and T is one of the sized variants of REAL for which misa valid

value.

5 misa Manifest_string and T is one of the sized variants of STRING for which misa valid

value.

6 misa Manifest_type, of the form {Y} for some type Y, and T is TYPE [X] for some

stand-alone type X to which Y conforms.

Informative text

require
class

End

8.19 Objects, values and entities

Informative text

End

8.19.1 Semantics: Type, generating type of an object; generator

Every run-time object is a direct instance of exactly one stand-alone type of the system, called the generating type of the object, or just “the type of the object” if there is no ambiguity. The base **class** of the generating type is called the object’s generating **class**, or generator for short.

8.19.2 Definition: Reference, void, attached, attached to

A reference is a value that is either: • **Void**, in which case it provides no more information. • Attached, in which case it gives access to an object. The reference is said to be **attached** to that object, and the object **attached** to the reference.

8.19.3 Semantics: Object principle

Every non-void value is either an object or a reference **attached** to an object.

8.19.4 Definition: Object semantics

Every run-time object has either copy semantics or reference semantics. An object has copy semantics if and only if its generating type is an **expanded** type.

Informative text

End

8.19.5 Definition: Non-basic class, non-basic type, field

Any **class** other than the basic types is said to be a non-basic **class**. Any type whose base **class** is non-basic is a non-basic type, and its instances are non-basic objects. A direct instance of a non-basic type is a sequence of zero or more values, called fields. There is one field for every attribute of the type's base **class**.

8.19.6 Definition: Subobject, composite object

Any **expanded** field of an object is a subobject of that object. An object that has a non-basic subobject is said to be composite.

8.19.7 Definition: Entity, variable, read-only

An entity is an Identifier, or one of two reserved words (**Current** and **Result**), used in one of the following roles:

1 Final name of an attribute of a **class**.

2 Local variable of a routine or Inline_agent, including **Result** for a query.

3 Formal argument of a routine or inline agent.

4 Object Test local.

5 **Current**, the predefined entity used to represent a reference to the current object (the

target of the latest not yet completed routine call). Names of non-constant attributes and local variables are variable entities, also called just vari- ables. Constant attributes, formal arguments, Object Test locals and **Current** are read-only entities.

Informative text

Current
Current
Result

End

8.19.8 Syntax: Entities and variables

Entity

Variable | Read_only Variable

Variable_attribute | Local Variable_attribute

Feature_name Local

Identifier | **Result** Read_only

Formal | Constant_attribute | **Current** Formal

Identifier Constant_attribute

Feature_name

8.19.9 Validity: Entity rule

Validity code: **VEEN** An occurrence of an entity e in the text of a **class** C (other than as the **feature** of a qualified call) is valid if and only if it satisfies one of the following conditions:

1 e is **Current**.

2 e is the final name of an attribute of C.

3 e is the local variable **Result**, and the occurrence is in a Feature_body, Postcondition or

Rescue part of an Attribute_or_routine text for a query or an Inline_agent whose signature includes a result type.

4 e is **Result** appearing in the Postcondition of a constant attribute's declaration.

5 e is listed in the Identifier_list of an Entity_declaration_group in a Local_declarations part

of a **feature** or Inline_agent fa, and the occurrence is in a Local_declarations, Feature_body or Rescue part for fa.

6 e is listed in the Identifier_list of an Entity_declaration_group in a Formal_arguments

part for a routine r, and the occurrence is in a declaration for r.

7 e is listed in the Identifier_list of an Entity_declaration_group in the Agent_arguments

part of an Agent a, and the occurrence is in the Agent_body of a.

8 e is the Object-Test Local of an Object_test, and the occurrence is in its scope.

Informative text

End

8.19.10 Validity: Variable rule

Validity code: **VEVA** A Variable entity v is valid in a **class** C if and only if it satisfies one of the following conditions:

1 v is the final name of a variable attribute of C.

2 v is the name of a local variable of the immediately enclosing routine or agent.

8.19.11 Definition: Self-initializing type

A type is self-initializing in a **class** C if it is one of:

1 A **detachable** type.

2 A self-initializing formal parameter.

3 An **attached** type (including **expanded** types and, as a special case of these, basic

types) whose creation procedures include a version of default_create from ANY available for creation to C.

Informative text

Void
detachable

attached
expanded
require
attached
class

End

8.19.12 Semantics: Default Initialization rule

Every self-initializing type T has a default initialization value as follows:

1 For a **detachable** type: a void reference.

2 For a self-initializing **attached** type: an object obtained by creating an instance of T

through default_create.

3 For a self-initializing formal parameter: for every generic derivation, (recursively) the

default initialization value of the corresponding actual generic parameter.

4 For BOOLEAN: the boolean value false.

5 For a sized **variant** of CHARACTER: null character.

6 For a sized **variant** of INTEGER: integer zero.

7 For a sized **variant** of REAL: floating-point zero.

8 For POINTER: a null pointer.

9 For TYPED_POINTER: an object representing a null pointer.

Informative text

End

8.19.13 Definition: Self-initializing variable

A variable is self-initializing if one of the following holds:

1 Its type is a self-initializing type.

2 It is an attribute declared with an Attribute part such that the entity **Result** is properly set

at the end of its Compound.

Informative text

2 the result of the attribute's own initialization. That initialization must

ensure that **Result** is "properly set" as defined next (partly recursively from the above definition) .

End

8.19.14 Definition: Evaluation position, precedes

An evaluation position is one of: • In a Compound, one of its Instruction components. • In an Assertion, one of its Assertion_clause components. • In either case, a special end position. A position p precedes a position q if they are both in the same Compound or Assertion, and either: • p and q are both Instruction or Assertion_clause components, and p appears before q in the corresponding list. • q is the end position and p is not.

Informative text

ensure

The relation as defined only applies to first-level components of the compound: if i2 itself contains a compound, for example if it is of the form if c then i4; i5 end, then i4 is not an evaluation position of the outermost compound, and so has no “precedes” relation with any of i1, i2 and i3.

End

8.19.15 Definition: Setter instruction

A setter instruction is an assignment or creation instruction. If xisa variable, a setter instruction is a setter for x if its assignment target or creation target is x.

8.19.16 Definition: Properly set variable

At an evaluation position ep in a **class** C, a variable x is properly set if one of the following conditions holds:

1 x is self-initializing.

2 ep is an evaluation position of the Compound of a **feature** or Inline_agent of the Internal

form, one of whose instructions precedes ep and is a setter for x.

3 xisa variable attribute, and is (recursively) properly set at the end position of every

creation procedure of C.

4 ep is an evaluation position in a Compound that is part of an instruction ep’, itself

belonging to a Compound, and x is (recursively) properly set at position ep’.

5 ep is in a Postcondition of a routine or Inline_agent of the Internal form, and x is

(recursively) properly set at the end position of its Compound.

6 ep is an Assertion_clause containing **Result** in the Postcondition of a constant attribute

Informative text

4 accounts for nested compounds. For

assertions other than postconditions, which cannot use variables other than attributes,

3 is the

only applicable condition. The somewhat special case

6 is a consequence of our classification of

Result among local variables even in the Postcondition of a constant attribute. As an artefact of the definition’s phrasing, every variable attribute is “properly set” in any effective routine of a **deferred class**, since such a **class** has no creation procedures. This causes no problem since a failure to set the attribute properly will be caught, in the validity rule below, for versions of the routine in effective descendants.

End

8.19.17 Validity: Variable Initialization rule

Validity code: `VEVI` It is valid for an Expression, other than the target of an Assigner_call, to be also a Variable if it is properly set at the evaluation position defined by the closest enclosing Instruction or Assertion_clause.

Informative text

class

• Give the attribute a specific initialization through an explicit Attribute part that sets **Result** to the appropriate value.

End

8.19.18 Definition: Variable setting and its value

A setting for a variable x is any one of the following run-time events, defining in each case the value of the setting:

1 Execution of a setter for x. (Value: the object **attached** to x by the setter, or a void

reference if none.)

2 If xisa variable attribute with an Attribute part: evaluation of that part, implying execution

of its Compound. (Value: the object **attached** to **Result** at the end position of that Compound, or a void reference if none.)

3 If the type T of x is self-initializing: assignment to x of T's default initialization value.

(Value: that initialization value.)

Informative text

until
ensure
ensure

End

8.19.19 Definition: Execution context

At any time during execution, the current execution context for a variable is the period elapsed since:

1 For an attribute: the creation of the current object.

2 For a local variable: the start of execution of the current routine.

8.19.20 Semantics: Variable Semantics

The value produced by the run-time evaluation of a variable x is:

1 If the execution context has previously executed at least one setting for x: the value of

the latest such setting.

2 Otherwise, if the type T of x is self-initializing: assignment to x of T's default initialization

value, causing a setting of x.

3 Otherwise, if xisa variable attribute with an Attribute part: evaluation of that part,

implying execution of its Compound and hence a setting for x.

4 Otherwise, if x is **Result** in the Postcondition of a constant attribute: the value of the

attribute.

Informative text

The Variable Initialization rule ensures that one of these cases will apply, so that x will always have a well-defined result for evaluation. This property was our main goal, and its achievement concludes the discussion of variable semantics.

End

8.19.21 Semantics: Entity Semantics rule

Evaluating an entity yields a value as follows:

1 For **Current**: a value **attached** to the current object.

2 For a formal argument of a routine or Inline_agent: the value of the corresponding actual

at the time of the current call.

3 For a constant attribute: the value of the associated Manifest_constant as determined

by the Manifest Constant Semantics rule.

4 For an Object-Test Local: as determined by the Object-Test Local Semantics rule.

5 For a variable: as determined by the Variable Semantics rule.

Informative text

End

8.20 Creating objects

Informative text

End

8.20.1 Semantics: Creation principle

Any execution of a creation operation must produce an object that satisfies the **invariant** of its generating **class**.

Informative text

End

8.20.2 Definition: Creation operation

A creation operation is a creation instruction or expression.

8.20.3 Validity: Creation Precondition rule

Validity code: `VGCP` A Precondition of a routine r is creation-valid if and only if its unfolded formuf satisfies the following conditions:

1 The predefined entity **Current** does not appear in uf.

2 No Unqualified_call appears in uf.

3 Every **feature** whose final name appears in the uf is available to every **class** to which r

is available for creation.

Informative text

ensure

3 also governs the use of operators: with plus alias "+", the expression a + b

will be acceptable only if the **feature** plus is available for creation as stated.

End

8.20.4 Syntax: Creators parts

Creators

- Creation_clause + Creation_clause
- **create** [Clients] [Header_comment] Creation_procedure_list Creation_procedure_list
- {Creation_procedure ","…} + Creation_procedure
- Feature_name

8.20.5 Definition: Unfolded creators part of a class

The unfolded creators part of a **class** C is a Creators defined as:

1 If C has a Creators partc: c.

2 If C is **deferred**: an empty Creators part.

3 Otherwise, a Creators part built as follows, dc_name being the final name in C of its

version of default_create from ANY: create dc_name

Informative text

class
deferred
class
deferred
class

Case 3 reflects the convention that an absent Creators part stands for create dc_name — normally create default_create, but dc_name may be another name if the **class** or one of its proper ancestors has renamed default_create.

End

8.20.6 Validity: Creation Clause rule

Validity code: `VGCC` A Creation_clause in the unfolded creators part of a **class** C is valid if and only if it satisfies the following conditions, the last four for every Feature_name cp_name in the clause's Feature_list:

1 C is effective.

2 cp_name appears only once in the Feature_list.

3 cp_name is the final name of some procedure cp of C.

4 cp is not a once routine.

5 The precondition of cp, if any, is creation-valid.

Informative text

End

8.20.7 Definition: Creation procedures of a class

The creation procedures of a **class** are all the features appearing in any Creation_clause of its unfolded creators part.

Informative text

class
class

End

8.20.8 Creation procedure property

An effective **class** has at least one creation procedure.

Informative text

End

8.20.9 Definition: Creation procedures of a type

The creation procedures of a type T are:

1 If T is a Formal_generic_name, the constraining creators for T.

2 Otherwise, the creation procedures of T's base **class**.

Informative text

End

8.20.10 Definition: Available for creation; general creation procedure

A creation procedure of a **class** C, listed in a Creation_clause cc of C's unfolded creators part, is available for creation to the descendants of the classes given in the Clients restriction of cc, if present, and otherwise to all classes. If there is no Clients restriction, the procedure is said to be a general creation procedure.

8.20.11 Syntax: Creation instructions

Creation_instruction

create [Explicit_creation_type] Creation_call Explicit_creation_type

"{" Type "}" Creation_call

Variable [Explicit_creation_call] Explicit_creation_call

"." Unqualified_call

8.20.12 Definition: Creation target, creation type

The creation target (or just "target" if there is no ambiguity) of a Creation_instruction is the Variable of its Creation_call. The creation type of a creation instruction, denoting the type of the object to be created, is: • The Explicit_creation_type

appearing (between braces) in the instruction, if present. • Otherwise, the type of the instruction's target.

8.20.13 Semantics: Creation Type theorem

The creation type of a creation instruction is always effective.

8.20.14 Definition: Unfolded form of a creation instruction

Consider a Creation_instruction ci of creation type CT. The unfolded form of ci is a creation instruction defined as:

1 If ci has an Explicit_creation_call, then ci itself.

2 Otherwise, a Creation_instruction obtained from ci by making the Creation_call explicit,

using as **feature** name the final name in CT of CT's version of ANY's default_create.

8.20.15 Validity: Creation Instruction rule

Validity code: `VGCI` A Creation_instruction of creation type CT, appearing in a **class** C, is valid if and only if it satisfies the following conditions:

1 CT conforms to the target's type.

2 The **feature** of the Creation_call of the instruction's unfolded form is available for

creation to C.

3 That Creation_call is argument-valid.

4 CT is generic-creation-ready.

Informative text

End

8.20.16 Validity: Creation Instruction properties

Validity code: `VGCP` A Creation_instruction ci of creation type CT, appearing in a **class** C, is valid only if it satisfies the following conditions, assuming CT is not a Formal_generic_name and calling BCT the base **class** of CT and dc the version of ANY's default_create in BCT:

1 BCT is an effective **class**.

2 If ci includes a Type part, the typeit lists (which is CT) conforms to the type of the

instruction's target.

3 If ci has no Creation_call, then BCT either has no Creators part or has one that lists dc

as one of the procedures available to C for creation.

4 If BCT has a Creators part which doesn't listdc, then ci has a Creation_call.

5 If ci has a Creation_call whose **feature** f is not dc, then BCT has a Creators part which

lists f as one of the procedures available to C for creation.

6 If ci has a Creation_call, that call is argument-valid.

If CT is a Formal_generic_name, the instruction is valid only if it satisfies the following conditions:

7 CT denotes a constrained generic parameter.

8 The Constraint for CT specifies one or more procedures as constraining creators.

9 If ci has no Creation_call, one of the constraining creators is the Constraint's version of

default_create from ANY.

10 If ci has a Creation_call, one of the constraining creators is the **feature** of the

Creation_call.

Informative text

End

8.20.17 Semantics: Creation Instruction Semantics

The effect of a creation instruction of targetx and creation type TC is the effect of the following sequence of steps, in order:

1 If there is not enough memory available for a new direct instance of TC, trigger an

exception of type NO_MORE_MEMORY in the routine that attempted to execute the instruction. The remaining steps do not apply in this case.

2 Create a new direct instance of TC, with reference semantics if CT is a reference type

and copy semantics if CT is an **expanded** type.

3 Call, on the resulting object, the **feature** of the Unqualified_call of the instruction's

unfolded form.

4 Attach x to the object.

Informative text

1 be "enough memory

available for a new direct instance" — the memory being reused — and so the exception cannot happen. One might expect, between steps 2 and 3, a step of default initialization of the fields of the new object, since this is the intuitive semantics of the language: integers initialized to zero, **detachable** references to void etc. There is no need, however, for such a step since the Variable Semantics rule **implies** that an attribute or other variable, unless previously set by an explicit attachment, is automatically set on first access. The rule **implies** for example that an integer field will be set to zero. More generally, the semantics of the language guarantees that in every run-time circumstance any object field and local variable, even if never explicitly assigned to yet, always has a well-defined value when the computation needs it. About step 3, remember that the notion of "unfolded form" allows us to consider that every creation instruction has an Unqualified_call; in the procedure-less form create x, this is a call to default_create. Also notethe order of steps: attachment to the targetx is the last operation. Until then, x retains its earlier value, void if xisa previously unattached reference. In step 2, "not enough memory available" is a precise notion (the definition appears below); it means that even after possible garbage collection the memory available for the system's execution is not sufficient for the requested object creation.

End

8.20.18 Syntax: Creation expressions

Creation_expression

create Explicit_creation_type [Explicit_creation_call]

8.20.19 Definition: Properties of a creation expression

The creation type and unfolded form of a creation expression are defined as for a creation instruction.

8.20.20 Validity: Creation Expression rule

Validity code: **VGCE** A Creation_expression of creation type CT, appearing in a **class** C, is valid if and only if it satisfies the following conditions:

1 The **feature** of the Creation_call of the expression's unfolded form is available for

creation to C.

2 That Creation_call is argument-valid.

3 CT is generic-creation-ready.

8.20.21 Validity: Creation Expression Properties

Validity code: **VGCX** A Creation_expression ce of creation type CT, appearing in a **class** C, is valid only if it satisfies the following conditions, assuming CT is not a Formal_generic_name and calling BCT the base **class** of CT and dc the version of ANY's default_create in BCT:

1 BCT is an effective **class**.

2 If ce has no Explicit_creation_call, then BCT either has no Creators part or has one that

lists dc as one of the procedures available to C for creation.

3 If BCT has a Creators part which doesn't listdc, then ce has an Explicit_creation_call.

4 If ce has an Explicit_creation_call whose **feature** f is not dc, then BCT has a Creators

part which lists f as one of the procedures available to C for creation.

5 If ce has an Explicit_creation_call, that call is argument-valid.

If CT is a Formal_generic_name, the expression is valid only if it satisfies the following conditions:

6 CT denotes a constrained generic parameter.

7 The Constraint for CT specifies one or more procedures as constraining creators.

8 If ce has no Creation_call, one of the constraining creators is the Constraint's version of

default_create from ANY.

9 If ce has a Creation_call, one of the constraining creators is the **feature** of the

Creation_call.

Informative text

End

8.20.22 Semantics: Creation Expression Semantics

The value of a creation expression of creation type TC is — except if step 1 below triggers an exception, in which case the expression has no value — a value **attached** to a new object as can be obtained through the following sequence of steps:

1 If there is not enough memory available for a new direct instance of TC, trigger an

exception of type NO_MORE_MEMORY in the routine that attempted to execute the expression. In this case the expression has no value and the remaining steps do not apply.

2 Create a new direct instance of TC, with reference semantics if CT is a reference type

and copy semantics if CT is an **expanded** type.

3 Call, on the resulting object, the **feature** of the Unqualified_call of the expression's

unfolded form.

Informative text

End

8.20.23 Definition: Garbage Collection, not enough memory available

Authors of Eiffel implementation are required to provide garbage collection, defined as a mechanism that can reuse for allocating new objects the memory occupied by unreachable objects, guaranteeing the following two properties:

1 Consistency: the garbage collector never reclaims an object unless it is unreachable.

2 Completeness: no allocation request for an object of a certain size s will fail if there exists

an unreachable object of size >= s. Not enough memory available for a certain size s means that even after possible application of the garbage collection mechanism the memory available to the program is not sufficient for allocating an object of size s.

8.21 Comparing and duplicating objects

Informative text

End

8.21.1 Object comparison features from ANY

The features whose contract views appear below are provided by **class** ANY. **frozen** default_is_equal (other : ANY): BOOLEAN -- Is other **attached** to object field-by-field equal -- to current object? **ensure** same_type: **Result implies** same_type (other) symmetric: **Result** = other.default_is_equal (**Current**) consistent: **Result implies** is_equal (other) is_equal (other : ANY): BOOLEAN -- Is other **attached** to object considered equal -- to current object? **ensure** same_type: **Result implies** same_type (other) symmetric: **Result** = other.is_equal (**Current**)

consistent: default_is_equal (other) **implies Result** The original version of is_equal in ANY has the same effect as default_is_equal.

Informative text

frozen
class
redefine
attached

End

8.21.2 Syntax: Equality expressions

Equality

≙ Expression Comparison Expression Comparison

≙ "=" | "/=" | "~" | "/~"

8.21.3 Semantics: Equality Expression Semantics

The Boolean_expression e ~ f has value true if and only if the values of e and f are both **attached** and such that e.is_equal (f) holds. The Boolean_expression e = f has value true if and only if the values of e and f are one of:

1 Both void.

2 Both **attached** to the same object with reference semantics.

3 Both **attached** to objects with copy semantics, and such that e ~ f holds.

Informative text

End

8.21.4 Semantics: Inequality Expression Semantics

The expression e /= f has value true if and only if e = f has value false. The expression e /~ f has value true if and only if e ~ f has value false.

8.21.5 Copying and cloning features from ANY

The features whose contract views appear below are provided by **class** ANY as secret features. copy (other : ANY) -- Update current object using fields of object -- **attached** to other, to yield equal objects. **require** same_type: other.same_type (**Current**) **ensure** equal : is_equal (other) **frozen** default_copy (other : ANY) -- Update current object using fields of object -- **attached** to other, to yield identical objects. **require** same_type: other.same_type (**Current**) **ensure**

equal : default_is_equal (other) **frozen** cloned: like **Current** -- New object equal to current object -- (relies on copy) **ensure** equal : is_equal (**Result**) **frozen** default_cloned: like **Current** -- New object equal to current object -- (relies on default_copy) **ensure** equal : default_is_equal (**Result**) The original versions of copy and cloned in ANY have the same effect as default_copy and default_cloned respectively.

Informative text

attached
attached
class
redefine
frozen
frozen
class
class
invariant
class
invariant
implies
Current
class
export
export
invariant
class
class
export

End

8.21.6 Deep equality, copying and cloning

The **feature** is_deep_equal of **class** ANY makes it possible to compare object structures recursively; the features , deep_copy and deep_cloned duplicate an object structure recursively.

Informative text

End

8.22 Attaching values to entities

Informative text

End

8.22.1 Definition: Reattachment, source, target

A reattachment operation is one of:

1 An Assignment x := y; then y is the attachment's source and x its target.

2 The run-time association, during the execution of a routine call, of an actual argument

(the source) to the corresponding formal argument (the target).

Informative text

convert

End

8.22.2 Syntax: Assignments

Assignment

≙ Variable ":=" Expression

8.22.3 Validity: Assignment rule

Validity code: **VBAR** An Assignment is valid if and only if its source expression is compatible with its target entity.

Informative text

convert
convert

End

8.22.4 Semantics: Reattachment principle

After a reattachement to a target entity t of type TT, the object **attached** to t, if any, is of a type conforming to TT.

8.22.5 Semantics: Attaching an entity, attached entity

Attaching an entity e to an object O is the operation ensuring that the value of e becomes **attached** to O.

Informative text

attached
attached

End

8.22.6 Semantics: Reattachment Semantics

The effect of a reattachment of source expression source and target entity target is the effect of the first of the following steps whose condition applies:

1 If source converts to target: perform a conversion attachment from source to target.

2 If the value of source is a void reference: make target's value void as well.

3 If the value of source is **attached** to an object with copy semantics: create a clone of that

object, if possible, and attach target to it.

4 If the value of source is **attached** to an object with reference semantics: attach target to

that object.

Informative text

3 is — as

also noted in the discussion of creation — often avoidable in practice if the target is **expanded** and already initialized, so that the instruction can reuse the memory of the previous object. Case 1 indicates that a conversion, if applicable, overrides all other possibilities. In those other cases, if follows from the Assignment rule that source must conform to target. Case 2 is, from the validity rules, possible only if both target and source are declared of **detachable** types. In case 3, a "clone" of an object is obtained by application of the function cloned from ANY; expression conformance ensures that cloned is available (exported) to the type of target; otherwise, cloning could produce an inconsistent object. The cloning might be impossible for lack of memory, in which case the semantics of the cloning operation specifies triggering an exception, of type NO_MORE_MEMORY. As usual with exceptions, the rest of case

3 does not then apply.

In case 4 we simply reattach a reference. Because of the validity rules (no reference type conforms to an **expanded** type), the target must indeed be of an reference type. This rule defines the effect of a construct through a sequence of cases, looking for the first one that matches. As usual with semantic rules, this only specifies the result, but does not imply that the implementation must try all of them in order.

End

8.22.7 Semantics: Assignment Semantics

The effect of a reassignment x := y is determined by the Reattachment Semantics rule, with source y and targetx.

Informative text

End

8.22.8 Definition: Dynamic type

The dynamic type of an expression x, at some instant of execution, is the type of the object to which x is **attached**, or NONE if x is void.

8.22.9 Definition: Polymorphic expression; dynamic type and class sets

An expression that has two or more possible dynamic types is said to be polymorphic. The set of possible dynamic types for an expression x is called the dynamic type set of x. The set of base classes of these types is called the dynamic **class** set of x.

8.22.10 Syntax: Assigner calls

Assigner_call

≙ Expression ":=" Expression

Informative text

as
as
or
as
as
alias
and

End

8.22.11 Validity: Assigner Call rule

Validity code: **VBAC** An Assigner_call of the form target := source, where target and source are expressions, is valid if and only if it satisfies the following conditions:

1 source is compatible with target.

2 The Equivalent Dot Form of target is a qualified Object_call whose **feature** has an

assigner command.

8.22.12 Semantics: Assigner Call semantics

The effect of an Assigner_call target := source, where the Equivalent Dot Form of target is x.forx .f (args) and f has an assigner command p, is, respectively, x.p (source) or x.p (source, args).

Informative text

End

8.23 Feature call

Informative text

End

8.23.1 Validity: Call Use rule

Validity code: **VUCN**

A Call of **feature** f denotes:

1 If f is a query (attribute or a function): an expression.

2 If f is a procedure: an instruction.

8.23.2 Syntax: Feature calls

Call

≙ Object_call | Non_object_call Object_call

≙ [Target "."] Unqualified_call Unqualified_call

≙ Feature_name [Actuals] Target

≙ Local | Read_only | Call | Parenthesized_target Parenthesized_target

≙ "(|" Expression "|)" Non_object_call

≙ "{" Type "}" "." Unqualified_call

Informative text

then
local
and
Current
or
as

End

8.23.3 Syntax: Actual arguments

Actuals

≙ "(" Actual_list ")" Actual_list

≙ {Expression "," …} +

8.23.4 Definition: Unqualified, qualified call

An Object_call is qualified if it has a Target, unqualified otherwise.

Informative text

End

8.23.5 Definition: Target of a call

Any Object_call has a target, defined as follows:

1 If it is qualified: its Targetcomponent.

2 If it is unqualified: **Current**.

Informative text

1. is a (b, c) + x. In a multidot case the target includes the Calldeprived of its last part, for example x.f (args).ginx.f (args).g.h (args1).

End

8.23.6 Definition: Targettypeof a call

Any Call has a target type, defined as follows:

1 For an Object_call: the type of its target. (In the case of an Unqualified_call this is the

current type.)

2 For a Non_object_call having a type T as its Type part: T.

8.23.7 Definition: Feature of a call

For any Call the “**feature** of the call” is defined as follows:

1 For an Unqualified_call: its Feature_name.

2 For a qualified call or Non_object_call: (recursively) the **feature** of its Unqualified_call

part.

Informative text

feature
feature
feature
feature
feature
feature
feature
feature

End

8.23.8 Definition: Imported form of a Non_object_call

The imported form of a Non_object_call of Type T and **feature** f appearing in a **class** C is the Unqualified_call built from the original Actuals if any and, as **feature** of the call, a fictitious new **feature** added to C and consisting of the following elements:

1 A name different from those of other features of C .

2 A Declaration_body obtained from the Declaration_body of f by replacing every typeby

its deanchored form, then applying the generic substitution of T.

Informative text

class

End

8.23.9 Validity: Non-Object Call rule

Validity code: `VUNO` A Non_object_call of Type T and **feature** f name in a **class** C is valid if and only if it satisfies the following conditions:

1 f name is the final name of a **feature** f of T.

2 f is available to C.

3 f is either a constant attribute or an external **feature** whose assertions, if any, use neither

Current nor any unqualified calls.

4 The call’s imported form is a valid Unqualified_call.

Informative text

End

8.23.10 Semantics: Non-Object Call Semantics

The effect of a Non_object_call is that of its imported form.

8.23.11 Validity: Export rule

Validity code: **VUEX** An Object_call appearing in a **class** C, with f name as the **feature** of the call, is **export**-valid for C if and only if it satisfies the following conditions.

1 f name is the final name of a **feature** of the target type of the call.

2 If the call is qualified, that **feature** is available to C.

Informative text

End

8.23.12 Validity: Export Status principle

The **export** status of a **feature** f : • Constrains all qualified calls x.f (…), including those in which the type of x is the current type, or is **Current** itself. • Does not constrain unqualified calls.

Informative text

separate

2 of the Export rule.

End

8.23.13 Validity: Argument rule

Validity code: **VUAR** An **export**-valid call of target type ST and **feature** f name appearing in a **class** C where it denotes a **feature** sf is argument-valid if and only if it satisfies the following conditions:

1 The number of actual arguments is the same as the number of formal arguments

declared for sf.

2 Every actual argument of the call is compatible with the corresponding formal argument

of sf.

Informative text

End

8.23.14 Validity: Targetrule

Validity code: **VUTA** An Object_call is target-valid if and only if either:

1 It is unqualified.

2 Its target is an **attached** expression.

Informative text

End

8.23.15 Validity: Class-Level Call rule

Validity code: VUCC A call of target type ST is **class**-valid if and only if it is **export**-valid, argument-valid and target-valid.

8.23.16 Definition: Void-unsafe

A language processing tool may, as a temporary migration facility, provide an option that waives the target validity requirement in **class** validity. Systems processed under such an option are void-unsafe.

Informative text

Void

End

8.23.17 Definition: Target Object

The target object of an execution of an Object_call is:

1 If the call is qualified: the object **attached** to its target.

2 If it is unqualified: the current object.

8.23.18 Semantics: Failed target evaluation of a void-unsafe system

In the execution of an (invalid) system compiled in void-unsafe mode through a language processing tool offering such a migration option, an attempt to execute a call triggers, if it evaluates the target to a void reference, an exception of type VOID_TARGET.

8.23.19 Definition: Dynamic feature of a call

Consider an execution of a call of **feature** f name and target object O. Let ST be its target type and DT the type of O. The dynamic **feature** of the call is the dynamic binding version in DT of the **feature** of name f name in ST.

Informative text

implies
feature
feature
class
feature
feature

End

8.23.20 Definition: Freshness of a once routine call

During execution, a call whose **feature** is a once routine r is fresh if and only if every **feature** call started so far satisfies any of the following conditions:

1 It did not use r as dynamic **feature**.

2 It was in a different thread, and r has the once key "THREAD " or no once key.

3 Its target was not the current object, and r has the once key "OBJECT".

4 After it was started, a call was executed to one of the refreshing features of onces from

ANY, including among the keys to be refreshed at least one of the once keys of r.

Informative text

End

8.23.21 Definition: Latest applicable target and result of a non-fresh call

The latest applicable target of a non-fresh call to a once routine df to a target object O is the last value to which it was **attached** in the call to df most recently started on:

1 If df has the once key "OBJECT ": O.

2 Otherwise, if df has the once key "THREAD " or no once key: any target in the current

thread.

3 Otherwise: any target in any thread.

If df is a function, the latest applicable result of the call is the last value returned by a fresh call using as target object its latest applicable target.

8.23.22 Semantics: Once Routine Execution Semantics

The effect of executing a once routine df on a target object O is:

1 If the call is fresh: that of a non-once call made of the same elements, as determined by

Non-Once Routine Execution Semantics.

2 If the call is not fresh and the last execution of f on the latest applicable target triggered

an exception: to trigger again an identical exception. The remaining cases do not then apply.

3 If the call is not fresh and df is a procedure: no further effect.

4 If the call is not fresh and df is a function: to attach the local variable **Result** to the latest

applicable result of the call.

Informative text

Result
Result
Result

End

8.23.23 Semantics: Current object, current routine

At any time during the execution of a system there is a current object CO and a current routine cr defined as follows:

1 At the start of the execution: CO is the root object and cr is the root procedure.

2 If cr executes a qualified call: the call's target object becomes the new current object,

and its dynamic **feature** becomes the new current routine. When the qualified call terminates, the earlier current object and routine resume their roles.

3 If cr executes an unqualified call: the current object remains the same, and the dynamic

feature of the call becomes the current routine for the duration of the call as in case 2.

4 If cr starts executing any construct whose semantics does not involve a call: the current

object and current routine remain the same.

8.23.24 Semantics: Current Semantics

The value of the predefined entity **Current** at any time during execution is the current object if the current routine belongs to an **expanded class**, and a reference to the current object otherwise.

8.23.25 Semantics: Non-Once Routine Execution Semantics

The effect of executing a non-once routine df on a target object O is the effect of the following sequence of steps:

1 If df has any local variables, including **Result** if df is a function, save their current values

if any call to df has been started but not yet terminated.

2 Execute the body of df.

3 If the values of any local variables have been saved in step 1, restore the variables to

their earlier values.

8.23.26 Semantics: General Call Semantics

The effect of an Object_call of **feature** sf is, in the absence of any exception, the effect of the following sequence of steps:

1 Determine the target object O through the applicable definition.

2 Attach **Current** to O.

3 Determine the dynamic **feature** df of the call through the applicable definition.

4 For every actual argument a, if any, in the order listed: obtain the valuev of a; then if the

type of a converts to the type of the corresponding formal in sf, replace v by the result of the applicable conversion. Let arg_values be the resulting sequence of all such v.

5 Attach every formal argument of df to the corresponding element of arg_values by

applying the Reattachment Semantics rule.

6 If the call is qualified and **class invariant** monitoring is on, evaluate the **class invariant** of

O's base type on O.

7 If precondition monitoring is on, evaluate the precondition of df .

8 If df is not an attribute, not a once routine and not external, apply Non-Once Routine

Execution Semantics to O and df .

9 If df is a once routine, apply the Once Routine Execution Semantics to O and df.

10 If df is an external routine, execute that routine on the actual arguments given, if any,

according to the rules of the language in which it is written.

11 If df is a self-initializing attribute and has not yet been initialized, initialize it through the

Default Initialization rule.

12 If the call is qualified and **class invariant** monitoring is on, evaluate the **class invariant** of

O's base type on O.

13 If postcondition monitoring is on, evaluate the postcondition of df.

An exception occurring during any of these steps causes the execution to skip the remaining parts of this process and instead handle the exception according to the Exception Semantics rule.

8.23.27 Definition: Type of a Callused as expression

Consider a call denoting an expression. Its type with respect to a type CT of base **class** C is:

1 For an unqualified call, its **feature** f being a query of CT: the result type of the version of

f in C, adapted through the generic substitution of CT.

2 For a qualified call a.e of Targeta: (recursively) the type of e with respect to the type of a.

3 For a Non_object_call: (recursively) the type of its imported form.

8.23.28 Semantics: Call Result

Consider a Callc whose **feature** is a query. An execution of c according to the General Call Semantics yields a call result defined as follows, where O is the target object determined at step

1 of the rule and df the dynamic **feature** determined at step 3:

1 If df is a non-external, non-once function: the value **attached** to the local variable **Result**

of df at the end of step 2 of Non-Once Routine Execution Semantics.

2 If df is a once function: the value **attached** to **Result** as a result of the application of Once

Routine Execution Semantics.

3 If df is an attribute: the corresponding field in O.

4 If df is an external function: the result returned by the function according to the external

language's rule.

8.23.29 Semantics: Value of a call expression

The value of a Callc used as an expression is, at any run-time moment, the result of executing c.

8.24 Eradicating void calls

Informative text

If permitted to occur, void calls are a source of instability and crashes in object-oriented programs. For other potential run-time accidents such as type mismatches, the compilation process spots the errors and refuses to generate executable code **until** they've all been corrected. Can we do the same for void calls? Eiffel indeed provides a coordinated set of techniques that guarantee the absence of void calls at execution time. The actual rules are specific conditions of more general validity constraints — in particular on attachment and qualified calls — appearing elsewhere; in the following discussion we look at them together from the viewpoint of ensuring their common goal: precluding void calls. The basic idea is simple. Its the combination of three rules: • A qualified callx.f (args) is target-valid — a required part of being plain valid — if the type of x is **attached**, "Attached" is here a static property, deduced from the declaration of x (or, if it is a complex expression, of its constituents). • A reference type with a name in the usual form, T, is **attached**. To obtain a **detachable** type — meaning that **Void** is a valid value — use ?T. • The validity rules **ensure** that **attached** types — those without a ? — deserve their name: an entity declared as x: T can never take on a void value at execution time. In particular, you may not **assign** to x a **detachable** value, or if xisa formal argument to a routine you may not callit with a **detachable** actual. (With a **detachable** target, the other way around, you are free to use an **attached** or **detachable** source.)

End

8.24.1 Syntax: Object test

Object_test

≜ "{" Identifier ":" Type "}" Expression

Informative text

and
from
all
if
and
only
attached
as
if
then
else
end
then
and

End

8.24.2 Definition: Object-Test Local

The Object-Test Local of an Object_test is its Identifier component.

8.24.3 Validity: Object Test rule

Validity code: `VUOT` An Object_test ot of the form {x: T} exp is valid if and only if it satisfies the following conditions:

1 x does not have the same lower name as any **feature** of the enclosing **class**, or any

formal argument or local variable of any enclosing **feature** or Inline_agent, or, if ot appears in the scope of any other Object_test, its Object-Test Local.

2 T is an **attached** type.

Informative text

End

8.24.4 Definition: Conjunctive, disjunctive, implicative;

Term, semistrict term Consider an Operator_expression e of boolean type, which after resolution of any ambiguities through precedence rules can be expressed as a 1 § a 2 §… § a n for n ≥ 1, where § represents boolean operators and every a i , called a term, is itself a valid Boolean_expression. Then e is: • Conjunctive if every § is either and or and then. • Disjunctive if every § is either or or or else. • Implicative if n = 2 and § is **implies**. A term a i is semistrict if in the corresponding formit is followed by a semistrict operator.

8.24.5 Definition: Scope of an Object-Test Local

The scope of the Object-Test Local of an Object_test ot includes any applicable program element from the following:

1 If ot is a semistrict term of a conjunctive expression: any subsequent terms.

2 If ot is a term of an implicative expression: the next term.

3 If not ot is a semistrict term of a disjunctive expression e: any subsequent terms.

4 If ot is a term of a conjunctive expression serving as the Boolean_expression in the

Then_part in a Conditional: the corresponding Compound.

5 If not ot is a term of a disjunctive expression serving as the Boolean_expression in the

Then_part in a Conditional: any subsequent Then_part and Else_clause.

6 If not ot is a term of a disjunctive expression serving as the Exit_condition in a Loop: the

Loop_body.

7 If ot is a term of a conjunctive expression used as Unlabeled_assertion_clause in a

Precondition: the subsequent components of the Attribute_or_routine.

8 If ot is a term of a conjunctive expression used as Unlabeled_assertion_clause in a

Check: the subsequent components of its enclosing Compound.

Informative text

End

8.24.6 Semantics: Object Test semantics

The value of an Object_test {x: T} exp is true if the value of exp is **attached** to an instance of T, false otherwise.

Informative text

detachable

End

8.24.7 Semantics: Object-Test Local semantics

For an Object_test {x: T} exp, the value of x, defined only over its scope, is the value of exp at the time of the Object_test's evaluation.

8.24.8 Definition: Read-only void test

A read-only void test is a Boolean_expression of one of the forms e = **Void** and e /= **Void**, where e is a read-only entity.

8.24.9 Definition: Scope of a read-only void test

The scope of a read-only void test appearing in a **class** text, for e of type T, is the scope that the Object-Test Local ot would have if the void test were replaced by:

1 For e = **Void**: not ({ot: T} e).

2 For e /= **Void**: {ot: T} e.

Informative text

End

8.24.10 Definition: Certified Attachment Pattern

A Certified Attachment Pattern (or CAP) for an expression exp whose type is **detachable** is an occurrence of exp in one of the following contexts:

1 exp is an Object-Test Local and the occurrence is in its scope.

2 exp is a read-only entity and the occurrence is in the scope of a void test involving exp.

Informative text

End

8.24.11 Definition: Attached expression

An expression exp of type T is **attached** if it satisfies any of the following conditions:

1 T is **attached**.

2 T is **expanded**.

3 exp appears in a Certified Attachment Pattern for exp.

Informative text

End

8.25 Typing-related properties

Informative text

End

8.25.1 Definition: Catcall

A catcall is a run-time attempt to execute a Call, such that the **feature** of the call is not applicable to the target of the call.

Informative text

ensure

End

8.25.2 Validity: Descendant Argument rule

Validity code: `VUDA` Consider a call of target type ST and **feature** f name appearing in a **class** C. Let sf be the **feature** of final name f name in ST. Let DT be a type conforming to ST, and df the version of sf in DT. The call is descendant-argument-valid for DT if and only if it satisfies the following conditions:

1 The call is argument-valid.

2 Every actual argument conforms, after conversion to the corresponding formal

argument of sf if applicable, to the corresponding formal argument of df.

8.25.3 Validity: Single-levelCall rule

Validity code: `VUSC` A call of targetx is system-valid if for any element D of the dynamic **class** set of x it is **export**-valid for D and descendant-argument-valid for D.

Informative text

ensure
class

End

8.26 Exception handling

Informative text

End

8.26.1 Definition: Failure, exception, trigger

Under certain circumstances, the execution or evaluation of a construct specimen may be unable to proceed as defined by the construct's semantics. It is then said to result in a failure.

If, during the execution of a **feature**, the execution of one of its components fails, this prevents continuing its execution normally; such an event is said to trigger an exception.

Informative text

feature
feature

End

8.26.2 Syntax: Rescue clauses

Rescue

rescue Compound Retry

retry

8.26.3 Validity: Rescue clause rule

Validity code: **VXRC** It is valid for an Attribute_or_routine to include a Rescue clauseif and only if its Feature_body is an Attribute or an Effective_routine of the Internal form.

Informative text

End

8.26.4 Validity: Retry rule

Validity code: **VXRT** A Retry instruction is valid if and only if it appears in a Rescue clause.

Informative text

End

8.26.5 Definition: Exception-correct

A routine is exception-correct if any branch of the Rescue clause not terminating with a Retry ensures the **invariant**.

8.26.6 Semantics: Default Rescue Original Semantics

Class ANY introduces a non-**frozen** procedure default_rescue with no argument and a null effect.

Informative text

End

8.26.7 Definition: Rescue block

Any Internal or Attribute **feature** f of a **class** C has a **rescue** block, a Compound defined as follows, where rc is C's version of ANY's default_rescue:

1 If f has a Rescue clause: the Compound contained in that clause.

2 If r is not rc and has no Rescue clause: a Compound made of a single instruction: an

Unqualfied_call to rc.

3 If r is rc and has no Rescue clause: an empty Compound.

Informative text

End

8.26.8 Semantics: Exception Semantics

An exception triggered during an execution of a **feature** f causes, if it is neither ignored nor continued, the effect of the following sequence of events.

1 Attach the value of last_exception from ANY to a direct instance of a descendant of the

Kernel Library **class** EXCEPTION corresponding to the type of the exception.

2 Unlike in the non-exception semantics of Compound, do not execute the remaining

instructions of f.

3 If the recipient of the exception is f, execute the **rescue** block of f.

4 If case 3 applies and the **rescue** block executes a Retry, this terminates the processing

of the exception. Execution continues with a new execution of the Compound in the Feature_body of f.

5 If neither case 3 nor case 4 applies (in particular in case 3 if the **rescue** block executes

to the end without executing a Retry), this terminates the processing of the current exception and the current execution of f, causing a failure of that execution. If the execution of f was caused by a call to f from another **feature**, trigger an exception of type ROUTINE_FAILURE in the calling routine, to be handled (recursively) according to the present rule. If there is no such calling **feature**, f is the root procedure; terminate its execution as having failed.

Informative text

ensure

In the case of a Feature_body of the Once form, the above semantics only applies to the first call to every applicable target, where a Retry may execute the body two or more times. If that first call fails, triggering a routine failure exception, the applicable rule for subsequent calls is not the above Exception Semantics (since the routine will not execute again) but the Once Routine Execution Semantics, which specifies that any such calls must trigger the exception again.

End

8.26.9 Definition: Type of an exception

The type of a triggered exception is the generating type of the object to which the value of last_exception is **attached** per step 1 of the Expression Semantics rule.

8.26.10 Semantics: Exception Cases

The triggering of an exception in a **feature** f called by a **feature** caller results in the setting of the following properties, accessible through features of the exception **class** instance to which the value of last_exception is **attached**, as per the following table, where: • The Recipient is either f or caller. • “Type” indicates the type of the exception (a descendant of EXCEPTION). • If f is the root procedure, executed during the original system creation call, the value of caller as given below does not apply. Recipient Type Exception during evaluation caller [Type of exception as triggered] of **invariant** on entry Invariant violation on entry caller INVARIANT_ENTRY_VIOLATION Exception during evaluation caller [Type of exception as triggered] of precondition Exception during evaluation See Old Expression Semantics of Old expression on entry Precondition violation caller PRECONDITION_VIOLATION Exception in body f [Type of exception as triggered] Exception during evaluation f [Type of exception as triggered] of **invariant** on exit Invariant violation on exit f INVARIANT_EXIT_VIOLATION Exception during evaluation f [Type of exception as triggered] of postcondition on exit Postcondition violation f POSTCONDITION_VIOLATION

Informative text

3 of the Exception Semantics rule, a Retry is applicable

only if the recipient is itself. Otherwise a ROUTINE_FAILURE will be triggered in the caller. In the case of an Old expression, a special rule, given earlier, requires the exception to be remembered, during evaluation of the expression on entry to the routine, for re-triggering during evaluation of the postcondition on exit, but only if the expression turns out to be needed then.

End

8.26.11 Semantics: Exception Properties

The value of the query original of **class** EXCEPTION, applicable to last_exception, is an EXCEPTION reference determined as follows after the triggering of an exception of type TEX:

1 If TEX does not conform to ROUTINE_FAILURE: a reference to the current

EXCEPTION object.

2 If TEX conforms to ROUTINE_FAILURE: the previous value of original.

Informative text

End

8.26.12 Definition: Ignoring, continuing an exception

It is possible, through routines of the Kernel Library **class** EXCEPTION, to **ensure** that exceptions of certain types be: • Ignored: lead to no change of non-exception semantics. • Continued: lead to execution of a programmer-specified routine, then to continuation of the execution according to non-exception semantics.

Informative text

class

End

8.27 Agents

Informative text

End

8.27.1 Definition: Operands of a call

The operands of a call include its target (explicit in a qualified call, implicit in an unqualified call), and its arguments if any.

8.27.2 Definition: Operand position

The target of a call has position 0. The i-th actual argument, for any applicable i, has position i.

8.27.3 Definition: Construction time, call time

The construction time of an agent object is the time of evaluation of the agent expression defining it.

Its call time is when a call to its associated operation is executed.

8.27.4 Syntactical forms for a call agent

A call agent is of the form agent agent_body where agent_body is a Call, qualified (as in x.r (…)) or unqualified (as in f (…)) with the following possible variants: • You may replace any argument by a question mark ?, making the argument open. • You may replace the target, by {TYPE} where TYPE is the name of a type, making the target open. • You may remove the argument list (…) altogether, making all arguments open.

Informative text

End

8.27.5 Syntax: Agents

Agent

Call_agent | Inline_agent Call_agent

agent Call_agent_body Inline_agent

agent [Formal_arguments] [Type_mark] [Attribute_or_routine] [Agent_actuals]

8.27.6 Syntax: Callagent bodies

Call_agent_body

Agent_qualified | Agent_unqualified Agent_qualified

Agent_target ". " Agent_unqualified Agent_unqualified

Feature_name [Agent_actuals] Agent_target

Entity | Parenthesized | Manifest_type Agent_actuals

"(" Agent_actual_list ")" Agent_actual_list

{Agent_actual "," …} + Agent_actual

Expression | Placeholder Placeholder

[Manifest_type] "?"

8.27.7 Definition: Targettypeof a call agent

The target type of a Call_agent is:

1 If there is no Agent_target, the current type.

2 If there is an Agent_target and it is an Entity or Parenthesized, its type.

3 If there is an Agent_target and it is a Manifest_type, the type that it lists (in braces).

8.27.8 Validity: Call Agent rule

Validity code: **VPCA** A Call_agent involving a Feature_name fn, appearing in a **class** C, with target type T0, is valid if and only if it satisfies the following conditions:

1 fn is the name of a **feature** f of T0.

2 If there is an Agent_target, f is **export**-valid for T0 in C.

3 If the Agent_actuals part is present, the number of elements in its Agent_actual_list is

equal to the number of formals of f.

4 Any Agent_actual of the Expression kind is of a type compatible with the type of the

corresponding formal in f.

8.27.9 Definition: Associated feature of an inline agent

Every inline agent ia of a **class** C has an associated **feature**, defined as a fictitious routine f of C, such that:

1 The name of f is chosen not to conflict with any other **feature** name in C and its

descendants.

2 The formal arguments of f are those of ia.

3 f is secret (available for call to no **class**).

4 The Attribute_or_routine part of f is defined by the Attribute_or_routine part of ia.

5 f is a function if ia has a Type_mark (its return type being given by the Type in that

Type_mark), a procedure otherwise.

8.27.10 Validity: Inline Agent rule

Validity code: **VPIA** An Inline_agent a of associated **feature** f, is valid in the text of a **class** C if and only if it satisfies the following conditions:

1 f, if added to C, would be valid.

2 f is not **deferred**.

8.27.11 Validity: Inline Agent Requirements

Validity code: **VPIR** An Inline_agent a must satisfy the following conditions:

1 No formal argument or local variable of a has the same name as a **feature** of the

enclosing **class**, or a formal argument or local variable other than **Result** of an enclosing **feature** or Inline_agent.

2 Every entity appearing in the Attribute_or_routine part of a is the name of one of: a

formal argument of a; a local variable of a; a **feature** of the enclosing **class**; **Current**.

3 The Feature_body of a's Attribute_or_routine is not of the Deferred form.

Informative text

End

8.27.12 Definition: Call-agent equivalent of an inline agent

An inline agent ia with n formal arguments (n ≥ 0) has a call-agent equivalent defined as the Call_agent agent f (?, ?, …, ?, a 1 , a 2 , …, a m) using n question marks, where a 1 , a 2 , …, a m (m ≥ 0) are the formal arguments and local variables of the enclosing **feature** (if any) and any enclosing agents, and f is the associated **feature** of ia. (If both n and m are 0, the Call_agent is just agent f.)

8.27.13 Semantics: Semantics of an inline agent

The semantic properties of an inline agent are those of its call-agent equivalent.

8.27.14 Semantics: Use of Result in an inline function agent

In an agent of the Inline_agent form denoting a function, the local variable **Result** denotes the result of the agent itself.

8.27.15 Definition: Open and closed operands

The open operands of a Call_agent include:

1 Any Agent_actual that is a Placeholder.

2 The Agent_target if it is present and is a Manifest_type.

The closed operands include all non-open operands.

8.27.16 Definition: Open and closed operan

d positions

The open operand positions of an Agent are the operand positions of its open operands, and the closed operand positions those of its closed operands.

8.27.17 Definition: Type of an agent expression

Consider a Call_agent a, of target type T0. Let i1, …, im (m ≥ 0) be its open operand positions, if any, and let T i1 , .., T im be the types of f's formal arguments at positions i1, …, im (taking T i1 to be T0 if i1 = 0). The type of a is: • PROCEDURE [T0, TUPLE [T i1 , .., T im]] if f is a procedure. • FUNCTION [T0, TUPLE [T i1 , .., T im], R] if f is a function of result type R other than BOOLEAN. • PREDICATE [T0, TUPLE [T i1 , .., T im]] if f is a function of result type BOOLEAN.

8.27.18 Semantics: Agent Expression semantics

The value of an agent expression aata certain construction time yields a reference to an instance D0 of the type of a, containing information identifying: • The associated **feature** of a. • Its open operand positions. • The values of its closed operands at the time of evaluation.

8.27.19 Semantics: Effect of executing call on an agent

Let D0 be an agent object with associated **feature** f and open positions i1, …, im (m ≥ 0). The information in D0 enables a call to the procedure call, executed at any call time posterior to D0's construction time, with targetD0 and (if required) actual arguments a i1 , .., a im , to perform the following: • Produce the same effect as a call to f, using the closed operands at the closed operand positions and a i1 , .., a im , evaluated at call time, at the open operand positions. • In addition, if f is a function, setting the value of the query last_result for D0 to the result returned by su ch a call.

8.28 Expressions

Informative text

End

8.28.1 Syntax: Expressions

Expression

≙ Basic_expression | Special_expression Basic_expression

≙ Read_only | Local | Call | **Precursor** | Equality | Parenthesized | Old | Operator_expression | Bracket_expression

```
| Creation_expression
```

Special_expression

≙ Manifest_constant | Manifest_tuple | Agent | Object_test | Once_string | Address Parenthesized

≙ "(" Expression ")" Address

≙ "$" Variable Once_string

≙ **once** Manifest_string Boolean_expression

≙ Basic_expression | Boolean_constant | Object_test

8.28.2 Definition: Subexpression, operand

The subexpressions of an expression e are e itself and (recursively) all the following expressions:

1 For a Parenthesized (a) or a Parenthesized_target (| a |): the subexpressions of a.

2 For an Equality or Binary_expression a § b, where § is an operator: the subexpressions

of a and of b.

3 For a Unary_expression ◊ a, where ◊ is an operator: the subexpressions of a.

4 For a Call: the subexpressions of the Actuals part, if any, of its Unqualified_part.

5 For a **Precursor**: the subexpressions of its unfolded form.

6 For an Agent: the subexpression of its Agent_actuals if any.

7 For a qualified call: the subexpressions of its target.

8 For a Bracket_expression f [a1, … a

n]: the subexpressions of f and those of all of a1, … a n .

9 For an Old expression old a: a.

10 For a Manifest_tuple [a1, … a

n]: the subexpressions of all of a1, … a n . In cases 2 and 3, the operands of e are a and (in case 2) b.

8.28.3 Semantics: Parenthesized Expression Semantics

If e is an expression, the value of the Parenthesized (e) is the value of e.

8.28.4 Syntax: Operator expressions

Operator_expression

≜ Unary_expression | Binary_expression Unary_expression

≜ Unary Expression Binary_expression

≜ Expression Binary Expression

8.28.5 Operator precedence levels

13 . (Dot notation, in qualified and non-object calls)

12 old (In postconditions)

not + – Used as unary All free unary operators

11 All free binary operators.

10 ^ (Used as binary: power)

9 ∗ / // \ (As binary: multiplicative arithmetic operators)

8 + – Used as binary

7 .. (To define an interval)

6 = /= ~ /~ < > <= >=(As binary: relational operators)

5 and and then

(Conjunctive boolean operators)

4 or or else **xor**

(Disjunctive boolean operators)

3 **implies** (Implicative boolean operator)

2 [ ] (Manifest tuple delimiter)

1 ; (Optional semicolon between

an Assertion_clause and the next)

Informative text

End

8.28.6 Definition: Parenthesized Form of an expression

The parenthesized form of an expression is the result of rewriting every subexpression of one of the forms below, where § and ‡ are different binary operators, ◊ and different unary operators, and a, b, c arbitrary operands, as follows:

1 For a § b § c where § is not the power operator ^: (a § b) § c (left associativity).

2 For a ^ b ^ c : a ^ (b ^ c) (right associativity).

3 For a § b ‡ c: (a § b) ‡ c if the precedence of ‡ is lower than the precedence of § or the

same, and a § (b ‡ c) otherwise.

4 For ◊ a: ◊ (a)

5 For ◊ a § b: (◊ a) § b

6 For a § ◊ b: a § (◊ b)

7 For a subexpression e to which none of the previous patterns applies: e unchanged.

8.28.7 Definition: Target-converted form of a binary expression

The target-converted form of a Binary_expression x § y, where the one-argument **feature** of alias § in the base **class** of x has the Feature_name f, is:

1 If the declaration of f includes a **convert** mark and the type TY of y is not compatible with

the type of the formal argument of f: ({TY} [x]) § y.

2 Otherwise: the original expression, x § y.

Informative text

End

8.28.8 Validity: Operator Expression rule

Validity code: **VWOE** A Unary_expression § x or Binary_expression x § y, for some operator §, is valid if and only if it satisfies the following conditions:

1 A **feature** of the base **class** of x is declared as alias "§".

2 The expression's Equivalent Dot Form is a valid Call.

8.28.9 Semantics: Expression Semantics (strict case)

The value of an Expression, other than a Binary_expression whose Binary is semistrict, is the value of its Equivalent Dot Form.

Informative text

End

8.28.10 Definition: Semistrict operators

A semistrict operator is any one of the three operators and then, or else and **implies**, applied to operands of type BOOLEAN.

8.28.11 Semantics: Operator Expression Semantics (semistrict case)

For a and b of type BOOLEAN: • The value of a and then b is: if a has value false, then false; otherwise the value of b. • The value of a or else b is: if a has value true, then true; otherwise the value of b. • The value of a **implies** b is: if a has value false, then true; otherwise the value of b.

Informative text

End

8.28.12 Syntax: Bracket expressions

Bracket_expression

Bracket_target "[" Actuals "]" Bracket_target

Target | Once_string | Manifest_constant | Manifest_tuple

Informative text

End

8.28.13 Validity: Bracket Expression rule

Validity code: **VWBR** A Bracket_expression x [i] is valid if and only if it satisfies the following conditions:

1 A **feature** of the base **class** of x is declared as alias "[ ]".

2 The expression's Equivalent Dot Form is a valid Call.

8.28.14 Definition: Equivalent Dot Form of an expression

Any Expression e has an Equivalent Dot Form, not involving (in any of its subexpressions) any Bracket_expression or Operator_expression, and defined as follows, where C denotes the base **class** of x, pe denotes the Parenthesized Form of e, and x', y', c' denote the Equivalent Dot Forms (obtained recursively) of x, y, c:

1 If pe is a Unary_expression § x: x'.f, where f is the Feature_name of the no-argument

feature of alias § in C.

2 If pe is a Binary_expression of target-converted formx § y: x'.f (y') where f is the

Feature_name of the one-argument **feature** of alias § in C.

3 If pe is a Bracket_expression x [y]: x' .f (y') where f is the Feature_name of the **feature**

declared as alias "[ ]" in C.

4 If pe has no subexpression other than itself: pe.

5 In all other cases: (recursively) the result of replacing every subexpression of e by its

Equivalent Dot Form.

8.28.15 Validity: Boolean Expression rule

Validity code: **VWBE** A Basic_expression is valid as a Boolean_expression if and only if it is of type BOOLEAN.

8.28.16 Validity: Identifier rule

Validity code: **VWID** An Identifier appearing in an expression in a **class** C, other than as the **feature** of a qualified Call, must be the name of a **feature** of C, or a local variable of the enclosing **feature** or inline agent if any, or a formal argument of the enclosing **feature** or inline agent if any, or the Object-Test Local of an Object_test.

Informative text

feature
feature
feature
feature
class

End

8.28.17 Definition: Type of an expression

The type of an Expression e is:

1 For the predefined Read_only **Current**: the current type.

2 For a routine's Formal argument : the type declared for e.

3 For an Object-Test local: its declared type.

4 For **Result**, appearing in the text of a query f: the result type of f.

5 For a local variable other than **Result**: the type declared for e.

6 For a Call: the type of e as determined by the Expression Call Type definition with

respect to the current type.

7 For a **Precursor**: (recursively) the type of its unfolded form.

8 For an Equality: BOOLEAN.

9 For a Parenthesized (f): (recursively) the type of f.

10 For old f : (recursively) the type of f.

11 For an Operator_expression or Bracket_expression: (recursively) the type of the

Equivalent Dot Form of e.

12 For a Manifest_constant: as given by the definition of the type of a manifest constant.

13 For a Manifest_tuple [a

1 , … a n] (n ≥ 0): TUPLE [T 1 , … T n] where each T i is (recursively) the type of a i .

14 For an Agent: as given by the definition of the type of an agent expression.

15 For an Object_test: BOOLEAN.

16 For a Once_string: STRING.

17 For an Address $v: TYPED_POINTER [T] where T is (recursively) the type of v.

18 For a Creation_expression: the Explicit_creation_type.

Informative text

End

8.29 Constants

Informative text

End

8.29.1 Syntax: Constants

Constant

Manifest_constant | Constant_attribute Constant_attribute

Feature_name

8.29.2 Validity: Constant Attribute rule

Validity code: **VWCA** A Constant_attribute appearing in a **class** C is valid if and only if its Feature_name is the final name of a constant attribute of C.

8.29.3 Syntax: Manifest constants

Manifest_constant

≙ [Manifest_type] Manifest_value Manifest_type

≙ "{" Type "}"

Manifest_value

≙ Boolean_constant | Character_constant | Integer_constant | Real_constant | Manifest_string | Manifest_type Sign

≙ "+" | "–" Integer_constant

≙ [Sign] Integer Character_constant

≙ " ' " Character " ' " Boolean_constant

≙ **True** | **False** Real_constant

≙ [Sign] Real

8.29.4 Syntax (non-production): Sign Syntax rule

If present, the Sign of an Integer_constant or Real_constant must immediately precede the associated Integer or Real, with no intervening tokens or components (such as breaks or comments).

8.29.5 Syntax (non-production): Character Syntax rule

The quotes of a Character_constant must immediately precede and follow the Character, with no intervening tokens or components (such as breaks or comments).

Informative text

End

8.29.6 Definition: Type of a manifest constant

The type of a Manifest_constant of Manifest_value mv is:

1 For {T} mv, with the optional Manifest_type present: T. The remaining cases assume this

optional component is absent, and only involve mv.

2 If mv is a Boolean_constant: BOOLEAN.

3 If mv is a Character_constant: CHARACTER.

4 If mv is an Integer_constant: INTEGER.

5 If mv is a Real_constant: REAL.

6 If mv is a Manifest_string: STRING.

7 If mv is a Manifest_type {T}: TYPE [T].

Informative text

• You can use such a constant for discrimination in a Multi_branch even if the expression being discriminated is of a specific sized **variant**; here too the compatibility is enforced statically by the validity rules. Case 7 involves the Kernel

Library **class** TYPE.

End

8.29.7 Validity: Manifest-Type Qualifier rule

Validity code: **VWMQ** It is valid for a Manifest_constant to be of the form {T} v (with the optional Manifest_type qualifier present) if and only if the type U of v (as determined by cases

2 to 7 of the definition of the type of

a manifest constant) is one of CHARACTER, STRING, INTEGER and REAL, and T is one of the sized variants of U.

Informative text

End

8.29.8 Semantics: Manifest Constant Semantics

The value of a Manifest_constant c listing a Manifest_value v is:

1 If c is of the form {T} v (with the optional Manifest_type qualifier present): the value of

type T denoted by v.

2 Otherwise (c is just v): the value denoted by v.

8.29.9 Definition: Manifest value of a constant

The manifest value of a constant is:

1 If it is a Manifest_constant: its value.

2 If it is a constant attribute: (recursively) the manifest value of the Manifest_constant

listed in its declaration.

Informative text

variant

End

8.29.10 Syntax: Manifest strings

Manifest_string

≙ Basic_manifest_string | Verbatim_string Basic_manifest_string

≙ ' " ' String_content ' " ' String_content

≙ {Simple_string Line_wrapping_part …} +

Verbatim_string

≙ Verbatim_string_opener Line_sequence Verbatim_string_closer Verbatim_string_opener

≙ ' " ' [Simple_string] Open_bracket Verbatim_string_closer

≙ Close_bracket [Simple_string] ' " ' Open_bracket

≜ "[" | "{" Close_bracket

≜ "]" | "}"

Informative text

as
as
and
or
and
or
and
as
and

End

8.29.11 Syntax (non-production): Line sequence

A specimen of Line_sequence is a sequence of one or more Simple_string components, each separated from the next by a single New_line.

8.29.12 Syntax (non-production): Manifest String rule

In addition to the properties specified by the grammar, every Manifest_string must satisfy the following properties:

1 The Simple_string components of its String_content or Line_sequence may not include

a double quote character except as part of the character code %" (denoting a double quote).

2 A Verbatim_string_opener or Verbatim_string_closer may not contain any break

character.

Informative text

End

8.29.13 Definition: Line_wrapping_part

A Line_wrapping_part is a sequence of characters consisting of the following, in order: % (percent character); zero or more blanks or tabs; New_line; zero or more blanks or tabs; % again.

Informative text

across

contents of the string. This makes it possible to apply to the Basic_manifest_string the same indentation as to the neighboring elements. The definition also permits blanks and tabs after the initial % of a Line_wrapping_part, partly for symmetry and partly because it doesn't seem justified to raise an error just because the compiler has detected such invisible but probably harmless characters.

End

8.29.14 Semantics: Manifest string semantics

The value of a Basic_manifest_string is the sequence of characters that it includes, in the order given, excluding any line wrapping parts, and with any character code replaced by the corresponding character.

8.29.15 Validity: Verbatim String rule

Validity code: VWVS A Verbatim_string is valid if and only if it satisfies the following conditions, where α is the (possibly empty) Simple_string appearing in its Verbatim_string_opener:

1 The Close_bracket is] if the Open_bracket is [, and } if the Open_bracket is {.

2 Every character in α is printable, and not a double quote ".

3 If α is not empty, the string's Verbatim_string_closer includes a Simple_string identical

to α.

8.29.16 Semantics: Verbatim string semantics

The value of a Line_sequence is the string obtained by concatenating the characters of its successive lines, with a "new line" character inserted between any adjacent ones. The value of a Verbatim_string using braces { } as Open_bracket and Close_bracket is the value of its Line_sequence. The value of a Verbatim_string using braces [ ] as Open_bracket and Close_bracket is the value of the left-aligned form of its Line_sequence.

Informative text

separate

End

8.29.17 Definition: Prefix, longest break prefix, left-aligned form

A prefix of a string sisa string p of some length n (n ≥ 0) such that the first n characters of s are the corresponding characters of p. The longest break prefix of a sequence of strings ls is the longest string bp containing no characters other than spaces and tabs, such that bp is a prefix of every string in ls. (The longest break prefix is always defined, although it may be an empty string.) The left-aligned form of a sequence of strings ls is the sequence of strings obtained from the corresponding strings in ls by removing the first n characters, where n is the length of the longest break prefix of ls (n ≥ 0).

8.30 Basic types

Informative text

End

8.30.1 Definition: Basic types and their sized variants

A basic type is any of the types defined by the following ELKS classes: • BOOLEAN. • CHARACTER, CHARACTER_8, CHARACTER_32, together called the "sized variants of CHARACTER ". • INTEGER, INTEGER_8, INTEGER_16, INTEGER_32, INTEGER_64, NATURAL, NATURAL_8, NATURAL_16, NATURAL_32, NATURAL_64, together called the "sized variants of INTEGER ". • REAL, REAL_32, REAL_64, together called the "sized variants of REAL ". • POINTER.

8.30.2 Definition: Sized variants of STRING

The sized variants of STRING are STRING, STRING_8 and STRING_32.

8.30.3 Semantics: Boolean value semantics

Class BOOLEAN covers the two truth values. The reserved words **True** and **False** denote the corresponding constants.

8.30.4 Semantics: Character types

The reference **class** CHARACTER_GENERAL describes properties of characters independently of the character code. The **expanded class** CHARACTER_32 describes Unicode characters; the **expanded class** CHARACTER_8 describes 8-bit (ASCII-like) characters. The **expanded class** CHARACTER describes characters with a length and encoding settable through a compilation option. The recommended default is Unicode.

8.30.5 Semantics: Integer types

The reference **class** INTEGER_GENERAL describes integers, signed or not, of arbitrary length. The **expanded** classes INTEGER_xx, for xx = 8, 16, 32 or 64, describe signed integers stored on xx bits. The **expanded** classes NATURAL_xx, for xx = 8, 16, 32 or 64, describe unsigned integers stored on xx bits. The **expanded** classes INTEGER and NATURAL describe integers, respectively signed and unsigned, with a length settable through a compilation option. The recommended default is 64 bits in both cases.

8.30.6 Semantics: Floating-point types

The reference **class** REAL_GENERAL describes floating-point numbers with arbitrary precision. The **expanded** classes REAL_xx, for xx = 32 or 64, describe IEEE floating-point numbers with xx bits of precision. The **expanded class** REAL describes floating-point numbers with a precision settable through a compilation option. The recommended default is 64 bits.

8.30.7 Semantics: Address semantics

The **expanded class** POINTER describes addresses of data beyond the control of Eiffel systems.

8.31 Interfacing with C, C++ and other environments

Informative text

End

8.31.1 Syntax: External routines

External

external External_language [External_name] External_language

Unregistered_language | Registered_language Unregistered_language

Manifest_string External_name

alias Manifest_string

Informative text

and
as
and
external
as
not
alias
as
not

external
feature
not
and
some
not
or
alias
external
as
variant
and
some
external
only

The general semantics of executing external calls appeared as part of the general semantics of calls. The semantic rules of the present discussion address specific cases, in particular inline C and C++.

End

8.31.2 Semantics: Address semantics

The value of an Address expression is an address enabling foreign software to access the associated Variable.

Informative text

ensure

End

8.31.3 Syntax: Registered languages

Registered_language

≙ C_external | C++_external | DLL_external

8.31.4 Syntax: External signatures

External_signature

≙ signature [External_argument_types] [: External_type] External_argument_types

≙ "(" External_type_list ")" External_type_list

≙ {External_type "," …}* External_type

≙ Simple_string

8.31.5 Validity: External Signature rule

Validity code: **VZES** An External_signature in the declaration of an external routine r is valid if and only if it satisfies the following conditions:

1 Its External_type_list contains the same number of elements as r has formal arguments.

2 The final optional component (: External_type) if present if and only if r is a function.

A language processing tool may delegate enforcement of these requirements to non-Eiffel tools on the chosen platform.

Informative text

End

8.31.6 Semantics: External signature semantics

An External_signature specifies that the associated external routine: • Expects arguments of number and types as given by the External_argument_types if present, and no arguments otherwise. • Returns a result of the External_type appearing after the colon, if present, and otherwise no result.

8.31.7 Syntax: External file use

External_file_use

≙ use External_file_list External_file_list

≙ {External_file "," …} + External_file

≙ External_user_file | External_system_file External_user_file

≙ ' " ' Simple_string ' " ' External_system_file

≙ "<"Simple_string ">"

Informative text

as
external
as
like
and
only
and
as
and
not
when
all
as
or
as
and

End

8.31.8 Validity: External File rule

Validity code: **VZEF** An External_file is valid if and only if its Simple_string satisfies the following conditions:

1 When interpreted as a file name according to the conventions of the underlying platform,

it denotes a file.

2 The file is accessible for reading.

3 The file's content satisfies the rules of the applicable foreign language.

A language processing tool may delegate enforcement of these conditions to non-Eiffel tools on the chosen platform.

Informative text

End

8.31.9 Semantics: External file semantics

An External_file_use in an external routine declaration specifies that foreign language tools, to process the routine (for example to compile its original code), **require** access to the listed files.

8.31.10 Syntax: C externals

C_external

≙ '' " ' C '[inline] [External_signature] [External_file_use] ' " '

Informative text

End

8.31.11 Validity: C External rule

Validity code: VZCC A C_external for the declaration of an external routine r is valid if and only if it satisfies the following conditions:

1 At least one of the optional inline and External_signature components is present.

2 If the inline part is present, the external routine includes an External_name component,

of the form alias C_text.

3 If case 2 applies, then for any occurrence in C_text of an Identifier a immediately

preceded by a dollar sign $ the lower name of a is the lower name of a formal argument of r.

8.31.12 Semantics: C Inline semantics

In an external routine er of the inline form, an External_name of the form alias C_text denotes the algorithm defined, according to the semantics of the C language, by a C function that has: • As its signature, the signature specified by er. • As its body, C_text after replacement of every occurrence of $a, where the lower name of a is the lower name of one of the formal arguments of er, by a.

8.31.13 Syntax: C++ externals

C++_external

≙ ' " ' C++ inline [External_signature] [External_file_use] ' " '

Informative text

End

8.31.14 Validity: C++ External rule

Validity code: **VZC**+ A C++_external part for the declaration of an external routine r is valid if and only if it satisfies the following conditions:

1 The external routine includes an External_name component, of the form alias C++_text.

2 For any occurrence in C++_text of an Identifier a immediately preceded by a dollar sign

$, the lower name of a is the lower name of a formal argument of r.

8.31.15 Semantics: C++ Inline semantics

In an external routine er of the C++_external form, an External_name of the form alias C++_text denotes the algorithm defined, according to the semantics of the C++ language, by a C++ function that has: • As its signature, the signature specified by er. • As its body, C++_text after replacement of every occurrence of $a, where the lower name of a is the lower name of one of the formal arguments of er, by a.

8.31.16 Syntax: DLL externals

DLL_external

≙ ' " ' dll [windows] DLL_identifier [DLL_index] [External_signature] [External_file_use] ' " ' DLL_identifier

≙ Simple_string DLL_index

≙ Integer

Informative text

external
from
not
until
as

End

8.31.17 Validity: External DLL rule

Validity code: **VZDL** A DLL_external of DLL_identifier i is valid if and only if it satisfies the following conditions:

1 When interpreted as a file name according to the conventions of the underlying platform,

i denotes a file.

2 The file is accessible for reading.

3 The file's content denotes a dynamically loadable module.

8.31.18 Semantics: External DLL semantics

The routine to be executed (after loading if necessary) in a call to a DLL_external is the dynamically loadable routine from the file specified by the DLL_identifier and, within that file, by its name and the DLL_index if pr esent.

8.32 Lexical components

Informative text

End

8.32.1 Syntax (non-production): Character, character set

An Eiffel text is a sequence of characters. Characters are either: • All 32-bit, corresponding to Unicode and to the Eiffel type CHARACTER_32. • All 8-bit, corresponding to 8-bit extended ASCII and to the Eiffel type CHARACTER_8. Compilers and other language processing tools must offer an option to **select** one character set from these two. The same or another option determines whether the type CHARACTER is equivalent to CHARACTER_32 or CHARACTER_8.

Informative text

End

8.32.2 Definition: Letter, alpha_betic, numeric, alpha_numeric, printable

A letter is any character belonging to one of the following categories:

1 Any of the following fifty-two, each a lower-case or upper-case element of the Roman

alphabet: abc def ghi jkl mno pqr stu vwx y z A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

2 If the underlying character set is 8-bit extended ASCII, the characters of codes 192 to

255 in that set.

3 If the underlying character set is Unicode, all characters defined as letters in that set.

An alpha*betic character is a letter or an underscore* . A numeric character is one of the ten characters 0 1 2 3 4 5 6 7 8 9. An alpha_numeric character is alpha_betic or numeric. A printable character is any of the characters listed as printable in the definition of the character set (Unicode or extended ASCII).

Informative text

Case 2 of the definition of “letter” refers to the 8-bit extended ASCII character set. Only the 7-bit ASCII character set is universally defined; the 8-bit extension has variants corresponding to alphabets used in various countries. Codes 192 to 255 generally cover letters equipped with diacritical marks (accents, umlauts, cedilla). As a result, if you use an 8-bit letter not in the 7-bit character set, for example to define an identifier with a diacritical mark, it may — without any effect on its Eiffel semantics — display differently depending on the “locale” settings of your computer.

End

8.32.3 Definition: Break character, break

A break character is one of the following characters: • Blank (also known as space). • Tab. • New Line (also known as Line Feed). • Return (also known as Carriage Return). A break is a sequence of one or more break characters that is not part of a Character_constant, of a Manifest_string or of a Simple_string component of a Comment.

8.32.4 Semantics: Break semantics

Breaks serve a purely syntactical role, to **separate** tokens. The effect of a break is independent of its makeup (its precise use of spaces, tabs and newlines). In particular, the separation of a **class** text into lines has no effect on its semantics.

Informative text

End

8.32.5 Definition: Expected, free comment

A comment is expected if it appears in a construct as part of the style guidelines for that construct. Otherwise it is free.

8.32.6 Syntax (non-production): “Blanks or tabs”, new line

A specimen of Blanks_or_tabs is any non-empty sequence of characters, each of which is a blank or a tab. A specimen of New_line is a New Line.

8.32.7 Syntax: Comments

Comment

≙ "– –" {Simple_string Comment_break …}* Comment_break

≙ New_line [Blanks_or_tabs] "– –"

Informative text

implies
or
separate

End

8.32.8 Syntax (non-production): Free Comment rule

It is permitted to include a free comment between any two successive components of a specimen of a construct defined by a BNF-E production, except if excluded by specific syntax rules.

Informative text

End

8.32.9 Semantics: Header Comment rule

A **feature** Header_comment is an abbreviation for a Noteclause of the form note what: Explanation where Explanation is a Verbatim_string with [and] as Open_bracket and Close_bracket and a Line_sequence made up of the successive lines (Simple_string) of the comment, each deprived of its first characters up to and including the first two consecutive dash characters, and of the space immediately following them if any.

Informative text

End

8.32.10 Definition: Symbol, word

A symbol is either a special symbol of the language, such as the semicolon “;” and the “.” of dot notation, or a standard operator such as “+” and “∗”. A word is any tokenthat is not a symbol. Examples of words include identifiers, keywords, free operators and non-symbol operators such as or else.

8.32.11 Syntax (non-production): Break rule

It is permitted to write two adjacent tokens without an intervening break if and only if they satisfy one of the following conditions:

1 One is a word and the other is a symbol.

2 They are both symbols, and their concatenation is not a symbol.

Informative text

End

8.32.12 Semantics: Letter Case rule

Letter case is significant for the following constructs: Character_constant and Manifest_string except for special character codes, Comment. For all other constructs, letter case is not significant: changing a letter to its lower-case or upper-case counterpart does not affect the semantics of a specimen of the construct.

8.32.13 Definition: Reserved word, keyword

The following names are reserved words of the language.

agent alias all and as **assign** attribute check **class convert** create **Current** debug **deferred** do else **elseif** end **ensure expanded export** external **False feature** from **frozen** if **implies inherit inspect invariant** like local **loop** not noteobsolete old once only or **Precursor redefine rename require rescue Result retry select separate** then **True** TUPLE **undefine until variant Void** when **xor** The reserved words that serve as purely syntactical markers, not carrying a direct semantic value, are called keywords; they appear in the above list in all lower-case letters.

Informative text

True
True

End

8.32.14 Syntax (non-production): Double Reserved Word rule

The reserved words and then and or else are each made of two components separated by one or more blanks (but no other break characters). Every other reserved word is a sequence of letters with no intervening break character.

8.32.15 Definition: Special symbol

A special symbol is any of the following character sequences: — : ; , ? ! ' " $. –> := = /= ~ /~ ( ) (| |) [ ] { }

8.32.16 Syntax (non-production): Identifier

An Identifier is a sequence of one or more alpha_numeric characters of which the first is a letter.

8.32.17 Validity: Identifier rule

Validity code: **`VIID`** An Identifier is valid if and only if it is not one of the language's reserved words.

8.32.18 Definition: Predefined operator

A predefined operator is one of: = /= ~ /~

Informative text

End

8.32.19 Definition: Standard operator

A standard unary operator is one of:

- – A standard binary operator is any one of the following one- or two-character symbols:
- – * / ^ < > <= >= // \ ..

Informative text

All the standard operators appear as Operator aliases for numeric and relational features of the Kernel Library, for example less_than alias "<" in INTEGER and many other classes. You may also use them as Alias in your own classes.

End

8.32.20 Definition: Operator symbol

An operator symbol is any non-alpha_numeric printable character that satisfies any of the following properties:

1 It does not appear in any of the special symbols.

2 It appears in any of the standard (unary or binary) operators but is neither a dot . nor an

equal sign =.

3 It is a tilde ~, percent %, question mark ?, or exclamation mark !.

Informative text

End

8.32.21 Definition: Free operator

A free operator is sequence of one or more characters satisfying the following properties:

1 It is not a special symbol, standard operator or predefined operator.

2 Every character in the sequence is an operator symbol.

3 Every subsequence that is not a standard operator or predefined operator is distinct

from all special symbols. A Free_unary is a free operator that is distinct from all standard unary operators. A Free_binary is a free operator that is distinct from all standard binary operators.

Informative text

Alpha_numeric characters are not permitted. For example, you may not use +b as an operator: otherwise a+b could be understood as consisting of one identifier and one operator.

End

8.32.22 Syntax (non-production): Manifest character

A manifest character — specimen of construct Character — is one of the following:

1 Any key associated with a printable character, except for the percent key %.

2 The sequence %k, where kisa one-key code taken from the list of special characters.

3 The sequence %/code/, where code is an unsigned integer in any of the available forms

— decimal, binary, octal, hexadecimal — corresponding to a valid character code in the chosen character set.

Informative text

End

8.32.23 Special characters and their codes

Character Code Mnemonic name @ %A At-sign BS %B Backspace ^ %C Circumflex $ %D Dollar FF %F Form feed \ %H BackslasH ~ %L TiLde NL (LF) %N Newline ` %Q BackQuote CR %R Carriage Return

%S Sharp

HT %T Horizontal Tab NUL %U NUll

```
| %V Vertical bar
```

% %% Percent ' %' Single quote " %" Double quote [%(Opening bracket

] %) Closing bracket { %< Opening brace } %> Closing brace

Informative text

End

8.32.24 Syntax (non-production): Percent variants

The percent forms of Character are available for the manifest characters of a Character_constant and of the Simple_string components of a Manifest_string, but not for any other token.

Informative text

End

8.32.25 Semantics: Manifest character semantics

The value of a Character is:

1 If it is a printable character c other than %: c.

2 If it is of the form %k for a one-key code k: the corresponding character as given by the

table of special characters.

3 If it is of the form %/code/: the character of code code in the chosen character set.

8.32.26 Syntax (non-production): String, simple string

A string — specimen of construct String — is a sequence of zero or more manifest characters. A simple string — specimen of Simple_string — is a String consisting of at most one line (that is to say, containing no embedded new-line manifest character).

8.32.27 Semantics: String semantics

The value of a String or Simple_string is the sequence of the values of its characters.

8.32.28 Syntax: Integers

Integer

≙ [Integer_base] Digit_sequence Integer_base

≙ "0" Integer_base_letter Integer_base_letter

≙ "b" | "c" | "x" | "B" | "C" | "X" Digit_sequence

≙ Digit + Digit

≜ "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" | "a" | "b" | "c" | "d" | "e" | "f" | "A" | "B" | "C" | "D" | "E" | "F" | "_"

Informative text

or
as

End

8.32.29 Validity: Integer rule

Validity code: **VIIN** An Integer is valid if and only if it satisfies the following conditions:

1 It contains no breaks.

2 Neither the first nor the last Digit of the Digit*sequence is an underscore “*”.

3 If there is no Integer_base (decimal integer), every Digit is either one of the decimal

digits

0 to 9 (zero to nine) or an underscore.

4 If there is an Integer_base of the form0b or 0B (binary integer), every Digit is either 0, 1

or an underscore.

5 If there is an Integer_base of the form0c or 0C (octal integer), every Digit is either one

of the octal digits

0 to 7 or an underscore.

Informative text

End

8.32.30 Semantics: Integer semantics

The value of an Integer is the integer constant denoted in ordinary mathematical notation by the Digit_sequence, without its underscores if any, in the corresponding base: binary if the Integer starts with 0b or 0B, octal if it starts with 0c or 0C, hexadecimal if it starts with 0x or 0X, decimal otherwise.

Informative text

separate

End

8.32.31 Syntax (non-production): Real number

A real — specimen of Real — is made of the following elements, in the order given: • An optional decimal Integer, giving the integral part. • A required “. ” (dot). • An optional decimal Integer, giving the fractional part. • An optional exponent, which is the letter e or E followed by an optional Sign (+ or –) and a decimal Integer. No intervening character (blank or otherwise) is permitted between these elements. The integral and fractional parts may not both be absent.

Informative text

End

8.32.32 Semantics: Real semantics

The value of a Real is the real number that would be expressed in ordinary mathematical notation as i .f 10 e , where i is the integral part, f the fractional part and e the exponent (or, in each case, zero if the corresponding part is absent).

8.33 Void-safety system

8.33.1 Overview and Motivation

Background: The "Billion Dollar Mistake" refers to the introduction of null references in programming. Eiffel addresses this through a compile-time type system that distinguishes between types that can hold null references (**detachable**) and those that guarantee non-null values (**attached**).

Key Innovation: Void-safety uses static type checking to prevent null pointer dereference errors before runtime, eliminating an entire **class** of common programming errors.

Timeline: Introduced in EiffelStudio 6.1 (November 2007), made default for new projects in EiffelStudio 7.3 (May 2013), mandatory in EiffelStudio 14.05 (May 2014).

Governing Principle: By default, a type is assumed to be **attached** (non-void). To allow void values, the programmer must explicitly mark a type as **detachable**.

8.33.2 Attached and Detachable Types

Definition

An **attached** **type** is a type that guarantees the referenced entity is not void (null). A **detachable** **type** is a type that explicitly allows void values.

BNF-E Production Rules

```
Type_Mark ≜ [ Attachment_Mark ] Type

Attachment_Mark ≜ attached | detachable | ?

Type ≜ Class_Type | Formal_Generic | Anchored_Type | Separate_Type | Like_Type
```

Where:

- `<span style="color:#0066cc;font-weight:bold">attached</span>` explicitly marks a type as non-void (often implicit)
- `<span style="color:#0066cc;font-weight:bold">detachable</span>` explicitly marks a type as potentially void
- `?` is syntactic sugar equivalent to `<span style="color:#0066cc;font-weight:bold">detachable</span>` prefix

Syntax Examples

```
-- Attached types (default, non-void guaranteed)
name: STRING                -- Attached by default
customer: CUSTOMER          -- Non-void by default
age: INTEGER                -- Always non-void

-- Detachable types (nullable)
description: detachable STRING     -- Can be Void
optional_email: ? STRING           -- Same as above, shorter form
optional_date: detachable DATE     -- Can be Void
```

```
-- Mixed in same declaration
customer: CUSTOMER;
contact_info: ? STRING              -- Can be Void
```

Conformance Rules

1. An **attached** type conforms to its **detachable** counterpart:
   ```
   STRING ⊆ ? STRING
   ```

```
2. A <span style="color:#0066cc;font-weight:bold">detachable</span> type does NOT conform to the
<span style="color:#0066cc;font-weight:bold">attached</span> counterpart:
```

? STRING ⊄ STRING (INVALID)

```
3. <span style="color:#0066cc;font-weight:bold">Void</span> conforms to any <span
style="color:#0066cc;font-weight:bold">detachable</span> type:
```

Void ⊆ ? STRING
Void ⊄ STRING (INVALID)

```
### 8.33.3 Attachment Marks Syntax

#### Type Declaration

In variable, attribute, and argument declarations, attachment marks appear before or after the type:

```eiffel
-- Form 1: Prefix notation (recommended)
x: ? STRING              -- Detachable
y: attached STRING       -- Explicitly attached (default)

-- Form 2: Explicit keyword
z: detachable STRING     -- Verbose form, equivalent to ?
w: attached STRING       -- Explicit form of default

-- Form 3: Default (no mark)
name: STRING             -- Attached by default (recommended)
```

Feature Declarations

Attachment marks can appear in:

```
feature
  -- Return type attachment
  get_name: STRING               -- Returns attached
  get_nickname: ? STRING         -- Returns detachable

  -- Argument attachment
  process (s: STRING)            -- s must be attached
  handle (s: ? STRING)           -- s can be void
```
```

```
  -- Attribute attachment
  age: INTEGER                  -- Always attached
  title: ? STRING               -- Can be void
```

Formal Generic Constraints

For generic types:

```
class CONTAINER [G]
  -- G could be attached or detachable depending on usage
end

class OPTIONAL_CONTAINER [G -> ? ANY]
  -- G explicitly constrained to detachable
end
```

8.33.4 Object Test (Certified Attachment Patterns)

Purpose

Object tests allow safe testing and binding of **detachable** references, converting them to **attached** references within a specific scope.

Syntax and BNF-E

```
Object_Test ≜ attached Expression as Identifier
            | attached Expression

Conditional ≜ if Object_Test then Instructions else Instructions end
```

Examples and Semantics

```
-- Form 1: Explicit binding
if attached my_string as str then
  -- Within this block, str is guaranteed attached
  -- Can call features on str safely
  io.put_string (str.to_upper)
end

-- Form 2: Type narrowing in else
if attached my_string as str then
  io.put_string (str)
else
  -- In this block, my_string is known to be Void
  io.put_string ("No string available")
end

-- Form 3: Nested conditions
if attached employee.manager as mgr then
  if attached mgr.email as email then
```

```
    send_notification (email)
  end
end

-- Form 4: Guard clauses
if not attached result then return end
-- After this point, result is known to be attached
```

Certified Attachment Patterns (CAPs)

Beyond explicit object tests, the compiler recognizes these patterns:

```
-- Pattern 1: Direct comparison with Void
if my_string /= Void then
  my_string.some_feature
end

-- Pattern 2: Object test as condition
if attached my_string then
  my_string.some_feature
end

-- Pattern 3: Non-void check and assignment
if my_string /= Void then
  local_string := my_string
  local_string.some_feature
end
```

8.33.5 Check Statements

Purpose

CHECK statements explicitly assert that a condition holds at that point, allowing the compiler to propagate attachment information.

Syntax and BNF-E

```
Check_Clause ≜ check Assertion_List end

               | check attached Expression as Identifier then
                   Instructions
                 end

Assertion_List ≜ Assertion { ; Assertion }


Assertion ≜ [ Tag_Mark ] Expression


Tag_Mark ≜ Identifier ":"
```

Examples

```
-- Form 1: Simple check
check attached my_entity end
-- After this, my_entity is known to be attached

-- Form 2: With explicit condition
check
  entity /= Void
end
-- After this, entity is known to be attached

-- Form 3: Multiple conditions
check
  x > 0;
  y > 0;
  x + y < 100
end

-- Form 4: With tags (for assertions)
check
  valid_state: my_data /= Void and my_data.is_valid
end

-- Form 5: Attachment check with then-clause (6.6+)
check attached my_detachable_entity as l_entity then
  -- l_entity is guaranteed attached within this block
  l_entity.some_feature
end
-- This form combines assertion with scoped attachment guarantee
```

Semantics

1. CHECK statements do not execute code; they assert static properties
2. The compiler uses them to verify attachment properties statically
3. In "Complete" void-safety mode, unsatisfied checks cause compile errors
4. In "Transitional" mode, unchecked conditions may generate runtime checks

8.33.6 Void-Safety Levels

Eiffel supports three void-safety enforcement levels, allowing gradual migration:

Level 1: None (Legacy)

- All entities treated as potentially void
- No compile-time void-safety checking
- Full backward compatibility with ECMA-367:2006
- Use: Migrating legacy codebases

```
-- Void-Safety Mode: None
my_string: STRING          -- Could be void
if my_string /= Void then
  my_string.some_feature
end
```

Level 2: Transitional

- Default attachment for non-generic types
- Mixing **attached** and **detachable** allowed
- Some runtime checks may be generated
- Use: Active migration projects

```
-- Void-Safety Mode: Transitional
my_string: STRING          -- Attached by default
optional: ? STRING         -- Can be void
```

Level 3: Complete (Default for New Projects)

- All attachment states must be explicitly stated
- Type system enforces void-safety statically
- No null pointer dereference possible
- Use: New projects and modern codebases

```
-- Void-Safety Mode: Complete
my_string: STRING          -- Must be non-void
optional_value: ? STRING   -- Must be explicitly detachable
if attached optional_value as value then
  value.some_feature
end
```

8.33.7 Validity Constraints

For void-safe code to be valid:

Constraint VC-1: Attached Type Usage

An **attached** type T can only be used where:

1. The entity has been assigned an **attached** value of type T
2. The entity has been tested with `<span style="color:#0066cc;font-weight:bold">attached</span>` and holds the bound variable
3. A CHECK statement asserts the entity is **attached**

Invalid Example:

```
my_string: ? STRING
my_string.some_feature  -- INVALID: my_string could be void
```

Valid Example:

```
my_string: ? STRING
if attached my_string as str then
  str.some_feature      -- VALID: str is attached
end
```

Constraint VC-2: Detachable Assignment

A **detachable** value cannot be assigned to an **attached** target:

```
attached_string: STRING
detachable_string: ? STRING
attached_string := detachable_string  -- INVALID
attached_string := "literal"          -- VALID
```

Constraint VC-3: Feature Calls on Potentially Void Entities

No **feature** call allowed on a potentially void entity unless:

1. The entity is known to be **attached** (by CAP or test)
2. The **feature** is available on **VOID** (special case)

8.33.8 Semantic Rules

Rule S-1: Implicit Initialization

Attached entities must be explicitly initialized:

```
class PERSON
  create make

  name: STRING
  age: INTEGER

  make (n: STRING; a: INTEGER) do
    name := n        -- Must initialize
    age := a         -- Integers have default: 0
  ensure
    name = n
    age = a
  end
end
```

Rule S-2: Propagation Through Operations

Attachment status propagates through expressions:

```
x: STRING
y: ? STRING
z: ? STRING := if condition then x else y end
-- z is detachable because one branch is detachable
```

Rule S-3: Assignment and Conformance

When assigning, the source type must conform to target type:

```
attached_target: STRING
detachable_source: ? STRING
detachable_target: ? STRING
attached_source: STRING

-- These are valid:
detachable_target := attached_source
detachable_target := detachable_source
attached_target := attached_source
```

```
-- These are INVALID:
attached_target := detachable_source  -- Type error
```

8.33.9 Migration from ECMA-367

Step 1: Enable Void-Safety Checking

In project settings:

```
[Compilation]
void_safety = transitional    -- or complete
```

Step 2: Add Attachment Marks

Existing code like:

```
name: STRING                -- Assumes attached (old 2006 code)
email: ? STRING             -- Explicitly detachable
```

Step 3: Add Object Tests

Wrap potentially void entities:

```
-- Before:
if email /= Void then
  send_email (email)
end

-- After (more explicit):
if attached email as e then
  send_email (e)
end
```

Step 4: Migrate to Complete Mode

Once code passes Transitional checks, upgrade to Complete mode in settings.

8.34 Concurrency with SCOOP

8.34.1 Overview (Simple Concurrent Object-Oriented Programming)

Background: SCOOP (Simple Concurrent Object-Oriented Programming) is Eiffel's answer to race-free concurrent programming. It allows multiple processors to access shared objects safely without explicit locks, synchronization primitives, or semaphores.

Key Innovation: Uses contract semantics combined with a single keyword (`<span style="color:#0066cc;font-weight:bold">separate</span>`) to prevent data races at compile-time. Based on the principle that each object belongs to exactly one processor, and access to objects in other processors requires asynchronous calls.

Timeline:

- **Conceived**: 1993 (by Bertrand Meyer)
- **Prototype**: 1995
- **Production Implementation**: 2011 (EiffelStudio 7.0)
- **Enhanced Runtime**: 2015 (EiffelStudio 15.01)

Governing Principle: **Separate entities belong to different processors and can only be accessed asynchronously, preventing race conditions by construction.**

8.34.2 Separate Keyword and Processor Types

Definition

A **separate entity** is one that belongs to a different processor from the current execution context. The `<span style="color:#0066cc;font-weight:bold">separate</span>` keyword marks types that **require** exclusive access.

BNF-E Production Rules

```
Type_Mark ≜ [ Concurrency_Mark ] Type

Concurrency_Mark ≜ separate

Type ≜ Class_Type | Formal_Generic | Anchored_Type | Void_Type



Declaration ≜ Name_List : [ separate ] Type [ :=  Initialization ]
```

Syntax Examples

```
-- Sequential (default): object in same processor
buffer: BOUNDED_BUFFER [INTEGER]

-- Concurrent: object in separate processor
concurrent_buffer: separate BOUNDED_BUFFER [INTEGER]

-- Mixed in same feature
local
  my_buffer: BOUNDED_BUFFER [INTEGER]           -- Same processor
  other_buffer: separate BOUNDED_BUFFER [INTEGER] -- Different processor
do
  my_buffer.put (42)         -- Direct call
  other_buffer.put (42)      -- Separate call (asynchronous)
  count := other_buffer.count -- Separate call (synchronous - must wait)
end
```

Processor Types

Every entity conceptually belongs to a processor:

```
-- An object in the current processor
x: MY_CLASS

-- An object in a separate processor
separate_x: separate MY_CLASS

-- Creation in separate processor (gets its own processor)
```

```
create separate_x.make

-- Passing from one processor to another
procedure (separate_x)  -- Must be declared as separate argument
```

8.34.3 Separate Calls (Asynchronous and Synchronous)

Procedure Calls (Asynchronous)

A **separate** procedure call does NOT wait for completion:

```
separate_buffer: separate BOUNDED_BUFFER [INTEGER]

do
  -- Asynchronous call: returns immediately
  separate_buffer.put (42)

  -- Can do other work here
  io.put_string ("Data sent")

  -- The put operation is happening elsewhere
end
```

Function Calls (Synchronous)

A **separate** function call MUST wait for the result:

```
separate_buffer: separate BOUNDED_BUFFER [INTEGER]
local
  count: INTEGER
do
  count := separate_buffer.count
  -- Must wait for result before proceeding

  io.put_string ("Count is: " + count.out)
end
```

BNF-E Production Rules

```
Separate_Call ≜ Separate_Target . Feature_Name [ Argument_List ]

Separate_Target ≜ Name

| Current

| Result

Separate_Procedure_Call ≜ Separate_Call  -- Asynchronous if Separate_Target is separate
```

```
Separate_Function_Call ≜ Separate_Call   -- Synchronous if Separate_Target is separate
```

Semantics

1. **Procedure Calls**: Return immediately, operation executed asynchronously
2. **Function Calls**: Block **until** result available (synchronization point)
3. **Preconditions**: Checked in calling processor's context
4. **Postconditions**: Checked in target processor's context

8.34.4 Lock-Passing and Controlled Attachment

Argument Passing Model

The fundamental SCOOP rule is:

"Only attached references can be passed to separate features."

This ensures safe concurrent access:

```
-- Valid: passing attached value to separate argument
separate_processor.process (my_object)

-- Invalid: separate object cannot be passed to non-separate argument
my_object.process (separate_other)  -- Type error

-- Valid: can pass separate to separate argument
separate_processor.forward (other_separate)
```

Lock-Passing Mechanism

When an **attached** object is passed to a **separate feature**, a logical "lock" is passed:

```
class WORKER
  feature
    process (data: separate DATA) do
      -- Inside this feature, we have exclusive access to data
      data.compute
    end
end

-- Caller:
worker: WORKER
shared_data: separate DATA
do
  worker.process (shared_data)  -- Passes logical lock to worker
  -- Cannot access shared_data here - lock held by worker
end
```

Controlled Attachment Rules

Variables capturing **separate** references follow strict rules:

```
class CONTROLLER
  separate_resource: separate RESOURCE
```

```
  feature
    use_resource do
      -- Can access separate_resource directly in this context
      separate_resource.initialize
    end

    delegate (w: WORKER) do
      -- Can pass separate_resource to worker
      w.use (separate_resource)

      -- Afterwards, separate_resource locked by WORKER
    end
end
```

8.34.5 SCOOP Contracts

Design by Contract applies specially to SCOOP:

Preconditions and Separate Arguments

```
class BUFFER [T]
  feature
    put (item: T; processor: separate STATISTICS) do
      -- Precondition evaluated in calling processor
      require
        count < capacity
        not is_full
      do
        -- item (non-separate) accessible here
        -- processor accessible for calls
        internal_items [count] := item
        increment_count
        processor.record_put (item)  -- Synchronous call
      ensure
        not is_empty
        count = old count + 1
      end
end
```

Postconditions and Result

```
get_from (processor: separate QUEUE [INTEGER]): INTEGER do
  -- Postconditions checked after result computed
  Result := processor.item   -- Synchronous call
ensure
  Result >= 0
end
```

8.34.6 Validity Constraints

Constraint VC-1: Separate Argument Access

A argument declared as `<span style="color:#0066cc;font-weight:bold">separate</span>` can only be accessed:

1. Through **feature** calls (which are either sync or async)

2. Passed to other **separate** arguments
3. NOT assigned to non-**separate** attributes

Invalid:

```
feature
  bad_use (s: separate STRING) do
    local_string: STRING
    local_string := s  -- INVALID: assigning separate to non-separate
  end
end
```

Valid:

```
feature
  good_use (s: separate STRING) do
    s.to_upper  -- Valid: feature call
    forward (s) -- Valid: pass to separate argument
  end

  forward (s: separate STRING) do ... end
end
```

Constraint VC-2: Separate Attribute Access

An attribute declared as **separate**:

```
class AGENT_POOL
  separate_workers: ARRAY [separate WORKER]

  feature
    process (task: TASK) do
      -- Can access separate_workers [i] through calls
      across separate_workers as cursor loop
        cursor.item.execute (task)  -- Separate call
      end
    end
end
```

Constraint VC-3: Creation in Separate Processor

Creating a **separate** object:

```
local
  separate_resource: separate RESOURCE
do
  -- Creates in separate processor, gets new execution context
  create separate_resource.initialize

  -- Now can use separate_resource
  separate_resource.setup
end
```

8.34.7 Semantic Rules

Rule S-1: Processor Boundary Enforcement

Each **separate** type declaration creates a potential processor boundary:

```
class MULTI_WORKER_SYSTEM
  worker1: separate WORKER
  worker2: separate WORKER
  shared_resource: RESOURCE

  feature
    coordinate do
      -- shared_resource in this processor
      -- worker1 in separate processor 1
      -- worker2 in separate processor 2
      worker1.process (shared_resource)
      worker2.process (shared_resource)
    end
end
```

Rule S-2: Asynchronous Semantics for Procedures

Procedure calls on **separate** targets are asynchronous:

```
buffer: separate QUEUE [INTEGER]
do
  buffer.enqueue (1)
  buffer.enqueue (2)
  buffer.enqueue (3)
  -- These three calls execute asynchronously in separate processor
  io.put_string ("Enqueued 3 items")  -- Executes immediately
end
```

Rule S-3: Synchronous Semantics for Functions

Function calls block **until** result available:

```
buffer: separate QUEUE [INTEGER]
local
  size: INTEGER
do
  buffer.enqueue (42)  -- Asynchronous
  size := buffer.count -- BLOCKS until count available
  io.put_string (size.out)  -- Guaranteed synchronized access
end
```

8.34.8 Once Keys for Execution Scope

Purpose

Once routines execute their body only on the first call, caching the result for subsequent calls. Once keys control the scope of this "first call" semantics **across** processors and threads.

BNF-E Production Rules

```
Once_Routine_Body ≜ once [ Once_Key ] Instructions

Once_Key ≜ "(" Manifest_String ")"


Manifest_String ≜ "THREAD" | "PROCESS" | "OBJECT"
```

Once Key Semantics

Once Key	Scope	Description
`once`	Per-thread (default)	Executed once per thread; same as `once ("THREAD")`
`once ("THREAD")`	Per-thread	Executed once per thread (explicit default)
`once ("PROCESS")`	Per-process (global)	Executed once for entire process; shared **across** all threads
`once ("OBJECT")`	Per-object instance	Executed once per object instance

Syntax Examples

```
class CONFIGURATION
feature -- Once per thread (default)

    thread_local_counter: INTEGER
            -- Counter local to current thread.
        once
            Result := 0
        end

feature -- Once per process (global singleton)

    shared_configuration: CONFIGURATION_DATA
            -- Global configuration, shared across all threads.
        once ("PROCESS")
            create Result.make_from_file ("config.ini")
        end

feature -- Once per object instance

    instance_cache: HASH_TABLE [STRING, INTEGER]
            -- Cache specific to this object instance.
        once ("OBJECT")
            create Result.make (100)
        end

feature -- Explicit per-thread (equivalent to bare once)

    thread_logger: LOGGER
            -- Logger for current thread.
        once ("THREAD")
            create Result.make ("thread_" + thread_id.out + ".log")
```

```
        end
end
```

Interaction with SCOOP

In a SCOOP environment, each **separate** processor constitutes its own execution context. Once-per-thread semantics apply per SCOOP processor:

```
class WORKER
feature
    processor_id: INTEGER
            -- Unique identifier for this processor's context.
        once
            -- Evaluated once per SCOOP processor (thread-local default)
            Result := generate_unique_id
        end
end
```

Note: `once` and `once ("THREAD")` are equivalent. Once-per-thread is the default because it is the safest option in concurrent environments.

8.35 ACROSS iteration

8.35.1 ACROSS Loop Syntax

Purpose

The ACROSS construct provides expressive iteration over any iterable data structure, eliminating off-by-one errors and abstracting iteration details.

BNF-E Production Rules

```
Across_Instruction ≜ across Expression as Identifier
                     loop
                       Instructions
                     [ until Condition ]
                     [ variant Expression ]
                     end

Cursor_Access ≜ Identifier . Feature_Name

| @ Identifier . Feature_Name
```

Syntax Examples

```
-- Form 1: Simple iteration
across my_list as cursor loop
  process (cursor.item)
end
```

```
-- Form 2: With termination condition
across my_list as cursor
until cursor.item > threshold
loop
  print (cursor.item)
end

-- Form 3: With loop variant (proves termination)
across my_array as i
variant
  my_array.count - i.index
loop
  process (my_array [i.index])
end

-- Form 4: Nested iteration
across outer_list as x loop
  across inner_list as y loop
    combine (x.item, y.item)
  end
end

-- Form 5: Iteration with filtering
across items as item
until item.index > 10
loop
  if item.item.is_valid then
    process (item.item)
  end
end
```

Iterating Over Different Collection Types

```
-- Over lists
across my_list as cursor loop
  process (cursor.item)
end

-- Over arrays
across my_array as cursor loop
  process (cursor.item)
end

-- Over hash tables
across my_hash as cursor loop
  key := @ cursor.key
  value := cursor.item
end

-- Over sets
across my_set as cursor loop
  process (cursor.item)
end

-- Over tuples (13.11+)
```

```
across my_tuple as cursor loop
  process (cursor.item)
end

-- Over custom iterables
across my_custom_collection as cursor loop
  process (cursor.item)
end
```

8.35.2 ITERABLE Protocol and Interface

ITERABLE Interface Definition

For a type to be iterable with ACROSS, it must conform to the ITERABLE interface:

```
class ITERABLE [G]
  -- Minimal interface for across loops

  feature
    new_cursor: ITERATION_CURSOR [G] deferred end
      -- Create new cursor for iteration

    has (item: G): BOOLEAN deferred end
      -- True if item is in structure

    is_empty: BOOLEAN deferred end
      -- True if no items

    count: INTEGER deferred end
      -- Number of items
end
```

ITERATION_CURSOR Interface

The cursor protocol used during iteration:

```
class ITERATION_CURSOR [G]
  feature
    item: G deferred end
      -- Current item being iterated

    index: INTEGER deferred end
      -- Position in iteration (1-based typically)

    target: ITERABLE [G] deferred end
      -- The collection being iterated

    key: G deferred end
      -- For hash tables, the current key

    forth do deferred end
      -- Advance to next item

    after: BOOLEAN deferred end
      -- True when iteration complete
```

```
    is_valid: BOOLEAN deferred end
      -- True if cursor in valid state
  end
end
```

Built-in Support for Standard Collections

Standard Eiffel collections automatically support ACROSS:

```
my_list: LIST [STRING]
my_array: ARRAY [STRING]
my_hash: HASH_TABLE [INTEGER, STRING]
my_set: SET [STRING]
my_tuple: TUPLE [STRING, INTEGER, BOOLEAN]

-- All of these work:
across my_list as c loop ... end
across my_array as c loop ... end
across my_hash as c loop ... end
across my_set as c loop ... end
across my_tuple as c loop ... end
```

8.35.3 Cursor Features and Advancement

Cursor Properties

The cursor variable in an ACROSS **loop** has special properties:

```
across my_list as cursor loop
  cursor.item       -- The current element
  cursor.index      -- Position (for indexed collections)
  cursor.target     -- The collection being iterated
  cursor.is_valid   -- True if cursor in valid position
end
```

Loop Advancement

The **loop** automatically advances the cursor:

```
across my_list as cursor loop
  -- At start of each iteration, cursor points to next item
  io.put_string (cursor.item)
  -- Cursor automatically advanced at end of iteration
  -- (no explicit forth needed)
end
```

Specialized Cursors for Hash Tables

For hash tables, additional features available:

```
my_table: HASH_TABLE [VALUE_TYPE, KEY_TYPE]

across my_table as entry loop
  key := entry.key          -- The hash key
  value := entry.item       -- The associated value
end
```

8.35.4 Loop Variables and Item Access

Basic Item Access

Direct access to current item:

```
across items as cursor loop
  x := cursor.item  -- Access current item
end
```

Using Cursor Identifier Directly

As shorthand, the cursor identifier can represent the item:

```
across items as item loop
  process (item)  -- Direct access to item
end
```

Extracted Variables

Extract and **rename** the item:

```
across items as cursor loop
  my_item := cursor.item
  process (my_item)
end
```

8.35.5 Cursor Feature Access with @ Operator

Purpose and Syntax (EiffelStudio 21.11+)

The @ operator (added in 21.11) accesses cursor features directly:

```
Cursor_Feature_Access ≜ @ Cursor_Identifier . Feature_Name
```

Examples

```
across my_table as entry loop
  -- Old form (still valid):
  table_key := entry.key

  -- New form with @ operator (21.11+):
  table_key := @ entry.key

  -- Item access (same with or without @):
  value := entry.item
  value := @ entry.item
end

-- Cursor index access
across my_list as cursor loop
  position := @ cursor.index
```

```
  current := cursor.item      -- or @ cursor.item
end
```

Semantics

The @ operator clarifies that a **feature** belongs to the cursor, not the item:

```
-- Key belongs to cursor (hash table)
key := @ cursor.key

-- Item is the actual data
data := cursor.item
```

8.35.6 Validity Constraints

Constraint VC-1: Expression Must Be ITERABLE

The expression in an ACROSS **loop** must conform to ITERABLE:

```
across non_iterable_entity as cursor loop  -- INVALID if not ITERABLE
  ...
end
```

Constraint VC-2: Cursor Variable Scope

The cursor variable is local to the **loop**:

```
across my_list as cursor loop
  process (cursor.item)
end
-- cursor is not accessible here (out of scope)
```

Constraint VC-3: No Modification During Iteration

Modifying the collection during iteration is restricted:

```
across my_list as cursor loop
  my_list.extend (new_item)  -- May be invalid depending on type
end
```

Constraint VC-4: Variant and Until

If both `<span style="color:#0066cc;font-weight:bold">variant</span>` and `<span style="color:#0066cc;font-weight:bold">until</span>` are specified:

```
across my_list as cursor
until cursor.after          -- Termination condition
variant
  my_list.count - cursor.index  -- Must decrease
loop
  process (cursor.item)
end
```

8.35.7 Comparison to Traditional Loops

Traditional Form

```
from
  i := 1
until
  i > my_list.count
loop
  process (my_list [i])

  variant
    my_list.count - i
  invariant
    i >= 1
    i <= my_list.count + 1

  i := i + 1
end
```

ACROSS Form (Modern)

```
across my_list as cursor loop
  process (cursor.item)

  variant
    my_list.count - cursor.index
  invariant
    cursor.is_valid or cursor.after
end
```

Advantages of ACROSS

1. **Cleaner**: No manual index management
2. **Safer**: Cursor handles advancement automatically
3. **Generic**: Works with any ITERABLE type
4. **Expressive**: Intent is clearer (iteration, not index manipulation)
5. **Reusable**: Works with custom collections implementing ITERABLE

8.36 Modern type system

8.36.1 Sized Integer Types

Overview

Eiffel provides fixed-size integer types for precise control over memory and range:

```
Type_Integer ≜ INTEGER
               | INTEGER_8  | INTEGER_16  | INTEGER_32  | INTEGER_64
               | NATURAL_8  | NATURAL_16  | NATURAL_32  | NATURAL_64
```

Sized Integer Specification

Type	Bits	Signed	Range	Use Case
INTEGER_8	8	Yes	-128 to 127	Small byte values
INTEGER_16	16	Yes	-32,768 to 32,767	Small integers
INTEGER_32	32	Yes	-2,147,483,648 to 2,147,483,647	Standard integers
INTEGER_64	64	Yes	±9.2×10^18	Large integers, timestamps
NATURAL_8	8	No	0 to 255	Unsigned bytes
NATURAL_16	16	No	0 to 65,535	Unsigned shorts
NATURAL_32	32	No	0 to 4,294,967,295	Unsigned integers
NATURAL_64	64	No	0 to 1.84×10^19	Unsigned longs

Syntax Examples

```
-- Sized integers
small_count: INTEGER_8           -- Values -128 to 127
medium_value: INTEGER_32         -- Standard 32-bit
large_timestamp: INTEGER_64      -- For time values (milliseconds)

-- Natural (unsigned)
byte_value: NATURAL_8            -- 0 to 255
color_component: NATURAL_8       -- RGB component
file_size: NATURAL_64            -- For large file sizes

-- Mixed arithmetic
local
  a: INTEGER_8 := 100
  b: INTEGER_32 := 50000
  c: INTEGER_64
do
  c := a.as_integer_64 + b.as_integer_64
end
```

Type Conversion Features

Built-in conversion between sizes:

```
small: INTEGER_8 := 10
medium: INTEGER_32 := small.as_integer_32
large: INTEGER_64 := small.as_integer_64

-- Reverse conversions (with overflow checking)
x: INTEGER_32 := 50000
y: INTEGER_8 := x.as_integer_8  -- Possible overflow
```

Conformance Rules

```
INTEGER_8 ⊆ INTEGER_16 ⊆ INTEGER_32 ⊆ INTEGER_64
NATURAL_8 ⊆ NATURAL_16 ⊆ NATURAL_32 ⊆ NATURAL_64
```

Sized integers do NOT conform to plain INTEGER (explicit conversion required).

8.36.2 Unsigned Integers (NATURAL Types)

Overview

NATURAL types provide unsigned (non-negative) integers when negative values are not needed:

```
class PIXEL_BUFFER
  feature
    pixel_count: NATURAL_32     -- Always non-negative
    red: NATURAL_8              -- 0-255
    green: NATURAL_8             -- 0-255
    blue: NATURAL_8              -- 0-255

    set_color (r, g, b: NATURAL_8) do
      red := r
      green := g
      blue := b
    end
end
```

Bitwise Operations

NATURAL types support bitwise operations:

```
mask: NATURAL_32 := 0xFF
flags: NATURAL_32 := 0x0F
combined := flags.bit_or (mask)     -- Bitwise OR
intersection := flags.bit_and (mask) -- Bitwise AND
shifted := flags.bit_shift_left (2)  -- Bit shift
```

Usage Patterns

```
-- Sizes and counts (always non-negative)
capacity: NATURAL_32
length: NATURAL_32
count: NATURAL_32

-- Bit flags
permissions: NATURAL_32
file_attributes: NATURAL_32

-- Memory addresses and pointers
address: NATURAL_64
pointer_value: NATURAL_64
```

8.36.3 Unicode Characters and Strings

Character Types

```
-- 8-bit characters (ASCII compatible)
c1: CHARACTER       -- or CHARACTER_8 (traditional)
c1 := 'A'

-- 32-bit characters (Unicode code points)
c2: CHARACTER_32
c2 := 'é'           -- Unicode character
c2 := {CHARACTER_32} 0x00E9  -- Code point assignment
```

String Types

```
-- 8-bit string (ASCII/Latin-1)
s1: STRING          -- or STRING_8
s1 := "Hello"

-- 32-bit string (Unicode)
s2: STRING_32
s2 := "Hello, 世界"  -- Unicode support
s2 := "Café"        -- Accented characters
```

String Conversion and Operations

```
local
  s8: STRING
  s32: STRING_32
do
  s8 := "test"
  s32 := s8.as_string_32    -- Widen 8-bit to 32-bit
  s8 := s32.as_string_8     -- Narrow (potential loss)

  -- Operations on Unicode strings
  s32.to_upper               -- Unicode-aware case conversion
  s32.starts_with ("Hello")
  s32.substring (1, 5)
end
```

Manifest String Typing

String literals automatically type based on content:

```
-- Manifest strings auto-typing (20.05+)
x := "ASCII only"           -- Infers STRING (8-bit)
y := "With é accent"        -- Infers STRING_32 (Unicode)
z: STRING_32 := "Unicode"   -- Explicit STRING_32
```

8.36.4 Type Conversions (CONVERT Clause)

Purpose

The CONVERT clause enables implicit type conversions, making code more natural:

Syntax and BNF-E

```
Convert_Clause ≜ convert Conversion_List

Conversion_List ≜ Conversion { , Conversion }


Conversion ≜ Feature_Name [ ( { Type_List } ) ]
```

Examples

```
class DURATION
  feature
    -- Conversion from STRING
    make_from_string (s: STRING) do
      -- Parse duration string like "2h30m"
    end

  feature
    convert
      make_from_string ({STRING})
end

-- Usage: automatic conversion
duration: DURATION
time_spec: STRING := "1h30m"
duration := time_spec     -- Calls make_from_string automatically

-- Or explicit conversion
duration := "2h" as DURATION  -- Explicit form (Eiffel 22+)
```

Multi-Conversion Example

```
class DATE
  feature
    make_from_string (s: STRING) do ... end
    make_from_integer (days: INTEGER) do ... end
    make_from_timestamp (ts: INTEGER_64) do ... end

  feature
    convert
      make_from_string ({STRING});
      make_from_integer ({INTEGER});
      make_from_timestamp ({INTEGER_64})
end

-- Usage
d1: DATE
d1 := "2026-01-30"        -- Calls make_from_string
d1 := 1000                -- Calls make_from_integer
d1 := 1704067200000_i64   -- Calls make_from_timestamp

<div style="font-family:Consolas,Courier New,monospace;background:#f5f5f5;padding:1rem 1.5rem;border-
left:3px solid #6a0dad;border-radius:2px;line-height:1.8;white-space:pre;overflow-x:auto;margin:1em
0;font-size:0.9em">
#### Validity Constraints

1. Conversion target must be unique (no ambiguity)
2. Cannot create cycles (A converts from B, B converts from A)
3. Must be publicly visible

### 8.36.5 Manifest Types
```

```
#### Purpose

Manifest types allow specifying types explicitly in expressions:

#### Syntax

```eiffel
<span style="color:#006600;font-weight:bold">Type_Specification</span> <span
style="color:#6a0dad;font-weight:bold;font-size:1.6em;vertical-align:middle">≜</span> { Class_Name }
</div>


#### Examples

```eiffel
-- Force specific type in expression
x := {STRING_32} "Hello"         -- Manifest type for string literal
y := {INTEGER_64} 1000           -- Manifest type for integer
z := {ARRAY [INTEGER]} << 1, 2, 3 >>

-- Used with agents
f: FUNCTION [{STRING}, STRING]
f := agent {STRING}.to_upper

-- Type-specific method calls
s := {MY_CLASS}.static_feature
```

Semantic Rules

1. The expression must conform to the specified type
2. Used primarily in context where type inference is ambiguous
3. Makes implicit conversions explicit

8.36.6 Relaxed Conditional Typing

Purpose

Modern Eiffel allows conditional expressions to return different but conforming types:

Syntax and Examples

```
-- Result can be STRING or Void (detachable STRING)
result: ? STRING
result := if condition then "text" else Void end

-- Branch types must conform pairwise
flag: BOOLEAN
value: ? STRING
value := if flag then "attached" else Void end
-- Types: STRING conforms to ? STRING, Void conforms to ? STRING

-- Multiple branches with common ancestor
status: ? COMPARABLE
status := if case1 then
            "status"          -- STRING
          elseif case2 then
```

```
            42                -- INTEGER
          else
            Void              -- Void
          end
-- All types conform to ? COMPARABLE
```

Conformance Rules

For conditional expression:

```
if Condition then Type1 else Type2 end
```

Result type is:

- `Type1` if `Type2 ⊆ Type1`
- `Type2` if `Type1 ⊆ Type2`
- Common ancestor if both conform to ancestor
- `? T` if branches are `T` and `<span style="color:#0066cc;font-weight:bold">Void</span>`

8.37 Enhanced agents

8.37.1 Simplified Agent Type Notation

Overview

EiffelStudio 15.12 introduced simplified notation for agent types, making functional programming more concise:

Old vs. New Notation

```
-- Old notation (still supported for backward compatibility)
my_function: FUNCTION [MY_CLASS, TUPLE [INTEGER], STRING]
  -- First argument implicit (object holding feature)

my_procedure: PROCEDURE [MY_CLASS, TUPLE [INTEGER, STRING]]
  -- First argument implicit

-- New notation (15.12+, recommended)
my_function: FUNCTION [INTEGER, STRING]
  -- Simpler: just argument and result types

my_procedure: PROCEDURE [INTEGER, STRING]
  -- Just the argument types
```

Type Specification

```
Function_Type ≜ FUNCTION [ [ Argument_Types ] , Result_Type ]

Procedure_Type ≜ PROCEDURE [ Argument_Types ]
```

```
Predicate_Type ≜ PREDICATE [ Argument_Types ]

-- Always returns BOOLEAN

Argument_Types ≜ Type { , Type }

Result_Type ≜ Type
```

Examples

```
-- Simple function: takes INTEGER, returns STRING
increment_function: FUNCTION [INTEGER, STRING]

-- Procedure: takes STRING and BOOLEAN
log_procedure: PROCEDURE [STRING, BOOLEAN]

-- Predicate: takes INTEGER, returns BOOLEAN
is_positive: PREDICATE [INTEGER]

-- Multi-argument function
calculate: FUNCTION [INTEGER, INTEGER, INTEGER]
  -- Takes two INTEGERs, returns INTEGER

-- No arguments
greeting: PROCEDURE
  -- Takes nothing, does something

-- Complex types
process_data: FUNCTION [ARRAY [STRING], HASH_TABLE [INTEGER, STRING], LIST [BOOLEAN]]
  -- Takes array and hash table, returns list
```

8.37.2 Inline Agent Syntax

Purpose

Inline agents allow defining routine references with implementations inline (added 5.7):

BNF-E Production Rules

```
Inline_Agent ≜ agent [ Formal_arguments : Result_Type ]
                    do Instructions end
                | agent [ Formal_arguments ] do Instructions end

Formal_arguments ≜ "(" Entity_declaration_list ")"
```

Basic Inline Agent Examples

```
-- Inline function with explicit types
square: FUNCTION [INTEGER, INTEGER]
square := agent (x: INTEGER): INTEGER do Result := x * x end

-- Inline procedure
print_item: PROCEDURE [STRING]
print_item := agent (s: STRING) do
  io.put_string (s)
  io.new_line
end

-- Using inline agent directly in code
button.select_actions.extend (
  agent (event: EVENT_DATA) do
    on_button_clicked (event)
  end
)
```

Type Inference

The compiler infers argument and return types when possible:

```
-- Type inference from context
process: PROCEDURE [STRING, INTEGER]
process := agent (s, n) do
  -- Compiler infers s: STRING, n: INTEGER from target type
  io.put_string (s)
  io.put_integer (n)
end

-- Partial specification
calculate: FUNCTION [?, ?, INTEGER]
calculate := agent (a, b): INTEGER do
  -- Compiler infers a and b types from usage
  Result := a + b
end
```

Open Arguments

Arguments can be left "open" (to be provided at call time):

```
-- One argument open
add_five: FUNCTION [INTEGER, INTEGER]
add_five := agent add (?, 5)

-- Both arguments open
add_func: FUNCTION [INTEGER, INTEGER, INTEGER]
add_func := agent add (?, ?)

-- Helper feature
add (a: INTEGER; b: INTEGER): INTEGER do Result := a + b end
```

```
-- Usage
x := add_five (10)      -- Equivalent to add (10, 5)
y := add_func (3, 7)    -- Equivalent to add (3, 7)
```

8.37.3 Open Arguments

Purpose

Open arguments allow creating partial functions by specifying some arguments while leaving others for later invocation:

Syntax

```
Open_Argument ≜ ?
```

Examples

```
class MATH_OPERATIONS
  feature
    add (a: INTEGER; b: INTEGER): INTEGER
      do Result := a + b end

    multiply (a: INTEGER; b: INTEGER): INTEGER
      do Result := a * b end

    power (base: INTEGER; exponent: INTEGER): INTEGER
      do Result := base ^ exponent end
end

-- Creating partial functions with open arguments
local
  add_five: FUNCTION [INTEGER, INTEGER]
  multiply_by_ten: FUNCTION [INTEGER, INTEGER]
  square: FUNCTION [INTEGER, INTEGER]
do
  add_five := agent add (?, 5)
  multiply_by_ten := agent multiply (?, 10)
  square := agent power (?, 2)

  io.put_integer (add_five (3))           -- 3 + 5 = 8
  io.put_integer (multiply_by_ten (7))    -- 7 * 10 = 70
  io.put_integer (square (4))             -- 4 ^ 2 = 16
end
```

Use Cases

1. **Functional composition**:
   ```
   apply_operations: FUNCTION [LIST [FUNCTION [INTEGER, INTEGER]], INTEGER]
   ```
2. **Higher-order functions**:
   ```
   map (list: LIST [INTEGER]; transform: FUNCTION [INTEGER, INTEGER]): LIST [INTEGER]
   ```

```
3. **Event handlers**:
   ```eiffel
   button.select_actions.extend (agent on_click (?, create_window))
```

8.37.4 Agent Validity Constraints

Constraint VC-1: Type Compatibility

Agent actual must conform to agent formal:

```
formal: FUNCTION [INTEGER, STRING]
actual: FUNCTION [INTEGER, STRING]
formal := actual  -- Valid if signatures match

actual2: FUNCTION [REAL, STRING]
formal := actual2  -- INVALID: REAL doesn't match INTEGER
```

Constraint VC-2: Open Argument Matching

Open arguments must have matching types:

```
add_integers (a: INTEGER; b: INTEGER): INTEGER do ... end
add_reals (a: REAL; b: REAL): REAL do ... end

-- Valid
f1: FUNCTION [INTEGER, INTEGER]
f1 := agent add_integers (?, 5)  -- 5 matches second argument

-- Invalid
f2: FUNCTION [STRING, INTEGER]
f2 := agent add_integers (?, 5)  -- STRING doesn't match first argument
```

Constraint VC-3: Target Consistency

An agent created from a target must maintain target consistency:

```
class WORKER
  feature
    process (item: STRING) do ... end
end

local
  worker: WORKER
  callback: PROCEDURE [STRING]
do
  create worker
  callback := agent worker.process  -- Valid
  -- Target (worker) remains consistent
end
```

8.37.5 FUNCTION and PROCEDURE Types

FUNCTION Type

Represents routines that return a value:
```

```
FUNCTION [Argument_Types, Result_Type]

-- Examples
add: FUNCTION [INTEGER, INTEGER, INTEGER]        -- (a, b) -> INTEGER
to_string: FUNCTION [ANY, STRING]                -- (obj) -> STRING
create_window: FUNCTION [STRING, WINDOW]         -- (title) -> WINDOW
evaluate: FUNCTION [EXPRESSION, ARRAY [ANY], ANY] -- (expr, vars) -> ANY
```

PROCEDURE Type

Represents routines that perform actions without returning:

```
PROCEDURE [Argument_Types]

-- Examples
log: PROCEDURE [STRING]                    -- (msg) -> void
set_value: PROCEDURE [STRING, ANY]         -- (name, value) -> void
initialize: PROCEDURE                      -- () -> void
process_all: PROCEDURE [LIST [ITEM]]       -- (items) -> void
```

PREDICATE Type

Special case: returns BOOLEAN

```
PREDICATE [Argument_Types]

-- Examples
is_positive: PREDICATE [INTEGER]
is_valid_email: PREDICATE [STRING]
contains_item: PREDICATE [LIST [ITEM], ITEM]
matches_pattern: PREDICATE [STRING, STRING]

-- Usage
local
  numbers: LIST [INTEGER]
  even_numbers: LIST [INTEGER]
  is_even: PREDICATE [INTEGER]
do
  is_even := agent (n: INTEGER): BOOLEAN do Result := n \\ 2 = 0 end

  across numbers as cursor loop
    if is_even.item (cursor.item) then
      even_numbers.extend (cursor.item)
    end
  end
end
```

Operational Features on Agents

Common operations on agent instances:

```
local
  f: FUNCTION [INTEGER, STRING]
  p: PROCEDURE [STRING]
  count: INTEGER
```

```
do
  f := agent x.compute

  -- Call the agent
  result := f.item (42)

  -- Get the associated target
  target := f.target

  -- Check if agent references a specific target
  if f.target = my_object then
    io.put_string ("Agent references my_object")
  end
end
```

8.38 Feature declaration enhancements

8.38.1 Alias Clause

Purpose

The ALIAS clause allows features to have alternative names or symbols, enabling operator notation and more natural syntax:

BNF-E Production Rules

```
Feature_Declaration ≜ Feature_Name [ Formal_arguments ]
                      [ : Return_Type ]
                      [ alias Alias_Identifier ]
                      [ do Instructions end ]

Alias_Identifier ≜ Operator_Symbol | Feature_Name | Manifest_String


Operator_Symbol ≜ "+" | "-" | "*" | "/" | "//" | "mod"

| "^" | "="  | "/=" | "<" | ">" | "<=" | ">="

| "~" | ".." | "@" | and | or | xor | implies

| "<<" | ">>"
```

Operator Examples

```
class VECTOR [G -> NUMERIC]
  feature
    plus alias "+" (other: VECTOR [G]): VECTOR [G] do
```

```
      -- Vector addition
      Result := deep_copy
      Result.add_component (other)
    ensure
      magnitude_preserved: Result.magnitude > 0
    end

    negate alias "prefix -": VECTOR [G] do
      -- Unary negation
      Result := deep_copy
      Result.multiply_by (-1.0)
    end

    scale alias "*" (factor: REAL): VECTOR [G] do
      Result := deep_copy
      Result.multiply_by (factor)
    end

    dot_product alias "·" (other: VECTOR [G]): REAL do
      -- Inner product
      Result := 0.0
      across items as cursor loop
        Result := Result + cursor.item * other.items [cursor.index]
      end
    end
end

-- Usage
local
  v1, v2: VECTOR [REAL]
  v3: VECTOR [REAL]
  scalar: REAL
do
  create v1.make_3d (1.0, 2.0, 3.0)
  create v2.make_3d (4.0, 5.0, 6.0)

  v3 := v1 + v2                    -- Uses plus alias
  v3 := -v1                        -- Uses negate alias
  v3 := v1 * 2.0                   -- Uses scale alias
  scalar := v1 · v2                -- Uses dot_product alias (if Unicode)
end
```

Feature Name Aliases

```
class COMPARABLE_WRAPPER [G -> COMPARABLE]
  feature
    compare (other: COMPARABLE_WRAPPER [G]): INTEGER
      do
        if Current.value < other.value then
          Result := -1
        elseif Current.value > other.value then
          Result := 1
        else
          Result := 0
        end
      end
```

```
    -- Alias: explicit name
    is_less alias "less_than" (other: COMPARABLE_WRAPPER [G]): BOOLEAN
      do Result := compare (other) < 0 end

    is_greater alias "greater_than" (other: COMPARABLE_WRAPPER [G]): BOOLEAN
      do Result := compare (other) > 0 end
end

-- Usage
x: COMPARABLE_WRAPPER [STRING]
y: COMPARABLE_WRAPPER [STRING]
if x.less_than (y) or x.is_less (y) then  -- Both work
  ...
end
```

8.38.2 Bracket Notation Features

Purpose

Bracket notation allows creating array-like or map-like access patterns on arbitrary objects:

BNF-E Production Rules

```
Bracket_Feature ≜ Feature_Name alias "[]" Formal_arguments [ : Return_Type ]

Access_Expression ≜ Object [ Index_Expression ]

| Object [ Index_Expression ] := Value
```

Getter Example

```
class CUSTOM_ARRAY [G]
  feature
    item alias "[]" (index: INTEGER): G
      require
        index >= 1
        index <= count
      do
        Result := internal_array [index - 1]
      ensure
        Result = internal_array [index - 1]
      end
end

-- Usage
my_array: CUSTOM_ARRAY [STRING]
s: STRING
s := my_array [5]      -- Calls item (5)
```

Setter Example

```
class MUTABLE_DICTIONARY [K, V]
  feature
    item alias "[]" (a_key: K): V assign put
            -- Value associated with `a_key`.
      require
        is_defined (a_key)
      do
        Result := internal_table [a_key]
      end

    put (a_value: V; a_key: K)
            -- Associate `a_value` with `a_key`.
      do
        internal_table [a_key] := a_value
      ensure
        is_defined (a_key)
        item (a_key) = a_value
      end

    is_defined (a_key: K): BOOLEAN
            -- Is there a value associated with `a_key`?
      do
        Result := internal_table.has (a_key)
      end
end

-- Usage:
dict: MUTABLE_DICTIONARY [STRING, INTEGER]
dict ["age"] := 30              -- Calls put (30, "age") via assigner
age: INTEGER := dict ["age"]    -- Calls item ("age")
```

Note: The `alias "[]"` goes on the query (`item`), not the command (`put`). The `<span style="color:#0066cc;font-weight:bold">assign</span> put` clause on `item` enables the `dict ["age"] := 30` assignment syntax.

Validity Constraints

1. Only one **feature** per **class** may have `alias "[]"`
2. The bracket **feature** must be a query (must have a return type)
3. To enable assignment syntax (`obj [key] := value`), use `<span style="color:#0066cc;font-weight:bold">assign</span>` clause on the bracket query
4. The assigner command's first argument must accept the value type

8.38.3 Assigner Commands

Purpose

Assigner commands make properties appear assignable while calling a setter routine:

BNF-E Production Rules

```
Assigner_Clause ≜ assign Assigner_Name

Assigner_Name ≜ Feature_Name
```

Examples

```
class PERSON
  feature
    age: INTEGER assign set_age
            -- Current age; assignment calls `set_age`.

    -- Setter: performs validation
    set_age (value: INTEGER)
      require
        value >= 0
        value <= 150
      do
        internal_age := value
      ensure
        age = value
      end
end

-- Usage
person: PERSON
create person.make
person.age := 25              -- Calls set_age (25)
person.age := -5              -- Precondition fails
```

Advanced Assigner Example

```
class ACCOUNT
  feature
    balance: REAL assign set_balance
            -- Current balance; assignment calls `set_balance`.

    set_balance (a_amount: REAL)
            -- Set balance to `a_amount`.
      require
        non_negative: a_amount >= 0.0
      do
        internal_balance := a_amount
      ensure
        balance_set: balance = a_amount
      end

    deposit (a_amount: REAL)
            -- Add `a_amount` to balance.
      require
        positive: a_amount > 0.0
      do
        internal_balance := internal_balance + a_amount
      ensure
        increased: balance = old balance + a_amount
      end

  feature {NONE} -- Implementation
```

```
    internal_balance: REAL
end

-- Usage:
account: ACCOUNT
account.balance := 100.0       -- Calls set_balance (100.0) via assigner
account.deposit (50.0)         -- Direct call, not via assigner
```

Note: Each query may have at most one assigner command. To perform different kinds of updates, define **separate** commands (like `deposit`) that are called directly.

8.38.4 Parenthesis Aliases

Purpose

Parenthesis aliases allow calling agents or callable objects with function-call syntax:

BNF-E Production Rules

```
Parenthesis_Alias ≜ Feature_Name alias "()"
```

Examples

```
class CALLABLE_OBJECT
  feature
    -- Define what calling the object does
    apply alias "()"
      require
        is_ready
      do
        execute_computation
      end

    apply alias "()" (x: INTEGER): INTEGER
      require
        x > 0
      do
        Result := x * x
      end
end

-- Usage
callable: CALLABLE_OBJECT
callable ()                -- Calls apply (no arguments)
value := callable (5)      -- Calls apply (5)
```

Agent Invocation (Built-in)

Standard agents support parenthesis call:

```
f: FUNCTION [INTEGER, INTEGER]
f := agent (x: INTEGER): INTEGER do Result := x * 2 end
```

```
-- Both equivalent:
result1 := f.item (10)      -- Traditional
result2 := f (10)           -- Parenthesis alias (built-in)

-- Similarly for procedures
p: PROCEDURE [STRING]
p := agent (s: STRING) do io.put_string (s) end

p ("Hello")                 -- Direct call with parenthesis
p.call ("Hello")            -- Traditional explicit call
```

8.38.5 Validity and Semantic Rules

Alias Uniqueness Constraint

No two features can have the same alias:

```
class MATRIX
  feature
    add alias "+" (other: MATRIX): MATRIX do ... end

    -- INVALID: alias "+" already used
    -- combine alias "+" (other: MATRIX): MATRIX do ... end
end
```

Alias Signature Constraint

For operator aliases, signatures must be standard:

```
-- Valid binary operator (2 operands)
plus alias "+" (other: MY_TYPE): MY_TYPE

-- Valid unary operator (1 operand)
negate alias "prefix -": MY_TYPE

-- Invalid: too many arguments
sum alias "+" (a: MY_TYPE; b: MY_TYPE; c: MY_TYPE): MY_TYPE  -- ERROR
```

Bracket Notation Constraints

```
-- Valid: getter
item alias "[]" (index: INTEGER): STRING

-- Valid: setter
put alias "[]" (value: STRING; index: INTEGER)

-- Invalid: no return type for getter
get alias "[]" (index: INTEGER)          -- Must have return type

-- Invalid: no value argument for setter
set alias "[]" (index: INTEGER)          -- Must have value as first param
```

8.39 Modern expression forms

8.39.1 Multi-Branch Conditional Expressions

Purpose

Modern Eiffel allows if/then/else constructs to be used as expressions (not just statements), enabling functional programming style (added 20.05):

BNF-E Production Rules

```
Conditional_Expression ≜ if Condition then Expression
                           { elseif Condition then Expression }
                           [ else Expression ]
                           end

Condition ≜ Expression
```

Basic Examples

```
-- Simple binary choice
status: STRING
status := if age >= 18 then "adult" else "minor" end

-- Three-way choice
grade: STRING
grade := if score >= 90 then "A"
         elseif score >= 80 then "B"
         elseif score >= 70 then "C"
         else "F"
         end

-- Boolean result
is_valid: BOOLEAN
is_valid := if count > 0 and count < 100 then True else False end
```

Type Inference in Conditional Expressions

The result type is inferred from branches:

```
-- All INTEGER - result type: INTEGER
result1: INTEGER
result1 := if x > 0 then 1 else 2 end

-- STRING or Void - result type: ? STRING
result2: ? STRING
result2 := if name /= Void then "Named" else Void end

-- Different but conforming - result type: common ancestor
result3: COMPARABLE
result3 := if flag then "text" else 123 end  -- Both conform to COMPARABLE

-- With attachment
```

```
result4: ? INTEGER
result4 := if initialized then age else Void end
```

Complex Nested Expressions

```
-- Nested conditionals in expression
category: STRING
category := if age < 13 then
              "child"
            elseif age < 18 then
              "teenager"
            elseif age < 65 then
              if is_employed then "working_adult" else "adult" end
            else
              "senior"
            end

-- Conditional in assignment
value := if activated then compute_intensive else get_cached end

-- Conditional as argument
process (
  if is_debug_mode then create_verbose_logger else create_silent_logger end
)
```

Comparison to Statement Form

```
-- Statement form (imperative)
if condition then
  action_a
else
  action_b
end

-- Expression form (functional)
result := if condition then compute_a else compute_b end
```

8.39.2 Unified Object Equality Operator (~)

Purpose

The ~ operator distinguishes object equality (value comparison) from identity (reference comparison):

Definition

- `=` : Identity (same object in memory)
- `~` : Equality (equivalent values)
- `/=` : Identity negation
- `/~` : Equality negation (some implementations)

BNF-E Production Rules

```
Comparison ≜ Expression Comparison_Operator Expression
```

```
Comparison_Operator ≜ "=" | "/=" | "~" | "/~"

| "<" | ">" | "<=" | ">="
```

Examples

```
class ACCOUNT
  feature
    id: INTEGER
    balance: DECIMAL
    owner: STRING

    is_equal (other: ACCOUNT): BOOLEAN
      do
        Result := id = other.id and
                  balance ~ other.balance and
                  owner ~ other.owner
      end
end

local
  acc1, acc2, acc3: ACCOUNT
do
  create acc1.initialize (100, "Alice")
  create acc2.initialize (100, "Alice")
  acc3 := acc1

  -- Identity vs. Equality
  acc1 = acc3              -- True (same object)
  acc1 = acc2              -- False (different objects)
  acc1 ~ acc2              -- True (same values)

  -- With basic types
  s1: STRING := "hello"
  s2: STRING := "hello"
  s1 = s2                  -- May be False (different locations)
  s1 ~ s2                  -- True (same content)
end
```

Custom Equality

Classes customize the behavior of `~` by redefining `is_equal` inherited from `ANY`:

```
class COMPLEX_NUMBER
inherit
    ANY
        redefine
            is_equal
        end

feature -- Access
```

```
    real_part: REAL
    imaginary_part: REAL

feature -- Comparison

    is_equal (a_other: like Current): BOOLEAN
            -- Are `Current` and `a_other` mathematically equal?
        do
            Result := real_part = a_other.real_part and
                      imaginary_part = a_other.imaginary_part
        end
end

-- Usage:
c1 ~ c2     -- Calls is_equal; True if same real and imaginary parts
c1 /~ c2    -- Negation of ~; True if NOT equal
```

Note: Eiffel does not have operator overloading. The `~` operator always calls `is_equal`. To customize equality semantics, **redefine** `is_equal` inherited from `ANY`.

8.39.3 Expression Validity Constraints

Constraint VC-1: Return Type Consistency

In conditional expressions, all branches must type-check:

```
-- Valid: all branches return compatible types
x := if condition then 1 else 2 end

-- Invalid: incompatible branch types
y := if condition then "string" else 42 end
```

Constraint VC-2: Void Propagation

Void values propagate to **detachable** types:

```
-- Valid: branches include Void
x: ? INTEGER
x := if condition then 42 else Void end

-- Invalid: Void cannot be assigned to attached type
y: INTEGER
y := if condition then 42 else Void end  -- Type error
```

Constraint VC-3: Operator Applicability

Comparison operators **require** appropriate operands:

```
-- Valid: both integers
x: INTEGER
y := x ~ 5          -- Valid

-- Valid: both strings
s: STRING
flag := s ~ "test" -- Valid
```

```
-- Needs care: comparing different types requires conformance
```

8.40 Non-conforming inheritance

8.40.1 Overview

Non-conforming inheritance allows a **class** to **inherit** features from a parent without establishing a subtype (conformance) relationship. This enables implementation reuse without the Liskov Substitution implications of conforming inheritance.

Timeline: Introduced experimentally, production-ready in EiffelStudio 21.11 (November 2021).

8.40.2 Syntax

BNF-E Production Rules

```
Inheritance_Clause ≜ inherit [ Non_Conformance_Mark ] Parent_List

Non_Conformance_Mark ≜ "{" NONE "}"


Parent_List ≜ Parent { ";" Parent }


Parent ≜ Class_Type [ Feature_Adaptation ]
```

Syntax Examples

```
class MY_CLASS
inherit
    COMPARABLE          -- Conforming: MY_CLASS IS-A COMPARABLE

inherit {NONE}
    LOGGING_SUPPORT     -- Non-conforming: reuse features but MY_CLASS
                        -- does NOT conform to LOGGING_SUPPORT

feature
    log_message (a_msg: STRING)
        do
            write_log (a_msg)  -- Inherited from LOGGING_SUPPORT
        end
end
```

8.40.3 Semantics

1. Features inherited non-conformingly are available in the **class** as if normally inherited
2. The **class** does NOT conform to the non-conforming parent

3. Polymorphic assignment to the non-conforming parent type is NOT valid
4. Non-conforming inheritance is useful for mixin-style reuse

```
-- Given: class CHILD inherit {NONE} PARENT
child: CHILD
parent: PARENT

parent := child   -- INVALID: CHILD does not conform to PARENT
child.inherited_feature  -- VALID: feature is available
```

8.40.4 Validity Constraints

Constraint VC-1: Feature Availability

Features inherited non-conformingly follow the same adaptation rules (**rename**, **redefine**, **undefine**, **export**) as conforming inheritance.

Constraint VC-2: No Conformance

A type T does NOT conform to a type U if T inherits from U only through non-conforming inheritance paths.

8.41 Multiple generic constraints

8.41.1 Overview

A formal generic parameter may be constrained by multiple types simultaneously. The actual generic parameter must conform to ALL specified constraint types.

Timeline: Introduced in EiffelStudio 6.0 (June 2007).

8.41.2 Syntax

BNF-E Production Rules

```
Formal_Generic ≜ Formal_Generic_Name [ Constraint ]

Constraint ≜ "->" Constraining_Types [ Creation_Constraint ]


Constraining_Types ≜ Single_Type

| "{" Type_List "}"


Type_List ≜ Type { "," Type }


Single_Type ≜ Type
```

Syntax Examples

```
-- Single constraint (original ECMA-367)
class SORTED_LIST [G -> COMPARABLE]

-- Multiple constraints (6.0+)
class INDEXED_CACHE [G -> {COMPARABLE, HASHABLE}]
    -- G must conform to BOTH COMPARABLE and HASHABLE

-- Multiple constraints with creation constraint
class FACTORY [G -> {SERIALIZABLE, COPYABLE} create make end]
    -- G must conform to both types AND have a creation procedure `make`
```

8.41.3 Semantics

When multiple constraints are specified:

1. The actual type parameter must conform to ALL listed constraint types
2. Within the **class**, features from ALL constraint types are available on entities of type G
3. If constraint types have conflicting features, standard name-clash rules apply

```
class MULTI_CONSTRAINED [G -> {COMPARABLE, HASHABLE}]
feature
    process (a_item: G)
        do
            -- Features from COMPARABLE are available:
            if a_item < other_item then ... end

            -- Features from HASHABLE are available:
            hash := a_item.hash_code
        end
end
```

8.41.4 Validity Constraints

Constraint VC-1: Actual Parameter Conformance

For a generic derivation `C [T]` where `G -> {U1, U2, ..., Un}`, type T must conform to every Ui.

Constraint VC-2: Feature Availability

Within the generic **class**, a **feature** call `x.f` on an entity `x` of type `G` constrained by `{U1, U2, ..., Un}` is valid if `f` is a **feature** of at least one Ui.

8.42 Annotation and metadata

8.42.1 The `note` Keyword

The `note` keyword (replacing the older `<span style="color:#0066cc;font-weight:bold">indexing</span>` keyword) provides metadata annotations on classes and features.

BNF-E Production Rules

```
Note_Clause ≜ note Note_List

Note_List ≜ Note_Entry { ";" Note_Entry }


Note_Entry ≜ Note_Name ":" Note_Value_List


Note_Name ≜ Identifier


Note_Value_List ≜ Note_Value { "," Note_Value }


Note_Value ≜ Manifest_String | Identifier | Manifest_Constant
```

Syntax Examples

```
note
    description: "Represents a customer account"
    author: "Jane Smith"
    date: "2026-01-30"
    revision: "1.0"
    license: "MIT"
    ca_ignore: "CA011", "CA033"

class ACCOUNT
    ...
end
```

Special Note Entries

Note Name	Purpose	Example
`description`	Human-readable **class**/**feature** description	`"Account management"`
`author`	Author name	`"Jane Smith"`
`date`	Creation or modification date	`"2026-01-30"`
`revision`	Version string	`"2.1"`
`transient`	Mark attribute as non-persistent (serialization)	`true`
`ca_ignore`	Suppress specific code analysis rules	`"CA011"`
`EIS`	Eiffel Information System link	`"protocol=URI", "src=..."`

Transient Attributes

The `transient` note marks attributes that should not be persisted during serialization:

```
class SESSION
feature
    session_id: STRING
            -- Persisted normally.

    cache: HASH_TABLE [ANY, STRING]
            -- Not persisted.
        note
            transient: true
        attribute
            create Result.make (10)
        end
end
```

8.42.2 Assertion Tags

Assertion tags (enhanced in EiffelStudio 18.11) allow assertions to use the expression itself as an implicit identifier:

Syntax

```
-- Explicit tag (traditional)
require
    positive_value: a_value > 0
    valid_name: not a_name.is_empty

-- Implicit tag (18.11+): expression serves as its own tag
require
    a_value > 0
    not a_name.is_empty
```

When an assertion fails, the tag is used in the exception message. With implicit tags, the expression text itself appears in the error output.

---

9 External interfaces

[This section incorporates updated information about calling C/C++/other languages with modern Eiffel features, including void-safety considerations for external calls.]

10 Standard library

The standard library has been significantly **expanded** to support modern Eiffel features:

Core Collections (ITERABLE Support)

All collections now implement ITERABLE:

- LIST [G]
- ARRAY [G]
- SET [G]
- HASH_TABLE [G, K]
- LINKED_LIST [G]
- ARRAYED_LIST [G]
- TUPLE

Agents and Functional Types

- FUNCTION [ARGS, RESULT]
- PROCEDURE [ARGS]
- PREDICATE [ARGS]
- ROUTINE [ARGS, RESULT]

Void-Safety Support

- OPTIONAL [G] - Wrapper for optional values
- Assertion predicates for validation

Concurrency Support

- THREAD
- PROCESSOR
- BOUNDED_BUFFER [G] with **separate** support
- Thread-safe collections

Unicode Support

- CHARACTER_32
- STRING_32
- UTF-8 / UTF-16 / UTF-32 converters

PART II: EIFFEL STYLE GUIDE

The following sections define the official naming conventions, code organization patterns, and style guidelines for Eiffel programming. These conventions apply to all Eiffel code and are mandatory for professional code quality.

---

11 Style guide overview

This section provides an overview of Eiffel naming conventions and the fundamental principle that drives all naming decisions.

11.1 Quick reference

Category	Pattern	Example	Visibility
Classes	`UPPERCASE_WITH_UNDERSCORES`	`BOUNDED_CACHE`, `CACHE_ENGINE`	Public
Features (access)	`lowercase_with_underscores`	`count`, `item`, `has`, `get`	Varies
Features (pred.)	`is_*`, `has_*`, `are_*`	`is_empty`, `has_error`	Public
Features (cmd.)	`set_*`, `put_*`, `add_*`	`set_level`, `put_with_ttl`	Public
Features (create)	`make`, `make_*`	`make`, `make_with_ttl`	Creation
Arguments	`a_*`	`a_key`, `a_value`, `a_level`	All
Local variables	`l_*` or simple (`i`, `j`, `c`, `n`)	`l_count`, `l_result`, `i`, `c`	Local
Attachment locals	`al_*`	`al_item`, `al_thing`, `al_cache`	Attached
Iteration cursors	`ic`	`ic.item`, `@ic.key`	Loop
Constants	`Proper_case`	`Default_ttl`, `Max_size`, `Version`	Global
Attributes	`lowercase_with_underscores`	`max_size`, `entries`, `context`	{NONE}
Model queries	`*_model`	`items_model`, `keys_model`	Public

11.2 Visibility-driven naming

Fundamental Rule: Use semantic prefixes (like `a_`, `l_`, `al_`) **because you cannot guarantee that the text-editor view will simultaneously show both the identifier usage AND its declaration point**.

Key Insight: You cannot predict or control editor visibility — text changes, files grow, screens vary. Therefore, **use prefixes defensively by default**.

This principle applies to:

- **Arguments (`a_`)**: Feature signature is at the top; code body scrolls below it
- **Locals (`l_`)**: Local declaration is near the top; code body scrolls below it
- **Attachment locals (`al_`)**: Conditional block may scroll past the `if <span style="color:#0066cc;font-weight:bold">attached</span>` line
- **Cursors (`ic`, `rc`)**: Loop body may scroll past the `<span style="color:#0066cc;font-weight:bold">across</span>` line
- **Any identifier** where the possibility exists that declaration and usage won't be simultaneously visible

Why This Matters:

- **You cannot measure visibility**: Text and code change constantly — what's visible now may not be visible in 6 months
- **Default defensively**: Since you can't guarantee simultaneous visibility, use prefixes by default
- **Human readers**: Scrolling separates usage from declaration; prefix is the only scope signal remaining
- **AI readers** (static analysis, IDEs, refactoring tools): Need explicit scope markers for accurate analysis
- **Code reviews**: Different screen sizes, zoom levels, and editor splits affect visibility
- **Refactoring risk**: Growing routines may hide declarations; prefixes protect against misunderstanding

Pragmatic Application:

The prefix exists as a **defensive measure against lost context**:

- **Use by default**: Assume declaration and usage might not be visible together
- **Exception only**: Skip only in trivial one-liners where context is unmistakable
- **Reason**: You cannot predict future visibility, so prefix defensively now

---

12 Class naming

12.1 Pattern

Classes use **UPPERCASE_WITH_UNDERSCORES**.

12.2 Official standards

From Eiffel.org and ISE Eiffel documentation:

- All public classes are UPPERCASE
- Use underscores to **separate** words
- Never use camelCase for classes

12.3 Facade and specialized classes

Facade/Main Classes:

```
HASH_TABLE [G, K]          -- key-value container (ISE Base)
ARRAYED_LIST [G]           -- array-backed list (ISE Base)
LINKED_LIST [G]            -- linked list (ISE Base)
SORTED_TWO_WAY_LIST [G]    -- sorted container (ISE Base)
```

Specialized Classes:

```
{DOMAIN}_ENGINE            -- internal implementation (e.g., CACHE_ENGINE)
{DOMAIN}_RESULT            -- return type (e.g., CACHE_RESULT)
{DOMAIN}_CONFIG            -- configuration object
TEST_{DOMAIN}              -- test class (e.g., TEST_HASH_TABLE)
BENCHMARK_APP              -- benchmark application
```

12.4 Examples

Correct:

```
class BOUNDED_CACHE [G]
class CACHE_ENGINE
class JSON_SCHEMA
class TEST_HTTP_CLIENT
```

Wrong:

```
class LinkedList          -- use UPPERCASE
class linked_list         -- use UPPERCASE
class CacheEngine          -- use UPPERCASE and underscores
class Cache_Engine         -- use all UPPERCASE
```

---

13 Feature naming

13.1 Access features

Pattern: `lowercase_with_underscores` or `is_/has_` for predicates

Simple Accessors:

```
count: INTEGER             -- number of items
item (key: STRING): VALUE  -- direct access
has (key: STRING): BOOLEAN -- check presence
name: STRING               -- simple noun queries
size: INTEGER              -- semantic naming preferred
```

Discouraged for queries:

```
Wrong: get_item (key: STRING): VALUE    -- Don't use get_* prefix
Wrong: get_name: STRING                 -- Prefix is redundant
Wrong: get_count: INTEGER               -- Use simple names instead
```

Why `get_*` is discouraged:

- Queries are already distinguished by returning values
- `item` is clearer than `get_item`

- `name` is simpler than `get_name`
- `count` is more idiomatic than `get_count`
- Eiffel convention favors simple nouns for queries

Type Conversion:

```
as_string_32: STRING_32    -- convert to STRING_32
as_integer: INTEGER        -- convert to INTEGER
to_string: STRING          -- alias for as_string_32
```

Predicates (boolean queries):

```
is_empty: BOOLEAN          -- container is empty
is_full: BOOLEAN           -- at capacity
has_header: BOOLEAN        -- CSV has header row
is_console_output: BOOLEAN -- output to console
```

Model Queries (specification):

```
model_keys: MML_SEQUENCE [STRING]     -- abstract sequence of keys
items_model: MML_MAP [STRING, VALUE]  -- mathematical model
context_model: MML_MAP [STRING, ANY]  -- logical state representation
```

13.2 Mutation features

Pattern: `set_*`, `put_*`, `add_*`, `remove_*`, `clear`, `reset_*`

Setters:

```
set_level (a_level: INTEGER)           -- change property
set_max_size (a_size: INTEGER)         -- update configuration
set_output_file (a_path: STRING)       -- set destination
put (a_key: STRING; a_value: VALUE)    -- store key-value
put_with_ttl (a_key: STRING; a_value: VALUE; a_ttl: INTEGER)
```

Adders:

```
add (a_item: VALUE)                    -- append to collection
add_context (a_key: STRING; a_value: ANY)  -- add to context map
extend (a_item: VALUE)                 -- Eiffel standard (not add)
```

Removers:

```
remove (a_key: STRING)                 -- delete item
clear                                  -- remove all items
reset                                  -- return to initial state
evict_lru                              -- remove least-recently-used
prune_expired                          -- remove expired items
```

Fluent API (Builder Pattern): *(see section 22.3 for full rationale)*

```
set_option (a_value: TYPE): like Current
    do
        option := a_value
```

```
        Result := Current
    end
```

13.3 Creation features

Standard Constructors:

```
make                                    -- basic initialization
make_with_ttl (a_ttl: INTEGER)          -- with argument
make_with_level (a_level: INTEGER)      -- alternative
make_from_string (a_string: STRING)     -- parsing constructor
make_to_file (a_path: STRING)           -- configured creation
make_child (a_parent: LOGGER)           -- hierarchical creation
```

Create Clause:

```
create
    make,
    make_with_ttl,
    make_with_level,
    make_from_string
```

13.4 Feature organization

Standard Section Organization:

```
feature {NONE} -- Initialization
    make
    make_with_values

feature -- Access
    count: INTEGER
    item (a_index: INTEGER): VALUE
    is_empty: BOOLEAN
    has (a_key: STRING): BOOLEAN

feature -- Element change
    put (a_key: STRING; a_value: VALUE)
    remove (a_key: STRING)

feature -- Configuration
    set_max_size (a_size: INTEGER)
    set_level (a_level: INTEGER)

feature -- Model queries
    model_entries: MML_MAP [STRING, VALUE]
    model_keys: MML_SEQUENCE [STRING]

feature {NONE} -- Implementation
    internal_table: HASH_TABLE [VALUE, STRING]
    max_size: INTEGER
```

---

14 Argument naming

14.1 The a_ prefix convention

Principle: The `a_` prefix is the **default** because the IDE editor **ALWAYS prevents simultaneous viewing of the feature signature and the code body where arguments are used**.

14.2 Rationale

The **feature** signature appears at the top of the **feature**. Code body appears below. **You can never see both simultaneously** (except in trivial one-liners):

```
-- Feature signature (top of screen)
process_items (a_items: LIST [ITEM]; a_filter: FUNCTION [ITEM, BOOLEAN])
    -- Screen scrolls down to show body
    -- Signature is now OFF screen
    local
        l_count: INTEGER
        l_result: ARRAYED_LIST [ITEM]
    do
        from l_count := 1 until l_count > a_items.count loop
            -- When reading HERE, signature is not visible
            -- Therefore, a_ prefix is ESSENTIAL for reader to know
            -- that a_items comes from the signature, not from a local
            if a_filter.item ([a_items [l_count]]) then
                l_result.extend (a_items [l_count])
            end
            l_count := l_count + 1
        end
    end
```

14.3 Default usage

USE `a_` prefix by default because:

- Signature and usage are never simultaneously visible
- Reader needs to understand "this identifier is an argument"
- Prefix provides essential scope disambiguation
- Any routine longer than ~3 lines loses signature visibility

```
-- STANDARD: Use a_ for clarity
put (a_key: STRING; a_value: VALUE)
    require
        key_not_void: a_key /= Void
    local
        l_index: INTEGER
    do
        l_index := find_slot (a_key)
        if l_index > 0 then
            entries [l_index] := a_value
        else
            add_new_entry (a_key, a_value)
        end
    ensure
        item_stored: has (a_key)
    end
```

14.4 Exceptions

OMIT `a_` prefix only when:

- Routine is a trivial one-liner where the meaning is unambiguous
- Semantic context makes "this is an argument" crystal clear to any reader
- You can guarantee reader understanding without needing the prefix

```
-- EXCEPTION: Trivial routine, context is unmistakable
square (x: INTEGER): INTEGER do Result := x * x end
-- Reader instantly understands: x must be the argument

-- EXCEPTION: Idiomatic one-liners
increment (n: INTEGER): INTEGER do Result := n + 1 end
abs (v: INTEGER): INTEGER do Result := if v < 0 then -v else v end

-- EXCEPTION: The semantic context makes it unavoidable that it's an argument
get_item (index: INTEGER): ITEM do Result := items [index] end
-- Reader cannot confuse 'index' - the syntax forces it to be an argument
```

14.5 Examples

```
-- GOOD: Standard practice with a_ prefix
process (a_list: LIST [ITEM])
    local
        l_count: INTEGER
        l_total: INTEGER
    do
        from l_count := 1 until l_count > a_list.count loop
            l_total := l_total + a_list [l_count].value
            l_count := l_count + 1
        end
    end

-- GOOD: Complex routine, a_ is essential for clarity
validate_and_store (a_items: LIST [ENTRY]; a_strict: BOOLEAN)
    local
        l_count: INTEGER
        l_current: ENTRY
        l_errors: LIST [STRING]
        l_is_valid: BOOLEAN
    do
        from l_count := 1 until l_count > a_items.count loop
            l_current := a_items [l_count]
            l_is_valid := validate (l_current, a_strict)
            if not l_is_valid then
                l_errors.extend (error_for (l_current))
            else
                store (l_current)
            end
            l_count := l_count + 1
        end
        if l_errors.is_empty then
            commit
```

```
        end
    end
```

14.6 Design rationale

- **IDE Limitation:** Feature signatures and code bodies are never simultaneously visible (except one-liners)
- **Reader Clarity:** When scrolled past the signature, the `a_` prefix is the ONLY signal that an identifier comes from the argument list
- **Consistency:** Uniform naming enables predictable reading patterns
- **Safety:** Using `a_` everywhere prevents mistakes and errors
- **Eiffel Standard:** ISE Eiffel libraries and ISE Eiffel Base library use this convention

14.7 Anti-patterns

Wrong: INCONSISTENT (creates reader confusion):

```
-- Mixed styles without a_ = reader confusion
put (key: STRING; a_value: VALUE)
    local
        l_index: INTEGER
    do
        l_index := find_slot (key)  -- Is 'key' a argument? local? attribute?
        if l_index > 0 then
            entries [l_index] := a_value  -- Only a_value is clearly an argument
        end
    end
```

Wrong: RISKY (missing prefix obscures scope):

```
-- No a_ prefix in multi-line routine = readability loss
validate (entries: LIST [ENTRY])
    local
        count: INTEGER
        current: ENTRY
    do
        from count := 1 until count > entries.count loop
            current := entries [count]
            -- At line 50 of this routine, is 'current' a argument?
            -- Ambiguity = bug risk
        end
    end
```

Correct: CLEAR (consistent a_ prefix):

```
-- Consistent a_ prefix = unambiguous scope
validate (a_entries: LIST [ENTRY])
    local
        l_count: INTEGER
        l_current: ENTRY
    do
        from l_count := 1 until l_count > a_entries.count loop
            l_current := a_entries [l_count]
            -- Reader instantly knows: a_entries is a argument
            -- l_current and l_count are locals
```

```
        end
    end
```

14.8 Summary

Use `a_` prefix by default. The IDE editor structure (signature at top, body below) means you can almost never see both simultaneously. The prefix is your signal to the reader: "this identifier comes from the **feature** signature." Omit it only in trivial one-liners where meaning is unmistakable.

---

15 Local variable naming

15.1 The l_ prefix convention

Principle: The `l_` prefix is the **default** because the IDE editor **ALWAYS prevents simultaneous viewing of the `local` declaration and the code body where locals are used**.

15.2 Rationale

The `local` declaration appears near the top of the **feature**. Code body appears below. **You can never see both simultaneously** (except in trivial one-liners):

```
-- Local declaration (near top of feature)
    local
        l_count: INTEGER
        l_result: ARRAYED_LIST [ITEM]
    -- Screen scrolls down to show body
    -- Local declaration is now OFF screen
    do
        from l_count := 1 until l_count > max_items loop
            -- When reading HERE, local declaration is not visible
            -- Therefore, l_ prefix is ESSENTIAL for reader to know
            -- that l_count and l_result come from the local clause
            l_result.extend (get_item (l_count))
            l_count := l_count + 1
        end
    end
```

15.3 Default usage

USE `l_` prefix by default because:

- Local declaration and usage are never simultaneously visible
- Reader needs to understand "this identifier is a local variable"
- Prefix provides essential scope disambiguation (vs. arguments, attributes, cursors)
- Any routine longer than ~3 lines loses local declaration visibility

```
-- STANDARD: Use l_ for clarity
process_collection (a_items: LIST [ITEM])
    local
        l_count: INTEGER           -- clear it's local, not argument
        l_result: ARRAYED_LIST [ITEM]
        l_current: ITEM
        l_is_valid: BOOLEAN
    do
        from l_count := 1 until l_count > a_items.count loop
```

```
            l_current := a_items [l_count]
            l_is_valid := validate (l_current)
            if l_is_valid then
                l_result.extend (l_current)
            end
            l_count := l_count + 1
        end
    end
```

15.4 Exceptions

OMIT `l_` prefix only when:

- Routine is a trivial one-liner where the meaning is unambiguous
- Semantic context makes "this is a local variable" crystal clear to any reader
- You can guarantee reader understanding without needing the prefix
- **Loop counters in small, stable routines**: If a routine is reasonably sized and unlikely to grow beyond the editor viewport (stable in terms of future changes), simple counter names like `i`, `j`, `k` are acceptable without the `l_` prefix

```
-- EXCEPTION: Trivial routine, context is unmistakable
sum (a_list: LIST [INTEGER]): INTEGER
    local
        total: INTEGER
    do
        across a_list as i loop total := total + i.item end
        Result := total
    end
-- Reader can see the whole thing on one screen, context is clear

-- EXCEPTION: Loop variable in idiomatic context (small, stable routine)
is_sorted (a_comparator: FUNCTION [G, G, BOOLEAN]): BOOLEAN
    local
        i: INTEGER
    do
        Result := True
        from i := 1 until i >= count loop
            if a_comparator (items [i + 1], items [i]) then
                Result := False
            end
            i := i + 1
        end
    end
-- Routine is 10 lines, simple loop structure, unlikely to grow
-- The context (from/until loop with i as counter) is so idiomatic that i is instantly understood

-- EXCEPTION: Simple loop in stable routine
test_extend
    local
        i: INTEGER
    do
        create l_list.make
        from i := 1 until i > 100 loop
            l_list.extend (i * 10)
            assert ("count_increases", l_list.count = i)
```

```
            i := i + 1
        end
    end
-- Routine structure is simple and stable; loop counter is clear

-- EXCEPTION: Single-character convenience in obvious context
c := '%N'  -- single char assignment is self-explanatory
n := items.count  -- 'n' for count is idiomatic
```

15.5 Loop counter exceptions

Use simple counter names (`i` , `j` , `k`) **without** `l_` **prefix** when:

1. **Routine size is small** (~10-15 lines max, easily visible in viewport)
2. **Routine structure is stable** (unlikely to grow significantly in future)
3. **Loop counter use is straightforward** (simple `from/<span style="color:#0066cc;font-weight:bold">until</span>` **loop**, no complex nesting)
4. **Reader context is clear** (idiomatic **loop** patterns are instantly recognized)

Use `l_i` , `l_j` , `l_k` **with** `l_` **prefix** when:

1. **Routine is longer** (grows beyond viewport, scrolling required)
2. **Routine may grow** (future maintenance may add logic)
3. **Multiple nested loops** (complexity requires explicit scope markers)
4. **Mixed variable types** (l_ prefix distinguishes from arguments and attributes)

15.6 Examples

```
-- GOOD: Standard practice with l_ prefix
process_and_validate (a_items: LIST [ITEM]; a_strict: BOOLEAN)
    local
        l_count: INTEGER
        l_current: ITEM
        l_is_valid: BOOLEAN
    do
        from l_count := 1 until l_count > a_items.count loop
            l_current := a_items [l_count]
            l_is_valid := validate (l_current, a_strict)
            if l_is_valid then
                store (l_current)
            end
            l_count := l_count + 1
        end
    end

-- GOOD: Complex routine, l_ is essential for clarity
validate_entries (a_entries: LIST [ENTRY])
    local
        l_count: INTEGER
        l_entry: ENTRY
        l_errors: LIST [STRING]
        l_is_critical: BOOLEAN
    do
        from l_count := 1 until l_count > a_entries.count loop
            l_entry := a_entries [l_count]
            if not validate (l_entry) then
```

```
                l_errors.extend (error_message (l_entry))
                l_is_critical := check_criticality (l_entry)
                if l_is_critical then
                    log_critical (l_entry)
                end
            end
            l_count := l_count + 1
        end
        -- ... more logic ...
    end
```

15.7 Loop variable special cases

Across loop cursors (not locals, don't use `l_`):

```
across items as ic loop
    -- ic is a cursor, not l_ic
    -- Scope is limited to loop block, always visible
    process (ic.item)
end

across table as kc loop
    -- kc is a cursor
    print (@kc.key, kc)
end
```

Trivial traditional loops (exception: simple counter):

```
from i := 1 until i > count loop
    -- i is a traditional counter, context is unmistakable
    process (i)
    i := i + 1
end
```

15.8 Design rationale

- **IDE Structure:** Local declarations are at the top, code body is below. You can never see both simultaneously
- **Reader Safety:** The `l_` prefix is the ONLY signal to the reader: "this identifier is a local variable"
- **Scope Disambiguation:** Distinguishes locals from arguments (`a_`), attributes (no prefix), and cursors (`ic` , `rc`)
- **Consistency:** Uniform naming prevents errors and maintains readability
- **Eiffel Standard:** ISE Eiffel libraries and ISE Eiffel Base library use this convention

15.9 Anti-patterns

Wrong: INCONSISTENT (creates reader confusion):

```
-- No prefixes in longer routine = readability loss
validate (a_entries: LIST [ENTRY])
    local
        count: INTEGER      -- is this a argument? local? attribute?
        current: ENTRY      -- unclear scope when scrolled
        errors: LIST [STRING]
    do
        from count := 1 until count > a_entries.count loop
            current := a_entries [count]
```

```
            -- At line 30, reader cannot see local declaration
            -- Ambiguity = bug risk
        end
    end
```

Correct: CLEAR (consistent l_ prefix):

```
-- Consistent l_ prefix = unambiguous scope
validate (a_entries: LIST [ENTRY])
    local
        l_count: INTEGER
        l_current: ENTRY
        l_errors: LIST [STRING]
    do
        from l_count := 1 until l_count > a_entries.count loop
            l_current := a_entries [l_count]
            -- Reader instantly knows: l_count and l_current are locals
            -- a_entries is a argument
            if validate (l_current) then
                store (l_current)
            else
                l_errors.extend (error_message (l_current))
            end
            l_count := l_count + 1
        end
    end
```

15.10 Summary

Use `l_` prefix by default. The IDE editor structure (local declaration at top, body below) means you can almost never see both simultaneously. The prefix is your signal to the reader: "this identifier comes from the local clause." Omit it only in trivial one-liners where meaning is unmistakable.

---

16 Attachment local naming

16.1 The al_ prefix convention

MANDATORY for void-safe programming: All attachment locals use `al_` prefix to signal type narrowing via the `<span style="color:#0066cc;font-weight:bold">attached</span>` keyword.

16.1.1 Definition

The `<span style="color:#0066cc;font-weight:bold">attached</span>` keyword is part of Eiffel's **void-safety system**. It performs **type narrowing** — converting a `<span style="color:#0066cc;font-weight:bold">detachable</span>` (possibly **Void**) type into an `<span style="color:#0066cc;font-weight:bold">attached</span>` (never **Void**) type. The attachment local is the new variable bound within the `<span style="color:#0066cc;font-weight:bold">attached</span>` block that holds the narrowed type.

```
item: detachable STRING    -- might be Void

if attached item as al_item then
    -- Inside this block, al_item is guaranteed to be a non-Void STRING
    -- We can safely call methods on al_item without Void-checking
```

```
    print (al_item.to_upper)
end
```

16.1.2 Rationale

The `al_` prefix distinguishes attachment locals from:

- `a_` **arguments**: Function arguments
- `l_` **locals**: Regular local variables
- **No prefix attributes**: Class state

This visual distinction makes the **scope and type narrowing** explicit in code.

16.2 Standard patterns

Simple Attachment:

```
cache: detachable BOUNDED_CACHE [STRING, VALUE]

if attached cache as al_cache then
    -- al_cache is guaranteed attached (non-Void)
    al_cache.put ("key", "value")
end
```

Type Narrowing with Type Cast:

```
value: detachable ANY     -- might be Void or wrong type

if attached {INTEGER} value as al_int then
    -- al_int is narrowed to INTEGER and is attached
    process_integer (al_int)
end
```

Chained Attachment (nested):

```
parent: detachable PARENT
child: detachable CHILD

if attached parent as al_parent then
    if attached al_parent.child as al_child then
        -- Both al_parent and al_child are guaranteed attached
        process (al_parent, al_child)
    end
end
```

Attachment in Loop Condition:

```
across items as ic loop
    if attached ic.item as al_item then
        -- al_item is the narrowed, attached version
        process (al_item)
    end
end
```

16.2.1 Feature-level example

```
get_cached (a_key: STRING): detachable STRING
    -- Return cached value or Void if not found
    do
        Result := cache [a_key]
    ensure
        -- Result might be Void (that's OK, it's detachable)
    end

process_value (a_key: STRING)
    -- Safe caller: handle both Void and attached cases
    local
        l_value: detachable STRING
    do
        l_value := get_cached (a_key)

        if attached l_value as al_value then
            -- al_value is guaranteed attached, safe to use
            print (al_value)
        else
            print ("Value not found")
        end
    end
```

16.3 Contract context

Attachment locals are often used in **postconditions** to reference returned values:

```
get_or_default (a_key: STRING): detachable STRING
    -- Get cached value or Void if missing
    do
        Result := cache [a_key]
    ensure
        -- Can reference attachment in postcondition
        result_not_void_when_exists:
            (has (a_key) implies attached Result as al_result)
    end
```

16.4 Rationale

- **Void-Safety:** Makes type narrowing explicit and prevents null-pointer dereferences
- **Type Clarity:** Shows that a value has been checked and is safe to use
- **Code Readability:** The `al_` prefix signals "this variable is guaranteed non-Void in this scope"
- **Scope Limitation:** Attachment locals exist only within the `<span style="color:#0066cc;font-weight:bold">attached</span>` block — clear ownership
- **Eiffel Standard:** Required by ISE Eiffel's void-safe programming model

16.5 Anti-patterns

Wrong:

```
item: detachable STRING

if attached item as item then
    -- BAD: Shadows the argument, confusing
    print (item)
```

```
end

if attached value as v then
    -- BAD: No al_ prefix, unclear scope
    process (v)
end

if attached cache as l_cache then
    -- BAD: Uses l_ prefix (suggests regular local)
    -- But this is type-narrowed, should be al_cache
    cache_size := l_cache.count
end
```

Correct:

```
item: detachable STRING

if attached item as al_item then
    -- GOOD: Clear prefix, safe to use within block
    print (al_item)
end

if attached value as al_value then
    -- GOOD: al_ signals type narrowing
    process (al_value)
end

cache: detachable BOUNDED_CACHE [K, V]

if attached cache as al_cache then
    -- GOOD: Clearly a type-narrowed local
    cache_size := al_cache.count
end
```

---

17 Other variable binding contexts

Beyond local variables and attachment locals, Eiffel has several other keywords and contexts that create scoped variable bindings with distinct naming considerations.

17.1 Iteration cursors

Pattern: Loop cursor variables use `ic` or `ic_*` for iteration context

The `<span style="color:#0066cc;font-weight:bold">across</span>` **loop** creates a **cursor binding** that is distinct from regular local variables:

```
across my_list as ic loop
    -- ic is a cursor, not a regular local variable
    -- ic.item is the current element
    process (ic.item)
end

across my_table as ic loop
    -- For tables: ic IS the value, use @ic.key for the key
```

```
    print (@ic.key, ic)  -- key and value
end
```

Why `ic_` instead of `l_`?

Cursor variables are created by `across...as`, not by `local` declarations. Using `ic_` prefix:

- Signals "iteration cursor" at first glance
- Avoids **VOIT(2)** errors (cursor variable can't shadow **class** features)
- Distinguishes from regular local variables created in the `local` clause

Common Cursor Prefixes:

- `ic` — iteration cursor (default, often just `ic` without underscore)
- `rc` — result cursor (for iterating over result collections)
- `kc` — key cursor (for tables/maps)
- `vc` — value cursor (less common, `ic` is the value by default)

Rationale:
The **VOIT(2)** compiler error prevents cursor names from matching **feature** names in the enclosing **class**. Using `ic_` prefix ensures no conflicts:

```
-- BAD: If BOUNDED_CACHE has a feature named "item"
across my_list as item loop  -- VOIT(2) error: item shadows feature
    process (item)
end

-- GOOD: Use ic_ to avoid shadowing
across my_list as ic loop
    process (ic.item)
end
```

17.2 Check assertions

Pattern: Check assertions mark but do NOT create new bindings

The `check` keyword asserts that a condition holds for void-safety verification:

```
check attached my_detachable_entity end
-- Asserts to the compiler that my_detachable_entity is attached
-- Does not create a new variable
```

No Prefix Needed:

`check` does not introduce a new binding, so no naming convention applies. It asserts properties of **existing** variables.

Difference from `attached as`:

```
-- Creates NEW attached local:
if attached my_detachable as al_item then
    -- al_item is NEW, attached, scoped to this block
end

-- Asserts but does NOT create binding:
```

```
check attached my_detachable end
-- my_detachable is still detachable everywhere else
```

17.3 Exception handling

Pattern: Exception object is implicitly available in `<span style="color:#0066cc;font-weight:bold">rescue</span>` clauses

Exception handling creates an implicit binding, but with **important restrictions**:

```
feature some_risky_operation
    do
        call_external_c_library
    rescue
        if exception.is_developer_exception then
            print ("Developer error occurred")
        end
        retry
    end
```

CRITICAL RULE (from Eiffel gotchas):

> "NEVER Use **rescue**/**retry** for Internal Code. ONLY use when talking to external systems beyond the current Eiffel project universe (e.g. COM, external C libraries, network calls)."

No Prefix Needed:

The `exception` object is **implicit** and only available within `<span style="color:#0066cc;font-weight:bold">rescue</span>` clauses. It's automatically provided by the runtime, so no local declaration or prefix is needed.

Exception Context Limited To:

- `<span style="color:#0066cc;font-weight:bold">rescue</span>` clauses only
- Cannot be accessed outside **rescue** block
- Use for logging/cleanup of failed external calls
- NOT for internal logic errors (use Design by Contract instead)

17.4 Agent closures

Pattern: Agent arguments are part of the routine signature, not local bindings

Agents create functional references with potential argument binding:

```
-- Simple agent reference
add_action: PROCEDURE [ANY, TUPLE [INTEGER]] := agent increment (?)
    -- ? is unbound argument
    -- increment argument becomes bound when called

-- Inline agent with implicit arguments
sort_by_name: AGENT [TUPLE, TUPLE [G, G], BOOLEAN]
    do
        Result := agent (a: G; b: G): BOOLEAN
            do
```

```
                Result := a.name < b.name
            end
    end
```

No Prefix Needed:

Agent arguments are **not local variables** — they are part of the agent's signature. They don't need `a_` or `l_` prefixes within the agent:

```
agent (x: INTEGER; y: STRING): STRING
    do
        -- x and y are arguments of the agent routine
        -- They can be abbreviated since they're agent-scoped
        Result := y + x.out
    end
```

Critical Limitation (from gotchas.md):

> "Inline Agents Cannot Access Local Variables. Inline agents cannot capture or modify local variables from the enclosing scope."

This means attachment locals (`al_*`) created in the outer **feature** cannot be referenced inside inline agents.

17.5 Once functions

Pattern: Once functions bind `<span style="color:#0066cc;font-weight:bold">Result</span>` once, persisting **across** calls

Once functions cache their result and execute only once:

```
shared_logger: LOGGER
    -- Shared singleton instance
    once
        create Result.make_default
        Result.set_level (Level_debug)
    ensure
        result_not_void: Result /= Void
    end
```

No Prefix Needed:

The `<span style="color:#0066cc;font-weight:bold">Result</span>` binding is **implicit** in once functions. The name `<span style="color:#0066cc;font-weight:bold">Result</span>` is reserved for return values and doesn't need a prefix.

Once Variations:

```
-- Once per object (default)
logger: LOGGER
    once
        create Result
    end

-- Once per thread (with key)
logger_per_thread: LOGGER
```

```
    once (thread_id)
        create Result.make (thread_id)
    end
```

17.6 Local declarations

Pattern: Traditional local variables declared in `local` clause

Local variables are the most common binding context:

```
feature example
    local
        l_count: INTEGER           -- local variable
        l_result: STRING           -- local variable
        l_cache: HASH_TABLE [K, V] -- local variable
        i: INTEGER                 -- simple counter (no prefix)
    do
        from i := 1 until i > 10 loop
            l_count := l_count + 1
            i := i + 1
        end
    end
```

Prefix Convention:

- `l_` prefix: Used when clarity is needed (recommended for production code)
- **No prefix** for simple counters (`i`, `j`, `k`)
- **No prefix** for single-character convenience variables (`c` for char, `n` for count)

Example with Mixed Styles:

```
process_collection (a_items: LIST [ITEM])
    local
        i: INTEGER                  -- simple counter, no prefix
        l_count: INTEGER            -- multi-use variable, use l_
        l_result: STRING            -- multi-use variable, use l_
        c: CHARACTER                -- single char, no prefix
    do
        from i := 1 until i > a_items.count loop
            l_result := get_value (a_items [i])
            if l_result /= Void then
                l_count := l_count + 1
            end
            i := i + 1
        end
    end
```

17.7 Tuple labels

Pattern: Named components in tuples access via field names

Tuples can have labeled components:

```
TUPLE [x: INTEGER; y: INTEGER]     -- named components
TUPLE [key: STRING; value: ANY]    -- key and value

-- Access via field name:
my_tuple: TUPLE [x: INTEGER; y: INTEGER]
result_x := my_tuple.x    -- access by name
result_y := my_tuple.y
```

Important Limitation (from gotchas.md):

> "Named tuple field access via dot notation doesn't work in `<span style="color:#0066cc;font-weight:bold">across</span>` loops."

Workaround:

```
-- BAD: Named field access in across loop
across results as ic loop
    print (ic.item.x)  -- May not work correctly
end

-- GOOD: Use positional access in across loops
across results as ic loop
    print (ic.item [1])  -- Positional access is reliable
end
```

No Prefix Needed:

Tuple component names are accessed via dot notation, not declared as local variables, so no prefix convention applies.

17.8 Summary of all binding contexts

Binding Context	Syntax	Variables Created	Prefix	Scope	Notes
Object test	`<span style="color:#0066cc;font-weight:bold">attached</span> x as al_y`	`al_y`	`al_`	Conditional block	Void-safety binding
Iteration	`<span style="color:#0066cc;font-weight:bold">across</span> x as ic`	`ic`	`ic`	Loop block	VOIT(2) conflict risk
Check assertion	`check <span style="color:#0066cc;font-weight:bold">attached</span> x`	None	N/A	N/A	Asserts, no binding
Exception	`<span style="color:#0066cc;font-weight:bold">rescue</span>`	`exception`	Implicit	Rescue block	External calls only
Agent	`agent (x)`	None	N/A	Agent signature	Argument-based
Once	`once <span style="color:#0066cc;font-weight:bold">Result</span> :=`	`<span style="color:#0066cc;font-weight:bold">Result</span>`	Implicit	Once-only	Caching/singleto

Binding Context	Syntax	Variables Created	Prefix	Scope	Notes
Local	`local l_x: TYPE`	`l_x`	`l_` or none	Feature body	Informal for counters
Tuple labels	`TUPLE [x: TYPE]`	None	N/A	Tuple scope	Use positional in loops

---

18 Constants and enumerations

18.1 Naming pattern

Constants use **Proper_case** naming: each word begins with an uppercase letter, followed by lowercase letters, with underscores separating words. This applies to **all constants**, whether numeric enumerations, strings, characters, booleans, or configuration values.

Correct:

```
Correct: Default_ttl: INTEGER = 3600
Correct: Max_size: INTEGER = 1000
Correct: Version: STRING = "1.0.0"
Correct: Level_debug: INTEGER = 1
Correct: Level_error: INTEGER = 4
```

Incorrect:

```
Wrong: DEFAULT_TTL       -- All caps is wrong
Wrong: default_ttl       -- All lowercase is wrong
Wrong: DefaultTTL        -- CamelCase is wrong
Wrong: defaultTtl        -- camelCase is wrong
```

18.2 Numeric enumerations

Constants representing enumerated values follow Proper_case:

```
feature {NONE} -- Constants
    Level_debug: INTEGER = 1
    Level_info: INTEGER = 2
    Level_warn: INTEGER = 3
    Level_error: INTEGER = 4
    Level_fatal: INTEGER = 5

    Status_idle: INTEGER = 0
    Status_running: INTEGER = 1
    Status_paused: INTEGER = 2
    Status_finished: INTEGER = 3
```

18.3 String constants

```
feature {NONE} -- Constants
    Default_version: STRING = "1.0.0"
    Application_name: STRING = "BOUNDED_CACHE"
```

```
    Config_filename: STRING = "config.ini"
    Empty_string: STRING = ""
```

18.3.1 Character constants

```
feature {NONE} -- Constants
    Quote_char: CHARACTER = '"'
    Comma: CHARACTER = ','
    Newline: CHARACTER = '%N'
    Tab: CHARACTER = '%T'
    Space: CHARACTER = ' '
```

18.3.2 Boolean constants

```
feature {NONE} -- Constants
    Lenient_mode: BOOLEAN = False
    Strict_validation: BOOLEAN = True
    Include_headers: BOOLEAN = True
    Enable_caching: BOOLEAN = True
    Debug_output: BOOLEAN = False
```

18.3.3 Real constants

```
feature {NONE} -- Constants
    Default_timeout: REAL = 30.5
    Max_ratio: REAL = 0.85
    Min_threshold: REAL = 0.01
```

18.3.4 Rationale

- **Consistency**: All constants use the same Proper_case pattern regardless of type
- **Visibility**: Proper_case makes constants distinct from variables (which use `lowercase_with_underscores`)
- **Readability**: Word boundaries are clear (e.g., `Max_size` vs `maxsize`)
- **Eiffel Standard**: This convention is standard **across** ISE Eiffel libraries and ISE Eiffel Base library
- **Not all-caps**: Unlike some languages, Eiffel does NOT use ALL_CAPS for constants

---

18.4 Magic numbers and semantic constants

18.4.1 The problem

Magic numbers are literal values in code that reveal no semantic meaning:

```
-- WRONG: What does 3 mean? 3.14? Context is hidden.
if buffer.count > 3 then
    process (buffer)
end

if value = 3.14 then
    -- Is this Pi? A price? A ratio?
    calculate_circumference (radius)
end

from i := 1 until i > 100 loop
```

```
    -- Why 100? Maximum entries? Max iterations? Unknown.
    validate (entries [i])
    i := i + 1
end
```

Magic numbers create three problems:

1. **Ambiguity**: `3` could mean anything
2. **Maintenance risk**: If you need to change the value, you must find all occurrences and understand context
3. **Specification loss**: The *reason* for the value is invisible

18.4.2 The solution

Always replace magic numbers with semantically named constants:

```
-- GOOD: Name reveals semantic meaning
feature {NONE} -- Constants
    Min_buffer_size: INTEGER = 3
    Pi: REAL = 3.14159265358979
    Max_entries: INTEGER = 100

feature -- Element change
    process_if_ready (a_buffer: BUFFER)
        do
            if a_buffer.count > Min_buffer_size then
                process (a_buffer)
            end
        end

feature -- Calculation
    circumference (a_radius: REAL): REAL
        do
            Result := 2.0 * Pi * a_radius
        end

feature -- Validation
    validate_all (a_entries: LIST [ENTRY])
        local
            l_count: INTEGER
        do
            from l_count := 1 until l_count > Max_entries loop
                validate (a_entries [l_count])
                l_count := l_count + 1
            end
        end
```

18.5 Scoping constants

Constants must be scoped based on their usage:

1. Class-Scoped Constants

Use when: Constant is used only within one **class**

```
class BOUNDED_CACHE [G]
    feature {NONE} -- Constants
        Default_ttl: INTEGER = 3600           -- Time-to-live seconds
```

```
        Max_capacity: INTEGER = 1000          -- Maximum cache size
        Eviction_threshold: REAL = 0.85       -- Evict when 85% full

    feature -- Element change
        put (a_key: STRING; a_value: G)
            local
                l_size: INTEGER
            do
                l_size := entries.count
                if (l_size.to_real / Max_capacity.to_real) > Eviction_threshold then
                    evict_lru
                end
                entries [a_key] := a_value
            end
end
```

2. Module-Scoped Constants (Multiple Related Classes)

Use when: Multiple related classes share constants

Create a shared `*_CONST` **class** with **feature** groups and visibility filters:

```
class CACHE_CONST
    -- Shared constants for BOUNDED_CACHE and related classes

    feature -- Cache configuration
        Default_ttl: INTEGER = 3600
            -- Time-to-live for cached items (seconds)

        Max_capacity: INTEGER = 1000
            -- Maximum items in cache

        Eviction_threshold: REAL = 0.85
            -- Trigger eviction when ratio exceeds this

    feature {BOUNDED_CACHE, FILE_LOGGER} -- Logging constants
        Log_cache_hit: STRING = "Cache hit: [key]"
        Log_cache_miss: STRING = "Cache miss: [key]"
        Log_eviction: STRING = "Evicting [count] items"

    feature {BOUNDED_CACHE, CACHE_ENGINE} -- Internal implementation constants
        Hash_load_factor: REAL = 0.75
        Resize_multiplier: INTEGER = 2
end

class BOUNDED_CACHE [G]
    inherit
        -- No inheritance needed, just reference CACHE_CONST

    feature -- Element change
        put (a_key: STRING; a_value: G)
            do
                if entries.count > CACHE_CONST.Max_capacity then
                    evict_lru
                end
                entries [a_key] := a_value
            end
```

```
end

class CACHE_ENGINE
    feature {NONE} -- Implementation
        resize_table
            local
                l_new_size: INTEGER
            do
                l_new_size := current_size * CACHE_CONST.Resize_multiplier
                -- Resize logic
            end
end
```

3. System-Wide Constants

Use when: Constants apply **across** entire system (magic values, system limits, configuration)

```
class SYSTEM_CONST
    -- System-wide constants for all classes

    feature -- System configuration
        Application_name: STRING = "BOUNDED_CACHE"
        Application_version: STRING = "1.0.0"
        Max_concurrent_operations: INTEGER = 100
        Default_timeout_seconds: INTEGER = 30

    feature -- HTTP/Network constants
        Http_ok: INTEGER = 200
        Http_not_found: INTEGER = 404
        Http_server_error: INTEGER = 500

    feature -- Standard limits
        Max_string_length: INTEGER = 65536
        Max_array_size: INTEGER = 1000000
        Min_sleep_ms: INTEGER = 1

    feature {LOGGER, AUDIT_LOG} -- Logging levels
        Level_debug: INTEGER = 1
        Level_info: INTEGER = 2
        Level_warn: INTEGER = 3
        Level_error: INTEGER = 4
        Level_fatal: INTEGER = 5

    feature {VALIDATOR, PARSER} -- Validation rules
        Min_username_length: INTEGER = 3
        Max_username_length: INTEGER = 32
        Min_password_length: INTEGER = 8
end
```

18.5.1 Identifying magic numbers

Checklist for finding magic numbers in code:

- [ ] Numeric literals not assigned to named constants
- [ ] String literals that represent configuration or semantics (URLs, keys, messages)
- [ ] Floating-point values (e.g., 0.5, 3.14, 0.85)

- [ ] Array/collection size limits (e.g., 100, 1000)
- [ ] Time values (seconds, milliseconds): e.g., `3600` (1 hour)
- [ ] Status/code values (e.g., `1`, `2`, `3` for states)
- [ ] Ratios and thresholds (e.g., `0.85` for eviction trigger)

Red flag patterns:

```
-- BAD: Magic number without context
if count > 10 then ...            -- What is 10?

-- BAD: Repeated magic number
if x = 100 then ...
if y < 100 then ...               -- Why appears twice?

-- BAD: Magic string
send_to_url ("http://api.example.com/v2/cache")

-- BAD: Time value without semantic meaning
sleep (3600)                       -- Is this 1 hour? 1 second? Unknown.
```

18.5.2 Declaration and usage patterns

Standard pattern for defining and using constants:

```
class MY_CLASS
    feature {NONE} -- Constants (private to this class)
        Max_retries: INTEGER = 3
        Timeout_ms: INTEGER = 5000
        Default_encoding: STRING = "UTF-8"

    feature -- Operations
        connect_with_retry (a_server: STRING)
            local
                l_attempt: INTEGER
                l_connected: BOOLEAN
            do
                from l_attempt := 1 until l_attempt > Max_retries or l_connected loop
                    l_connected := try_connect (a_server, Timeout_ms)
                    if not l_connected then
                        l_attempt := l_attempt + 1
                    end
                end
            end
end

-- Reference shared constants
class CACHE_USER
    feature -- Usage
        cache_get (a_key: STRING)
            do
                if not cache.has (a_key) then
                    -- Use system-wide constant
                    log (SYSTEM_CONST.Log_cache_miss)
                end
```

```
            end
end
```

18.6 Configuration constants

Enhancement: When magic numbers represent **values that should be computed or loaded once and cached**, use `once` functions. This applies to:

1. **External configuration**: Values from config files, environment variables, databases
2. **Computed internal values**: Mathematical constants, derived calculations, singleton instances

In both cases, `once` ensures the value is computed/loaded exactly once, then cached for reuse:

Example 1: External Configuration (loaded once, cached)

```
class APPLICATION_CONFIG
    -- System-wide configuration loaded from external sources

    feature -- External configuration (read once, cached forever)
        Database_url: STRING
            once
                Result := load_config_file ("database.url")
            end

        Max_concurrent_connections: INTEGER
            once
                Result := load_config_file ("max_connections").to_integer
            end

        Cache_ttl_seconds: INTEGER
            once
                Result := load_config_file ("cache.ttl_seconds").to_integer
            end

    feature {NONE} -- Implementation
        load_config_file (a_key: STRING): STRING
            do
                -- Read from config.ini, environment variables, etc.
            end
end
```

Example 2: Computed Values (calculated once, cached)

```
class CACHE_CONSTANTS
    -- Mathematical and derived constants computed once

    feature -- Mathematical constants (computed once, reused)
        Pi: REAL
            once
                Result := 3.14159265358979
            end

        Sqrt_two: REAL
            once
                Result := 1.41421356237
            end
```

```
    feature -- Derived constants (calculated from other values)
        Max_buffer_size_bytes: INTEGER
            once
                -- Derived from configuration and item size
                Result := APPLICATION_CONFIG.Max_buffer_items *
                          Item_size_bytes
            end

        Eviction_threshold_count: INTEGER
            once
                -- Calculated threshold based on max capacity
                Result := (APPLICATION_CONFIG.Max_capacity *
                           Eviction_ratio).truncated_to_integer
            end

        Eviction_ratio: REAL = 0.85

    feature -- Singleton instances (created once, reused)
        Logger_instance: LOGGER
            once
                create Result.make_default
            end

        Cache_instance: BOUNDED_CACHE [STRING, ANY]
            once
                create Result.make_with_capacity (
                    APPLICATION_CONFIG.Max_capacity)
            end
end
```

Usage throughout the system:

```
class BOUNDED_CACHE [G]
    feature -- Element change
        put (a_key: STRING; a_value: G)
            do
                if entries.count > MAX_CAPACITY then
                    -- Use configuration constant
                    evict_lru
                end
                entries [a_key] := a_value
            end

    feature {NONE}
        Max_capacity: INTEGER
            once
                Result := APPLICATION_CONFIG.Cache_max_items
            end
end

class DATABASE_CONNECTOR
    feature -- Connection
        connect: DATABASE_CONNECTION
            do
                -- Use configuration constant (read once, reused)
```

```
                Result := create_connection (APPLICATION_CONFIG.Database_url)
            end
end
```

Process-Local or Thread-Local Configuration:

When configuration varies **per process** or **per thread**, use `once` with string keys:

```
class LOGGER
    feature -- Thread-local logging configuration
        Log_level: STRING
            once ("THREAD")
                -- Each thread can have different log level
                Result := load_thread_config ("log_level")
            end

        Output_file: STRING
            once ("THREAD")
                -- Each thread logs to its own file
                Result := "/logs/thread_" + thread_identifier + ".log"
            end

    feature {NONE}
        thread_identifier: STRING
            do
                -- Get current thread identifier
                Result := current_thread.id.out
            end
end

class CACHE_POOL
    feature -- Per-process cache configuration
        Cache_size_per_process: INTEGER
            once ("PROCESS")
                -- Each process has its own cache size configuration
                Result := load_process_config ("cache_size").to_integer
            end

        Node_cache_singleton: BOUNDED_CACHE [STRING, ANY]
            once ("OBJECT")
                -- Once per object instance (must synchronize in multithreaded context)
                create Result.make_with_capacity (Cache_size_per_process)
            end
end
```

Key differences:

Scenario	Pattern	Caching	Use Case
Hard-coded constant	`<span style="color:#0066cc;font-weight:bold">feature</span> {NONE} -- Constants: Max_size: INTEGER = 1000`	Compile-time	Values that never change
System-wide config	`<span style="color:#0066cc;font-weight:bold">feature</span> Max_size: INTEGER once ... end` or `once ("PROCESS")`	Global cache	Configuration from files/env
Per-thread config	`<span style="color:#0066cc;font-weight:bold">feature</span> Size: STRING once ("THREAD")`	Per-thread cache	Different per thread

Scenario	Pattern	Caching	Use Case
	`... end`		
Per-object singleton	`<span style="color:#0066cc;font-weight:bold">feature</span> Singleton: TYPE once ("OBJECT") ... end`	Per-object cache	Different per object instance

Why this matters:

- **Configuration externalized**: Values come from config files, not hardcoded
- **Loaded once**: The `once` function reads from external source only once, then caches
- **System flexibility**: Change configuration without recompiling code
- **Performance**: External configuration is read once, then reused
- **Semantic clarity**: `APPLICATION_CONFIG.Database_url` is self-documenting vs. `"postgres://db.example.com:5432"`

18.6.1 Rationale

- **Self-documenting code**: `Default_ttl: INTEGER = 3600` is clearer than literal `3600`
- **Single source of truth**: Change the constant once, affects all usages
- **Maintenance safety**: Find constant definition, understand the *why*, update confidently
- **Specification clarity**: Constant names document intent and constraints
- **Testability**: Test with named constants is easier to understand and verify
- **Collaboration**: Team members understand code without asking "what does 42 mean?"
- **Configuration flexibility**: External configuration via `once` functions allows system reconfiguration without code changes

---

19 Attribute naming

19.1 Pattern

Attributes are private (`<span style="color:#0066cc;font-weight:bold">feature</span> {NONE}`) and use direct lowercase names:

```
feature {NONE} -- Implementation
    entries: HASH_TABLE [VALUE, STRING]
    access_order: ARRAYED_LIST [STRING]
    expiration_times: HASH_TABLE [INTEGER, STRING]
    context_fields: HASH_TABLE [STRING, ANY]
    max_size: INTEGER
    default_ttl: INTEGER
    file_path: STRING
    is_console_output: BOOLEAN
```

19.2 State attributes

```
items: ARRAYED_LIST [G]          -- main data
rows: ARRAYED_LIST [ROW]         -- CSV/tabular data
header_map: HASH_TABLE [STRING, INTEGER]  -- column mapping
cache_entries: HASH_TABLE [K, V] -- key-value storage
```

19.3 Flag attributes

```
is_connected: BOOLEAN
is_empty: BOOLEAN
has_error: BOOLEAN
is_file_output: BOOLEAN
is_json_output: BOOLEAN
```

19.4 Configuration attributes

```
level: INTEGER                  -- log level
max_size: INTEGER               -- capacity
default_ttl: INTEGER            -- time-to-live seconds
file_path: STRING               -- output destination
delimiter: CHARACTER            -- CSV field separator
```

19.5 Statistics attributes

```
hits: INTEGER                   -- cache hits
misses: INTEGER                 -- cache misses
evictions: INTEGER              -- items evicted
indent_level: INTEGER           -- logging indentation
```

---

20 Model query naming

20.1 Pattern

Model queries return **formal mathematical representations** used in contracts:

20.2 Usage in libraries

```
class BOUNDED_CACHE [G]
    ...
    feature -- Model queries
        model_keys: MML_SEQUENCE [STRING]
            -- Mathematical sequence of keys in LRU order

        model_entries: MML_MAP [STRING, G]
            -- Mathematical map of all key-value pairs
```

20.3 Postcondition example

```
put (a_key: STRING; a_value: G)
    -- Store key-value pair
    ensure
        item_stored: model_entries.has (a_key)
        value_correct: model_entries [a_key] = a_value
        -- Uses model_entries for specification, not implementation
    end
```

20.4 Invariant example

```
invariant
    capacity_preserved: model_keys.count <= max_size
```

```
    consistency: model_entries.count = internal_table.count
    -- Invariants reference mathematical model
```

20.5 Common model names

```
model_keys: MML_SEQUENCE [K]              -- keys in order
model_entries: MML_MAP [K, V]             -- key-value pairs
model_context: MML_MAP [STRING, ANY]      -- logging context
items_model: MML_SEQUENCE [G]             -- list contents
rows_model: MML_SEQUENCE [MML_SEQUENCE [STRING]]  -- CSV rows
header_map_model: MML_MAP [STRING, INTEGER]      -- column indices
```

20.6 Rationale

- **model_** prefix clearly marks specification-level features
- Used **only** in contracts (postconditions, invariants)
- Separates **what** (specification) from **how** (implementation)
- Enables formal verification independent of code changes

---

21 Design by Contract patterns

21.1 Precondition naming

Tag names are descriptive, lowercase_with_underscores:

```
require
    key_not_empty: not a_key.is_empty
    positive_size: a_max_size > 0
    valid_ttl: a_ttl_seconds >= 0
    target_exists: target /= Void
    index_in_bounds: 1 <= a_index and a_index <= count
```

21.2 Postcondition naming

Similar convention, describing the guarantee:

```
ensure
    item_added: count = old count + 1
    item_stored: model_entries.has (a_key)
    value_correct: model_entries [a_key] = a_value
    capacity_respected: model_keys.count <= max_size
```

21.3 Invariant naming

Describe the always-true property:

```
invariant
    non_negative_hits: hits >= 0
    valid_max_size: max_size > 0
    model_consistent: model.count = internal_list.count
    target_invariant: target /= Void
```

---

22 Command-query separation patterns

22.1 Query features

```
-- No side effects, return value
count: INTEGER
item (a_index: INTEGER): VALUE
is_empty: BOOLEAN
has (a_key: STRING): BOOLEAN
get (a_key: STRING): VALUE
```

22.2 Command features

```
-- Modify state, return nothing
put (a_key: STRING; a_value: VALUE)
remove (a_key: STRING)
clear
set_max_size (a_size: INTEGER)
add (a_item: VALUE)
```

22.3 Fluent commands (the Builder Pattern in Eiffel)

The CQS objection

Experienced Eiffel programmers will immediately object to this pattern:

```
set_level (a_level: INTEGER): like Current
    do
        level := a_level
        Result := Current
    end
```

The objection is understandable: this **feature** **modifies state** (sets `level`) **and** returns a value (`like <span style="color:#0066cc;font-weight:bold">Current</span>`). On its face, this appears to violate Command-Query Separation — the principle that a **feature** should either change state (command) or return information (query), never both.

Why this is NOT a CQS violation

The purpose of CQS is to prevent a situation where you must **call a command to obtain information**. A true CQS violation would be a `remove` command that returns the removed element — forcing the caller to execute a side effect just to get data.

The fluent pattern is fundamentally different:

1. **No new information is returned.** The return value is `<span style="color:#0066cc;font-weight:bold">Current</span>` — the exact same object the caller already holds. The caller learns nothing new by receiving it.
2. **The feature is semantically a command.** Its purpose is to modify state. The return type exists solely for syntactic convenience.
3. **The caller never needs the return value.** You can always call `set_level (5)` as a plain command and ignore the result. The chaining form is optional sugar. To explicitly discard the final `<span style="color:#0066cc;font-weight:bold">Result</span>` in a chain, terminate with `.do_nothing`:

```
-- Swallow the Result at the end of a fluent chain
fl
    .set_level (Level_debug)
    .set_output_path ("debug.log")
    .set_format (Format_json)
    .do_nothing                    -- discards the final like Current Result
```

The `do_nothing` procedure is inherited from `ANY` — every Eiffel object has it. Because it is a procedure (not a function), the chain ends as a proper command call with no unused return value. This satisfies both the compiler and CQS purists.

In CQS terms: a query tells you something you did not know; a command changes something. Returning `<span style="color:#0066cc;font-weight:bold">Result</span>` (i.e. `<span style="color:#0066cc;font-weight:bold">Current</span>`) tells the caller nothing — it already has `<span style="color:#0066cc;font-weight:bold">Current</span>`. The return `<span style="color:#0066cc;font-weight:bold">Result</span>` is a routing convenience, not an information channel.

Why `like <span style="color:#0066cc;font-weight:bold">Current</span>` is essential (not just the class name)

The return type **must** be declared as `like <span style="color:#0066cc;font-weight:bold">Current</span>`, not as the concrete **class** type. This is not a stylistic choice — it is a type-safety requirement driven by Eiffel's inheritance model.

Consider this hierarchy:

```
class LOGGER
    feature -- Configuration
        set_level (a_level: INTEGER): like Current
            do
                level := a_level
                Result := Current
            end
end

class FILE_LOGGER
    inherit
        LOGGER
    feature -- File-specific configuration
        set_output_path (a_path: STRING): like Current
            do
                output_path := a_path
                Result := Current
            end
end
```

With `like <span style="color:#0066cc;font-weight:bold">Current</span>`, the inherited `set_level` adapts its return type:

```
-- On LOGGER: set_level returns LOGGER (like Current = LOGGER)
-- On FILE_LOGGER: set_level returns FILE_LOGGER (like Current = FILE_LOGGER)

local
    fl: FILE_LOGGER
do
    create fl.make
```

```
    fl
        .set_level (Level_debug)       -- returns FILE_LOGGER (not LOGGER)
        .set_output_path ("debug.log") -- compiles: FILE_LOGGER has set_output_path
end
```

If `set_level` had returned `LOGGER` instead of `like <span style="color:#0066cc;font-weight:bold">Current</span>`, the chain would break at `.set_output_path` — because the static type after `.set_level` would be `LOGGER`, which has no `set_output_path` **feature**. The programmer would be forced to use an explicit downcast, destroying the entire purpose of fluent chaining.

`like <span style="color:#0066cc;font-weight:bold">Current</span>` is an **anchored type**: it resolves to the dynamic type of the object at each point in the inheritance tree. This is what makes fluent APIs composable **across** inheritance hierarchies.

Practical comparison

Traditional imperative style (always valid):

```
create fl.make
fl.set_level (Level_debug)
fl.set_output_path ("debug.log")
fl.set_format (Format_json)
fl.enable_timestamps
```

Fluent style (equivalent, more concise):

```
create fl.make
fl
    .set_level (Level_debug)
    .set_output_path ("debug.log")
    .set_format (Format_json)
    .enable_timestamps
```

Both forms are valid. The fluent form groups related configuration calls visually, making the intent clearer — especially for builder-style initialization where many properties are set in sequence.

Implementation pattern

Every fluent command follows the same two-line body:

```
config_with_level (a_level: INTEGER): like Current
        -- Configure `level` to `a_level`.
    do
        level := a_level
        Result := Current
    ensure
        level_set: level = a_level
        same_object: Result = Current
    end
```

The postcondition `<span style="color:#0066cc;font-weight:bold">Result</span> = <span style="color:#0066cc;font-weight:bold">Current</span>` documents the contract explicitly: this **feature** returns the same object, not a copy, not a new object.

The `*_with_*` naming convention

A fluent **feature** uses the **infix** `_with_` naming pattern:

```
prefix_with_property
```

The **prefix** names the semantic frame — *what kind of specification* the chain expresses. The `_with_` infix is the universal fluent signal. The **suffix** names the property being set.

This convention is distinct from `set_*`, which remains a CQS-pure command with no return value. The reader knows from the name alone whether a **feature** is chainable.

Semantic frames

Different kinds of work call for different prefix verbs. The `_with_` infix is constant; the prefix changes to match the context:

Prefix	Semantic Frame	What the chain expresses
`config_with_*`	Property configuration	Setting operating parameters before use
`build_with_*`	Incremental assembly	Constructing a complex structure piece by piece
`format_with_*`	Presentation specification	Defining display or serialization rules
`filter_with_*`	Selection criteria	Narrowing a dataset by accumulating predicates
`connect_with_*`	Resource binding	Wiring to external services or endpoints

Each frame reads naturally in a chain:

```
-- Configuration frame: setting up an object's operating parameters
create logger.make
logger
    .config_with_level (Level_debug)
    .config_with_output ("debug.log")
    .config_with_format (Format_json)
    .do_nothing

-- Building frame: constructing a structure piece by piece
create report.make
report
    .build_with_title ("Q4 Results")
    .build_with_header (column_names)
    .build_with_row (row_1)
    .build_with_row (row_2)
    .build_with_footer (totals)
    .do_nothing

-- Filtering frame: narrowing a dataset by accumulating predicates
create query.make_from_table ("accounts")
query
    .filter_with_status ("active")
    .filter_with_date_range (start_date, end_date)
    .filter_with_minimum_balance (1000)
    .do_nothing
```

The prefix verb names the frame. A **class** may use only one frame (most common) or several if it genuinely operates in multiple modes. The table above lists common frames; programmers may introduce others following the same `prefix_with_property` pattern.

Anchoring: pairing fluent and pure-command features

A **class** may offer **both** a fluent **feature** and a CQS-pure command for the same property. The key design decision is: which one holds the canonical implementation?

Recommended pattern: the fluent **feature** IS the implementation. The pure command delegates to it:

```
feature -- Configuration (fluent)

    config_with_level (a_level: INTEGER): like Current
            -- Configure `level` to `a_level`.
        do
            level := a_level
            Result := Current
        ensure
            level_set: level = a_level
            same_object: Result = Current
        end

feature -- Configuration (command)

    set_level (a_level: INTEGER)
            -- Set `level` to `a_level`.
        do
            config_with_level (a_level).do_nothing
        ensure
            level_set: level = a_level
        end
```

This anchoring pattern has three properties:

1. **No code duplication.** The logic lives in `config_with_level`. Change it once, both features update.
2. **The pair is anchored.** The pure command delegates to the fluent version — they cannot drift apart.
3. `.do_nothing` **is explicit.** The pure command visibly discards the `<span style="color:#0066cc;font-weight:bold">Result</span>`, documenting the CQS boundary.

The programmer's design choice

The anchoring pattern above is a *recommendation*, not a mandate. The **class** designer chooses which features to expose based on how the **class** will be used:

Fluent only — when the **class** is primarily used in builder chains:

```
-- Only config_with_* features. No set_* commands.
-- Callers always chain or terminate with .do_nothing.
create logger.make
logger
    .config_with_level (Level_debug)
    .config_with_output ("debug.log")
    .do_nothing
```

Command only — when chaining adds no value:

```
-- Only set_* commands. No fluent features.
-- Traditional CQS-pure style.
```

```
logger.set_level (Level_debug)
logger.set_output ("debug.log")
```

Both — when the **class** serves both calling styles:

```
-- CQS-strict callers use set_*
logger.set_level (Level_debug)

-- Builder-style callers use config_with_*
logger
    .config_with_level (Level_debug)
    .config_with_output ("debug.log")
    .do_nothing
```

The choice depends on the **class**'s audience and usage patterns. A configuration-heavy **class** (database connector, logger, report builder) benefits from fluent features. A **class** with few settings or complex per-call semantics (`put` , `remove` , `extend`) should stick with pure commands.

When fluent chaining fits

- **Configuration sequences**: Multiple properties set during object setup
- **Builder/assembly patterns**: Constructing a complex structure incrementally
- **Optional settings**: Many defaults where the caller sets only a few
- **Filtering/query refinement**: Accumulating predicates or criteria

When fluent chaining does NOT fit

- **Structural mutations**: `put` , `remove` , `extend` — these are semantically distinct actions, not configuration. Keep them as pure commands.
- **Commands that can fail**: If the caller needs to check success after each step, chaining hides the control flow.
- **Single-call commands**: If a **feature** is typically called alone, the fluent return adds no value.

22.4 Naming convention implications

- **Queries** use nouns or `is_/has_` patterns
- **Commands** use imperative verbs: `set_` , `put_` , `add_` , `remove_` , `clear_`
- **Fluent features** use the `*_with_*` infix: `config_with_` , `build_with_` , `filter_with_` , `format_with_` , `connect_with_`
- **No mixing:** Never have a **feature** that returns new information AND modifies state
- **Fluent features return** `like <span style="color:#0066cc;font-weight:bold">Current</span>` — always, without exception — to preserve type safety **across** inheritance

---

23 Visibility and **feature** organization

23.1 Standard **feature** groups

EiffelStudio provides standard **feature** groupings when creating a new **class**. These organize features by type and visibility. Follow these conventions for consistent, professional code organization:

```
feature {NONE} -- Initialization
    -- Creation methods: make, make_*, default_create
    -- Visibility: {NONE} (private, called via create clause)
    -- Naming: make, make_with_*, make_from_*
```

```
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Access
    -- Query features (no side effects, return values)
    -- Visibility: Public (exported to all clients)
    -- Naming: lowercase_with_underscores (e.g., count, item, has, get)
    -- Naming: is_*, has_*, are_* for boolean queries
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Measurement
    -- Queries that measure or calculate numeric properties
    -- Visibility: Public
    -- Naming: lowercase_with_underscores (e.g., count, size, length, capacity)
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Status report
    -- Queries that report state/condition (boolean or status codes)
    -- Visibility: Public
    -- Naming: is_*, has_*, are_*, is_*_valid, is_*_empty
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Status setting
    -- Commands that change state/conditions
    -- Visibility: Public
    -- Naming: set_*, reset_*, enable_*, disable_*
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Cursor movement
    -- Commands that modify cursor/iteration position
    -- Visibility: Public
    -- Naming: forth, back, start, finish, go_to_*, move_to_*
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Element change
    -- Commands that modify collection/container elements
    -- Visibility: Public
    -- Naming: put, add, set_*, insert, replace, update
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Removal
    -- Commands that delete/remove elements
    -- Visibility: Public
    -- Naming: remove, remove_*, delete, clear, prune, wipe_out, evict
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Resizing
    -- Commands that change capacity/size
    -- Visibility: Public
    -- Naming: grow, shrink, resize, set_capacity, set_size, expand
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Transformation
    -- Commands that transform/convert data without removing
    -- Visibility: Public
    -- Naming: reverse, sort, shuffle, rotate, transpose, normalize
    -- Naming conventions: a_ prefixes for arguments, l_ for locals
```

```
feature -- Conversion
    -- Queries that convert to different types
    -- Visibility: Public
    -- Naming: as_*, to_*, convert_to_*
    -- Examples: as_string, to_integer, convert_to_seconds
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Duplication
    -- Queries that create copies
    -- Visibility: Public
    -- Naming: copy, duplicate, clone, deep_clone, deep_copy
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Miscellaneous
    -- Public features that don't fit standard categories
    -- Visibility: Public
    -- Naming: varies by purpose
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Basic operations
    -- Core algorithmic operations (sort, search, etc.)
    -- Visibility: Public
    -- Naming: sort, search, find, filter, map, reduce, fold
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Model queries
    -- Specification-level mathematical models for contracts
    -- Visibility: Public
    -- Naming: *_model, model_*, model_entries, model_keys, items_model
    -- Purpose: Used ONLY in postconditions, invariants, preconditions
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature {NONE} -- Implementation
    -- Private helper features
    -- Visibility: {NONE} (private, internal only)
    -- Naming: same conventions as public, but with semantic clarity that reader won't be confused
    -- Naming conventions: a_ prefixes for arguments, l_ for locals (consistency matters)
    -- Examples: internal_table, cache_entries, validate_internal

feature {SPECIALIZED_CLASS} -- Shared implementation
    -- Implementation shared with specific subclasses
    -- Visibility: {ChildClass} (limited visibility)
    -- Naming: same conventions as public
    -- Example: feature {FILE_LOGGER} for child loggers
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Obsolete
    -- Deprecated features (kept for backward compatibility)
    -- Visibility: Public (but marked for removal)
    -- Naming: same as original, documented as obsolete
    -- Naming conventions: a_ prefixes for arguments, l_ for locals

feature -- Inapplicable
    -- Features inherited but not applicable for this class
    -- Visibility: Public (but redefined as not_applicable)
```

```
    -- Naming: same as inherited
    -- Naming conventions: a_ prefixes for arguments, l_ for locals
```

23.2 Organization best practices

Declare features in this order:

1. `<span style="color:#0066cc;font-weight:bold">feature</span> {NONE} -- Initialization` (creation methods first)
2. Public **feature** groups in logical order (Access, Measurement, Status report, etc.)
3. `<span style="color:#0066cc;font-weight:bold">feature</span> -- Model queries` (specification features together)
4. `<span style="color:#0066cc;font-weight:bold">feature</span> {NONE} -- Implementation` (private helpers at end)
5. `<span style="color:#0066cc;font-weight:bold">invariant</span>` clause (after all features)

Rationale:

- Clients see initialization first (they create instances)
- Public API features are grouped by function
- Implementation details are hidden at the bottom
- Consistent organization **across** codebase enables predictable reading

23.3 Naming across feature groups

The naming conventions apply uniformly **across** all **feature** groups:

Feature Type	Naming Pattern	Visibility	Example
Access queries	`lowercase_with_underscores`	Public	`count`, `item`, `get`
Boolean queries	`is_*`, `has_*`, `are_*`	Public	`is_empty`, `has_error`
Commands	`set_*`, `put_*`, `add_*`, `remove_*`	Public	`set_level`, `put_with_ttl`
Creation	`make`, `make_*`	{NONE}	`make`, `make_with_ttl`
Conversion	`as_*`, `to_*`, `convert_to_*`	Public	`as_string`, `to_integer`
Model queries	`*_model`, `model_*`	Public	`model_entries`, `items_model`
Implementation	Same as public (consistency)	{NONE}	`internal_table`, `validate_internal`
Arguments (any group)	`a_*`	(all)	`a_key`, `a_value`, `a_level`
Locals (any group)	`l_*`	(all)	`l_count`, `l_result`, `l_cache`

---

24 Type annotations and genericity

24.1 Generic constraints

```
class BOUNDED_CACHE [G -> detachable separate ANY]
    -- G can be any type, including Void
    -- separate enables SCOOP (concurrent programming)

class LIST_UTILITIES [G -> detachable separate ANY]
    -- Consistent constraint across ecosystem
```

24.2 Void-safety annotations

```
detachable STRING          -- can be Void
detachable G               -- generic parameter can be Void
attached STRING            -- cannot be Void
```

24.3 Downcasting pattern

```
if attached {INTEGER} l_value as l_int then
    -- l_int is safely typed as INTEGER
    process_integer (l_int)
end
```

---

25 Anti-patterns

25.1 Shadowing TUPLE reserved features

```
-- BAD: Don't shadow TUPLE features
class MY_CLASS
    feature
        count: INTEGER     -- conflicts with TUPLE.count
        is_empty: BOOLEAN  -- conflicts with TUPLE.is_empty
        lower: INTEGER     -- conflicts with TUPLE.lower
```

25.2 Ambiguous argument names

```
put (key: STRING; value: VALUE)     -- no a_ prefix
set_size (sz: INTEGER)              -- cryptic abbreviation
add (x: ITEM)                        -- unclear
```

25.3 Ambiguous local variables

```
local
    count: INTEGER          -- is this argument or local?
    result: STRING          -- unclear scope
    cache: BOUNDED_CACHE [G] -- which cache?
```

25.4 Mixed case in classes

```
class LinkedList           -- should be UPPERCASE_WITH_UNDERSCORES (UPPERCASE)
class cacheEngine           -- should be CACHE_ENGINE
class Cache_Engine          -- should be CACHE_ENGINE (all caps)
```

25.5 Inconsistent feature naming

```
getCount: INTEGER           -- use lowercase_with_underscores
SetLevel (a_level: INTEGER) -- use set_level
Is_Empty: BOOLEAN           -- use is_empty
```

25.6 Side effects in queries

```
-- BAD: Query with side effects
get_and_log (a_key: STRING): VALUE
```

```
    do
        print ("Getting: " + a_key)  -- I/O in query!
        Result := table [a_key]
    end

-- GOOD: Separate command and query
log (a_key: STRING)        -- command: side effect only
    do
        print ("Getting: " + a_key)
    end

get (a_key: STRING): VALUE -- query: no side effects
    do
        Result := table [a_key]
    end
```

---

26 Alias patterns

26.1 Array-style access

```
item alias "[]" (i: INTEGER): VALUE
    do
        Result := data [i]
    end

-- Usage: my_cache [key]
```

26.2 Semantic aliasing: client-frame naming

Eiffel allows a single **feature** to carry multiple names (aliases). This is not mere convenience — it is a **semantic tool** that bridges the gap between a Supplier **class**'s internal vocabulary and the diverse vocabularies of its Clients.

Why semantic aliases exist

A Supplier **class** lives in one domain. Its Clients live in many. Consider a **class** `TRANSACTION_PROCESSOR` with a **feature** that commits a batch:

- A **banking** Client thinks of this as `post_batch` (accounting term)
- A **data-pipeline** Client thinks of this as `flush` (stream-processing term)
- A **testing** Client thinks of this as `finalize_pending` (workflow term)

All three names describe the same operation. Without aliases, two of these three Clients must use a name that reads unnaturally in their code. With aliases, each Client reads fluently within its own semantic frame:

```
feature -- Batch operations

    post_batch, flush, finalize_pending
            -- Commit all pending transactions.
            -- `post_batch`:        accounting/banking frame
            -- `flush`:             data-pipeline/stream frame
            -- `finalize_pending`:  workflow/testing frame
        do
            across pending_transactions as ic loop
                ic.item.commit
            end
```

```
            pending_transactions.wipe_out
        ensure
            all_committed: pending_transactions.is_empty
        end
```

The Client code reveals its own semantic context without requiring the reader to **inspect** the Supplier's implementation:

```
-- In a banking application:
processor.post_batch                    -- reads as accounting action

-- In a data pipeline:
processor.flush                         -- reads as stream operation

-- In a test harness:
processor.finalize_pending              -- reads as workflow step
```

Four properties of semantic aliasing

1. **Reader-frame independence.** A banker, a data engineer, and a test author each see a name natural to their domain. The Supplier code is written once; the vocabulary adapts to whoever is reading.
2. **Client-frame maintenance.** Each Client **class** maintains a consistent semantic vocabulary throughout its own code. A banking module that calls `post_batch` everywhere does not suddenly shift register to call `flush` — even though both resolve to the same **feature**. The alias set lets each Client stay in its own frame.
3. **Semantic expansion.** New Clients with new frames can be served by adding an alias without modifying any existing Client. The original **feature** name and all prior aliases remain stable. This is the Open/Closed Principle applied at the naming level.
4. **Meaning without inspection.** The alias name itself carries domain intent. A reader of the Client code can deduce *what is happening* and *why* without opening the Supplier **class** and reading the **feature** body. The name is the documentation.

Logging example (multiple semantic frames)

```
feature -- Logging (debug level)

    debug_log, trace, verbose (a_message: STRING)
            -- Log `a_message` at debug level.
            -- `debug_log`:  operations/sysadmin frame
            -- `trace`:      developer/debugging frame
            -- `verbose`:    CLI/user-facing frame
        do
            log_at_level (Level_debug, a_message)
        end

feature -- Logging (info level)

    info, log, log_info, message (a_message: STRING)
            -- Log `a_message` at info level.
            -- `info`:      terse/pipeline frame
            -- `log`:       generic/default frame
            -- `log_info`:  explicit-level frame
            -- `message`:   user-notification frame
        do
            log_at_level (Level_info, a_message)
        end
```

Each alias is a *semantic lens* on the same operation. The Client chooses the lens that fits its context.

Relationship to fluent features (section 22.3)

Semantic aliasing and fluent naming serve different purposes and should not be conflated:

- `set_level` is a CQS-pure command (no return value)
- `config_with_level` is a fluent **feature** (returns `like <span style="color:#0066cc;font-weight:bold">Current</span>`)

These are *not* aliases of each other — they have different signatures and different call semantics. As described in section 22.3, `set_level` may *delegate* to `config_with_level` via `.do_nothing`, but they are **separate** features, not names on the same **feature**.

Aliases apply when the operation is truly identical and only the vocabulary differs:

```
-- CORRECT: aliases for the same operation, same signature
set_level, set_log_level, assign_level (a_level: INTEGER)
        -- Set logging level.
        -- `set_level`:      default/terse frame
        -- `set_log_level`:  explicit-domain frame
        -- `assign_level`:   configuration/admin frame
    do
        level := a_level
    ensure
        level_set: level = a_level
    end

-- SEPARATE: fluent version is a different feature, not an alias
config_with_level (a_level: INTEGER): like Current
        -- Configure `level` to `a_level`. Returns Current for chaining.
    do
        level := a_level
        Result := Current
    ensure
        level_set: level = a_level
        same_object: Result = Current
    end
```

---

27 Testing and verification naming

27.1 Test **class** names

```
TEST_BOUNDED_CACHE       -- main test suite
LIB_TESTS                -- library-wide tests
ADVERSARIAL_TESTS        -- edge cases and boundary conditions
STRESS_TESTS             -- performance and load testing
TEST_JSON_SCHEMA         -- feature-specific tests
```

27.2 Test method names

```
test_make_creates_empty_cache
test_put_stores_value
test_get_retrieves_stored_value
test_count_increases_on_put
```

```
test_remove_deletes_item
test_eviction_when_full
test_ttl_expiration
test_concurrent_access  -- SCOOP test
```

27.3 Assertion tag names

```
assert (\"cache_empty_on_init\", cache.is_empty)
assert (\"count_increased\", cache.count = old cache.count + 1)
assert (\"value_correct\", cache.get (\"key\") = expected_value)
```

---

28 Complete example

```
class BOUNDED_CACHE [G -> detachable separate ANY]
    -- Thread-safe, size-limited cache with LRU eviction

    create
        make,
        make_with_ttl

    feature {NONE} -- Initialization

        make
            -- Create cache with default settings
            do
                create entries.make (100)
                create access_order.make
                max_size := 100
                default_ttl := 3600  -- 1 hour
            ensure
                is_empty: count = 0
                max_capacity_set: max_size = 100
            end

        make_with_ttl (a_ttl: INTEGER)
            -- Create with custom TTL in seconds
            require
                positive_ttl: a_ttl > 0
            do
                make
                default_ttl := a_ttl
            ensure
                ttl_set: default_ttl = a_ttl
            end

    feature {NONE} -- Implementation
        entries: HASH_TABLE [G, STRING]
        access_order: ARRAYED_LIST [STRING]
        expiration_times: HASH_TABLE [INTEGER, STRING]
        max_size: INTEGER
        default_ttl: INTEGER

    feature -- Access
```

```
        count: INTEGER
            -- Number of items currently in cache
            do
                Result := entries.count
            end

        item alias "[]" (a_key: STRING): G
            -- Retrieve value by key
            require
                key_not_empty: not a_key.is_empty
                key_exists: has (a_key)
            do
                Result := entries [a_key]
                update_access_order (a_key)
            end

        has (a_key: STRING): BOOLEAN
            -- Check if key exists and not expired
            do
                if entries.has (a_key) then
                    if is_expired (a_key) then
                        evict (a_key)
                        Result := False
                    else
                        Result := True
                    end
                end
            end

        is_empty: BOOLEAN
            -- Check if cache is empty
            do
                Result := entries.is_empty
            end

    feature -- Element change

        put (a_key: STRING; a_value: G)
            -- Store key-value pair with default TTL
            require
                key_not_empty: not a_key.is_empty
            do
                put_with_ttl (a_key, a_value, default_ttl)
            ensure
                item_stored: has (a_key)
                value_correct: item (a_key) = a_value
            end

        put_with_ttl (a_key: STRING; a_value: G; a_ttl: INTEGER)
            -- Store with custom TTL
            require
                key_not_empty: not a_key.is_empty
                positive_ttl: a_ttl > 0
            do
                entries [a_key] := a_value
                expiration_times [a_key] := get_current_time + a_ttl
```

```
            update_access_order (a_key)
            if entries.count > max_size then
                evict_lru
            end
        ensure
            item_stored: has (a_key)
            within_capacity: count <= max_size
        end

feature -- Removal

    remove (a_key: STRING)
        -- Delete item from cache
        do
            entries.remove (a_key)
            expiration_times.remove (a_key)
            access_order.prune_all (a_key)
        ensure
            item_removed: not has (a_key)
        end

    clear
        -- Remove all items
        do
            entries.wipe_out
            expiration_times.wipe_out
            access_order.wipe_out
        ensure
            cache_empty: is_empty
        end

feature -- Configuration

    set_max_size (a_size: INTEGER)
        -- Update cache capacity
        require
            positive_size: a_size > 0
        do
            max_size := a_size
            prune_to_capacity
        ensure
            size_set: max_size = a_size
            within_capacity: count <= max_size
        end

    set_ttl (a_ttl: INTEGER)
        -- Update default TTL
        require
            positive_ttl: a_ttl > 0
        do
            default_ttl := a_ttl
        ensure
            ttl_set: default_ttl = a_ttl
        end

feature -- Statistics
```

```
    hits: INTEGER
    misses: INTEGER
    evictions: INTEGER

feature -- Model queries

    model_entries: MML_MAP [STRING, G]
        -- Mathematical model of stored entries
        do
            create Result.make_empty
            across entries as e loop
                if not is_expired (e.key) then
                    Result := Result.extended (e.key, e.item)
                end
            end
        end

    model_keys: MML_SEQUENCE [STRING]
        -- Keys in LRU order
        do
            create Result.make_empty
            across access_order as c loop
                if entries.has (c.item) then
                    Result := Result.appended (c.item)
                end
            end
        end

feature {NONE} -- Implementation

    evict_lru
        -- Remove least-recently-used item
        local
            l_oldest_key: STRING
        do
            if not access_order.is_empty then
                l_oldest_key := access_order.first
                remove (l_oldest_key)
                evictions := evictions + 1
            end
        end

    update_access_order (a_key: STRING)
        -- Move key to end (most recently used)
        do
            access_order.prune_all (a_key)
            access_order.extend (a_key)
        end

    is_expired (a_key: STRING): BOOLEAN
        -- Check if TTL has passed
        do
            if expiration_times.has (a_key) then
                Result := expiration_times [a_key] <= get_current_time
            end
```

```
            end

        prune_to_capacity
            -- Remove items until within capacity
            do
                from until count <= max_size loop
                    evict_lru
                end
            end

        get_current_time: INTEGER
            -- Current Unix timestamp
            do
                -- Implementation
            end

    invariant
        valid_max_size: max_size > 0
        non_negative_hits: hits >= 0
        non_negative_misses: misses >= 0
        non_negative_evictions: evictions >= 0
        model_consistent: model_entries.count = entries.count
end
```

APPENDIX A: BNF-E NOTATION REFERENCE

This specification uses Extended Backus-Naur Form (BNF-E) as defined in ECMA-367:

```
Production ≜ Left_Hand_Side [ | Production_List ]

Left_Hand_Side ≜ Production_Name


Production_List ≜ Production { | Production }


Production ≜ Symbol | Production [ Symbol ]

| { Symbol }    -- 0 or more

| [ Symbol ]    -- 0 or 1

| ( Production )

| Production Production
```

APPENDIX B: BACKWARD COMPATIBILITY MATRIX

Feature	ECMA-367	2026 Edition	Migration
Classes	Correct:	Correct:	Direct
Inheritance	Correct:	Correct:	Direct
Design by Contract	Correct:	Correct:	Direct
Generics	Correct:	Correct:	Direct
Attached Types	Wrong:	Correct:	Gradual (**Void**-Safety None/Transitional/Complete)
ACROSS Loops	Wrong:	Correct:	Can coexist with traditional loops
SCOOP Concurrency	Wrong:	Correct:	Optional (use `<span style="color:#0066cc;font-weight:bold">separate</span>`)
Inline Agents	Correct: (5.7)	Correct:	Direct
Operator Aliases	Wrong:	Correct:	Additive
Bracket Notation	Wrong:	Correct:	Additive
Assigner Commands	Wrong:	Correct:	Additive
Multi-Branch Expressions	Wrong:	Correct:	Additive
Non-Conforming Inheritance	Wrong:	Correct:	Additive
Multiple Generic Constraints	Wrong:	Correct:	Additive
Note/Metadata Annotations	Correct: (**indexing**)	Correct: (note)	Keyword **rename**
Unicode Types	Correct: (minimal)	Correct:	Enhanced
Sized Integers	Correct: (some)	Correct:	Enhanced

APPENDIX C: FEATURE ADOPTION TIMELINE

Year	Version	Major Feature	Impact
2006	5.7	ECMA-367 Standardization	Baseline freeze
2007	6.1	**Void**-Safety (**attached**/**detachable**)	Prevents null errors
2010	6.5	ACROSS Loops	Better iteration
2011	7.0	SCOOP Full Support	Race-free concurrency
2013	7.3	**Void**-Safety Default	Mandatory safety
2015	15.12	Simplified Agent Types	Cleaner syntax
2017	17.05	Relaxed Conditional Typing	More flexible types
2020	20.05	Multi-Branch Expressions	Functional style
2021	21.11	Unified Cursor Access (@)	Better iteration API
2023	23.09	.NET Core Support	Modern runtime
2026	(This Edition)	ECMA-367:2026 Standard Update	Complete specification

---

APPENDIX D: VOID-SAFETY MIGRATION GUIDE

Phase 1: Assessment

1. Enable void-safety level: NONE (legacy compatibility)
2. Identify void-safety issues with void checker
3. Document code patterns that need updating

Phase 2: Transitional Migration

1. Set void-safety level: TRANSITIONAL
2. Mark obviously **detachable** entities with `?`
3. Add object tests where needed:

```
if attached optional_value as value then
  value.feature
end
```

```
### Phase 3: Complete Migration

1. Set void-safety level: COMPLETE
2. Verify all potential-void entities are marked `?`
3. All code paths must be void-safe
4. No unguarded calls on potentially-void references

### Phase 4: Maintenance

1. All new code written in COMPLETE mode
2. Leverage void-safety for correctness proofs
3. Use type system as specification

---

# APPENDIX E: SCOOP BEST PRACTICES

### 1. Processor Boundaries

Define clear processor boundaries early:

```eiffel
class APPLICATION
  ui_thread: separate UI_MANAGER
  worker_thread: separate WORKER
  database_thread: separate DATABASE

  feature
    main do
      -- Distribute work across processors
    end
end
```

2. Synchronization Points

Minimize synchronization:
```

```
-- GOOD: Batch operations, fewer sync points
separate_worker.process_batch (items)

-- AVOID: Multiple sync points
across items as cursor loop
  size := separate_worker.process (cursor.item)  -- Blocks each iteration
end
```

3. Lock-Passing

Understand lock semantics:

```
-- When you pass an object to separate routine,
-- you lose access to it temporarily
my_data: MY_DATA
worker: separate WORKER

worker.process (my_data)    -- my_data locked by worker
-- Cannot use my_data here

-- After worker.process returns, you regain access
```

APPENDIX F: CROSS-REFERENCES TO ECMA-367:2006

This standard maintains compatibility with ECMA-367:2006. The following sections are updated to incorporate new features:

- Section 8.2: Class and Feature Declaration (now includes SEPARATE, aliases, assigners)
- Section 8.5: Feature Declarations (now includes ALIAS, BRACKET, ASSIGNER clauses)
- Section 8.8: Type System (now includes void-safety marks, SEPARATE types)
- Section 8.11: Basic Types (now includes sized integers, Unicode types)
- Section 8.15: Instructions (now includes ACROSS loops)
- Section 8.17: Expressions (now includes conditional expressions, ~ operator)

All original sections remain valid; new features are additions that do not override existing content.

APPENDIX G: MODERNIZATION CHECKLIST FOR PROJECTS

For teams migrating ECMA-367:2006 code to 2026 standard:

Syntax Modernization

- [ ] Replace traditional loops with ACROSS where applicable
- [ ] Use `create` instead of `!!` for object instantiation
- [ ] Use `<span style="color:#0066cc;font-weight:bold">attached</span>` tests instead of `<span style="color:#0066cc;font-weight:bold">Void</span>` checks

Type Safety

- [ ] Enable void-safety (gradual: None → Transitional → Complete)
- [ ] Mark **detachable** entities explicitly with `?`
- [ ] Add object tests for safe access

Concurrency (Optional)

- [ ] Identify concurrent access patterns
- [ ] Mark shared objects with `<span style="color:#0066cc;font-weight:bold">separate</span>`
- [ ] Implement SCOOP synchronization points

Modern Features

- [ ] Use inline agents for simple callbacks
- [ ] Consider operator aliases for domain-specific types
- [ ] Use bracket notation for collection access
- [ ] Use assigner commands for properties

Performance

- [ ] Profile code with modern compiler optimizations
- [ ] Leverage void-safety for optimization opportunities
- [ ] Use sized types for memory efficiency

GLOSSARY OF NEW TERMS (2006-2026)

Term	Definition	Introduced
Attached Type	Type that guarantees non-void value	6.1 (2007)
Detachable Type	Type that explicitly allows void	6.1 (2007)
Certified Attachment Pattern (CAP)	Compiler-recognized void safety pattern	6.1 (2007)
SCOOP	Simple Concurrent Object-Oriented Programming	7.0 (2011)
Separate	Mark for entities in different processors	7.0 (2011)
ACROSS Loop	Modern iteration construct	6.5 (2010)
ITERABLE	Protocol for traversable collections	6.5 (2010)
Cursor	Iterator state during ACROSS traversal	6.5 (2010)
Inline Agent	Agent with implementation defined in expression	5.7 (2006)
Alias	Alternative name or symbol for **feature**	5.6 (2006)
Assigner Command	Setter called by assignment syntax	5.6 (2006)
Bracket Notation	Array-like access syntax for features	5.6 (2006)
Conditional Expression	If/then/else used as expression	20.05 (2020)
Non-Conforming Inheritance	Inherit features without subtype conformance	21.11 (2021)
Multiple Generic Constraints	Constrain generic parameter by multiple types	6.0 (2007)
Note Clause	Metadata annotation (replaces `<span style="color:#0066cc;font-weight:bold">indexing</span>`)	6.0 (2007)
Assertion Tag	Named or implicit label for contract assertions	18.11 (2018)

DOCUMENT INFORMATION

Title: ECMA-367:2026 - Eiffel Programming Language
Edition: 2026 (Updated from 2006)
Format: Markdown with UTF-8 BOM
Date: January 2026
Status: Reference Standard

Features Incorporated:

- 20 years of language evolution (2006-2026)
- 5 major **feature** areas
- 40+ specific enhancements
- Complete backward compatibility

For Official ECMA Publication, **convert** to:

- PDF/A with proper formatting
- Technical styling per ECMA guidelines
- Formal BNF-E notation with standard notation
- Official reference version publication

END OF ECMA-367:2026 SPECIFICATION

Original Alphabetical Index (ECMA-367:2006)

A abstract data type6 actual, see under “argument”, “generic” Actual_generics 78 Actual_list 120 Actuals 120 Address 137 address semantics 147, 149 Agent 135 agent 134–137 body syntax 135 callagent equivalent to an inline agent 136 syntactical forms 135 targettype135 callsemantics 137 expression semantics 137 type137 inline agent associated **feature** 135 call-agent equivalent 136 semantics 136 use of **Result** 136 operand closed 136 open 136 position open 136 syntax 135 agent 135, 136 Agent_actual 135 Agent_actual_list 135 Agent_actuals 135 Agent_qualified 135 Agent_target 135 Agent_unqualified 135 aggregate production 26 Algol 60 25 Alias 39 alias same 40 alias 39, 90, 139, 141, 148, 151, 152 Alias Syntax rule39 Alias Validity rule43 Alias_name 39 all 51 alpha_betic 153 alpha_numeric 153 ancestor 46, 92 proper 47 repeated 92 ancestor types of a **class** 46 of a type46 ancestors of a **class** 16 Anchor 72 anchor 75 cyclic 75 set 75 Anchored 72 anchored entity 75 type75 Anchored Type rule76 and 41, 65, 66, 72, 128, 138 and then 41, 61, 62, 65, 66, 128, 138, 140, 156 ANY 2, 20, 22, 23, 45, 52, 53, 79, 82, 85, 103, 108, 110, 111, 113, 114, 115, 116, 124, 131, 132 architecture of Eiffel software 24, 30–33 argument actual 54 syntax 120 formal 54 declaration syntax 55 Argument rule122 argument signatureof a **feature** 38 ARRAY 3 as 47, 48, 83 Assertion 57 assertion 11–14 availability of an assertion clause58 covariance-aware form65 evaluation 63 extension 65 local unfolded form62 monitoring 63 levels 64 semantics 64 syntax 57 unfolded form65 violation 63 weakening and strengthening in descendants 20 Assertion Syntax rule57 Assertion_clause 57 ASSERTION_VIOLATION 3 **assign** 41 assigner call semantics 119 syntax 119 Assigner Callrule119 Assigner Command rule41 assigner mark syntax 41 Assigner_call 119 Assigner_mark 41 Assignment 117 assignment semantics 119 syntax 117 Assignment rule117 associated exception marker of an Old expression 59 **feature** of an inline agent 135 variable of an Old expression 59 Associated Variable Semantics 60 **attached** 101 “**attached** to” 101 entity 118 expression 129 object 7 type77 Index Page numbers in italics refer to the (informative only) introduction of clauses 1 to 7.

semantics 77 ATTACHED_TARGET_VIOLATION 3 attaching an entity 118 attachment 117–119 of an entity 118 Attachment_mark 72 Attribute 100 attribute 8, 100–101 body syntax 100 attribute 100 Attribute_or_routine 37 available 52 assertion clause58 for call52 for creation 110 selectively 52 B Backus-Naur Form 25 base **class** 73 of an expression 74 Base principle 73 Base rule74 base type73 expression 74 multi-constraintformal generic type83 single-constrained formal generic 82 unconstrained formal generic 82 based 73 see also “base type” basic syntax description rule27 basic

type11, 75, 146–147 Basic_expression 137 Basic_manifest_string 144 Binary 41 binary expression target-converted form139 Binary_expression 138 Blanks or tabs 154 block **rescue** 132 BNF 25 BNF-E 25 body of a **feature** syntax 37 BOOLEAN 3, 147 boolean semantics 147 Boolean Expression rule141 Boolean_constant 142 Boolean_expression 137 Bracket 39 bracket syntax 140 Bracket Expression rule141 bracket **feature** 40 Bracket_expression 140 Bracket_target 140 break 154 character 154 semantics 154 Break rule155 C C external syntax 150 inline semantics 151 C 151 C External rule151 C++ external syntax 151 inline semantics 152 C++ 151 C++ External rule152 C_external 150 Call120 call120–126 assigner call, see this term callagent, see under “agent” calltime of an agent 134 dynamic **feature** 124 expression value126 **feature** of a call121 freshness (for once routine) 124 non-object semantics 122 operand 134 position 134 qualified 120 result 126 semantics general 125 syntax 120 target120 targettype121 time (of an agent) 134 type (of a Callexpression) 126 unqualified 120 CallAgent rule135 CallSharing rule93 CallUse rule120 Call_agent 135 Call_agent_body 135 CAP, see “Certified Attachment Pattern” Case Insensitivity principle 29 catcall 130 Certified Attachment Pattern 129 CHARACTER 3, 147 character 153 codes (for special characters) 158 percent 159 manifest 158 semantics 159 set 153 special 158 percent 159 types 147 Character Syntax rule143 CHARACTER_32 3, 147 CHARACTER_8 3, 147 Character_constant 142 CHARACTER_GENERAL 147 Check 62 check syntax 62 check 62 check-correct 62 Choice 97 choice production 26 Choices 97 clash (of **feature** names) 67, 95 **class** 6, 7–10 ancestor types 46 ancestors 16 base 73 client relation 49 clients 7 consistency 61 contract view 54

converting to 89 correctness 62 creation procedures 109 declaration syntax 33 descendant 16 dynamic **class** set 119 effective 34 **expanded** 34 flat-short form54 **frozen** 34 generic 79 header syntax 34 heirs 16 incremental contract view 53 **invariant** 11, 57 involved in a type76 name semantics 31 syntax 31 needs another 24 non-basic 102 parent part45 parents 16 short form53 unfolded inheritance part45 **class** 34, 80 Class ANY rule45 Class Header rule34 Class Namerule31 Class Type rule74 Class_declaration 33 Class_header 34 Class_list 51 Class_name 31 Class_or_tuple_type 72 Class_type 72 Class-LevelCall rule123 client relation 7, 49–54 between classes and types 49 between types 49 **expanded** 50 generic 50 indirect 50 simple 50 client set of a Clients part50 of a **feature** 52 Clients 51 clients syntax 51 cloning objects 115 deep 116 Close_bracket 144 closed operand 136 cluster 24, 30 codes for special characters 158 percent 159 collection, see “garbage collection” combined precondition, postcondition 66 command 38 Comment 154 comment expected 154 free 154 syntax 154 Comment_break 154 Comparison 115 comparison 114–116 of objects 114 compatibility between expressions 85 between types 85 component 25 composite object 102 Compound semantics (non-exception) 96 Compound 56 Conditional 96 conditional semantics 97 syntax 96 conformance 84–88 between expressions 85 **expanded** types 87 formal generic 87 general 86 path 87 reference types 87 signature85 tuple types 87 conforming parent 46 conjunctive 128 consistency of a **class** 61 Constant 142 constant 142–146 attribute 38 manifest semantics 144 syntax 142 type143 manifest value144 syntax 142 Constant Attribute rule142 Constant_attribute 102, 142 Constant_interval 97 constrained generic parameter 79 constraining creation **feature** 80 type79 Constraining_types 79 Constraint 79 constraint (of a Formal_generic) 79 Constraint_creators 80 Constraint_list 80 construct 25 Construct Nameconvention 25 Construct Specimen convention 25 construction time of an agent 134 contains 30 directly 30 context of execution 106 continuing an exception 134 contract 13, 14, 57–64 of a **feature** 58 subcontract 19, 58 view 54 incremental 53 control structure 96–100 conventions textual 27 conversion 88–92 explicit 90 expression 91 **feature** 88 procedure 88 query 88 semantics 90 syntax 88 through 90 to a **class** 89 to and from a type89 type88 Conversion Asymmetry principle 88

Conversion Non-Transitivity principle 88 Conversion principle 88 Conversion Procedure rule88 Conversion Query rule89 Conversion_procedure 88 Conversion_query 88 **convert** 39, 40, 44, 88, 139 Converter 88 Converter_list 88 Converters 88 convertibility, converting, see “conversion” copy semantics 101 copying objects 115 deep 116 correct exception 131 correctness 57–64 total 61 correctness of a **class** 62 covariance-aware formof an inherited assertion 65 covariant argument 85 create 80, 90, 108, 110, 112 creation 107–114 availability 110 constraining creation **feature** 80 expression properties 113 semantics 113 instruction properties 111 semantics 112 unfolded form110 operation 108 procedure general 110 of a **class** 109 of a type109 syntax 108, 110, 112 target110 type110 Creation Clauserule109 Creation Expression Properties 113 creation instruction 7 Creation Instruction rule110 Creation Precondition rule108 Creation principle 107 Creation procedure property 109 Creation Type theorem 110 Creation_call 110 Creation_clause 108 Creation_expression 112 Creation_instruction 110 Creation_procedure 108 Creation_procedure_list 108 Creators 108

Current 72, 76, 102, 107, 108, 122, 125, 136, 141 current **class** 33 object 10, 125 semantics 125 routine 125 type82 cyclic anchor 75 D deanchored formof a type76 Debug 100 debug semantics 100 syntax 100 debug 100 declaration for a **feature** 49 declaration of formal arguments and entities syntax 55 Declaration_body 36 Default Initialization rule104 default value9, 104 default_rescue 131, 132 Deferred 55 **deferred class** 21–22, 34 **feature** 69 inherited as 66 **deferred** 34, 35, 55, 69, 80 Deferred Class property 69 derivation, generic 79 descendant 46, 92 of a **class** 16 proper 47 repeated 92 Descendant Argument rule130 design 22 Design by Contract 13–14, 19, 57–64 **detachable** type77 Digit 159 Digit_sequence 159 direct instance, see “direct” under “instance” disjunctive 128 DLL external semantics 153 syntax 152 dll 152 DLL_external 152 DLL_identifier 152 DLL_index 152 do 37, 55 Double Reserved Word rule156 duplicating objects, see “cloning” dynamic binding 18–20 definition 19 version94 **class** set 119 **feature** of a call124 type119 typeset 119 E effect 66, 69 effecting 21, 66, 69 effective 21 **class** 34 **feature** 69 inherited as 66 Effective Class property 69 Effective_routine 55 efficiency 6 else 57, 96 Else_part 96 **elseif** 96, 97 end 33, 37, 44, 62, 79, 80, 96, 97, 99, 100 **ensure** 57, 70 **ensure** then 65, 70 Entity 102 entity 7, 102 anchored 75 **attached** 118 attaching 118 declaration syntax 55 syntax 102 Entity Declaration rule55 Entity rule102 Entity Semantics rule107

Entity_declaration_group 55 Entity_declaration_list 55 Equality 115 equality deep 116 semantics 115 syntax 115 Equivalent Dot Form 141 evaluation of an assertion 63 evaluation position 104 EXCEPTION 3 exception 14–15, 130–134 cases 133 continuing 134 definition 130 ignoring 134 properties 133 see also “**rescue**” semantics 132 type133 exception-correct 131 execution context 106 of a system 33 terminology 28 Exit_condition 99 **expanded class** 34 client 50 conformance 87 formal generic parameter 82 type10, 74 **expanded** 34 expected comment 154 explicit conversion 90 Explicit_creation_call 110 Explicit_creation_type 110 Explicit_value 36 **export** 49–54 **export** 51 **export** adaptation syntax 51 Export rule122 Export Status principle 122 exported **feature** 52 Expression 137 expression 137–142 agent semantics 137 type137 **attached** 129 base type, base **class** 74 binary target-converted form139 bracket syntax 140 call value126 conformance 85 convertibility 91 creation properties 113 semantics 113 syntax 112 equality semantics 115 syntax 115 Equivalent Dot Form 141 inequality semantics 115 **inspect** 97 old, see “old” operand 138 operator syntax 138 parenthesized semantics 138 parenthesized form139 polymorphic 119 semantics semistrict case 140 strict case 139 syntax 137 type141 Expression_list 84 extended final name 48 Extended_feature_name 39 extendibility 6 extension of an assertion 65 External 148 external C semantics 151 syntax 150 C++ semantics 152 syntax 151 DLL semantics 153 syntax 152 file 150 semantics 150 syntax 150 signature syntax 149 software 148–153 syntax 148 external 148 External DLL rule152 External File rule150 External Signature rule149 External_argument_types 149 External_file 150 External_file_list 150 External_file_use 150 External_language 148 External_name 148 External_signature 149 External_system_file 150 External_type 149 External_type_list 149 External_user_file 150 F failure 14, 130 **False** 66, 142, 147 **feature** 8, 35–44 adaptation 64–72 associated (for an inline agent) 135 body syntax 37 bracket 40 call, see this term constraining creation **feature** 80 conversion **feature** 88 declaration syntax 36 **deferred** 69 dynamic (of a call) 124 effecting 21, 66, 69 effective 69 exported 52

features of a type82 **frozen** 37, 43, 67 identifier-only 40 immediate 35 inherited 35, 94 inherited as **deferred** 66 inherited as effective 66 introducing 35 name generically constrained 83 identifier40 same 40 syntax 39 new **feature** list syntax 37 of a call121 operator 40 part syntax 36 secret 52 selectively available 52 version94 **feature** 36, 53 Feature adaptation rule45 Feature Body rule37 **feature** categories 36 Feature Declaration rule43 Feature Identifier principle 40 Feature Namerule95 Feature principle 39 Feature_adaptation 44 Feature_body 37 Feature_clause 36 Feature_declaration 36 Feature_declaration_list 36 Feature_list 51 Feature_name 39 Feature_set 51 Feature_value 36 Features 36 field 102 file external syntax 150 final name 48 extended 48 set 48 flat-short form54 floating-point IEEE standard 3, 147 types 147 form deanchored 76 flat-short 54 short 53 Formal 102 Formal Argument rule55 Formal Generic rule78 formal parameter, see under “generic‘” formal, see under “argument”, “generic” Formal_arguments 55 Formal_generic 78 Formal_generic_list 78 Formal_generic_name 78 Formal_generics 78 free comment 154 operator 157 Free Comment rule154 freshness of a once routine cal 124 from 99 **frozen class** 34 **feature** 37, 43, 67 formal generic parameter 78, 86 **frozen** 34, 37, 43, 67, 78, 86 FUNCTION 3 function 8, 38 inline agent use of **Result** 136 G garbage collection 114 General conformance 86 general creation procedure 110 General Validity rule28 generating type101 generator 101 generic actual parameter 16 syntax 78 **class** 79 client 50 constraint syntax 79 derivation 79 formal parameter 16 conformance 87 constrained 79 **frozen** 78, 86 multi-constrained base type83 reference or **expanded** status 82 self-

initializing 79 syntax 78 unconstrained 79 substitution 82 supplier 50 type79 Generic Constraint rule80 Generic Derivation rule80 Generic Type Adaptation rule82 generically constrained **feature** name 83 generically derived **class** type semantics 81 generic-creation-ready type81 genericity 15, 20, 78–83 combined with inheritance 20 constrained 20 unconstrained 16 H header comment semantics 155 Header Comment rule155 Header_comment 36 Header_mark 34 heir 16, 46 Hoare triple notation 61 I I nstruction creation semantics 112 identifier156 of a **feature** name 40 Identifier rule141, 156 Identifier_list 55 identifier-only **feature** 40 IEEE floating-point numbers 3, 147 if 97 ignoring an exception 134 immediate **feature** 35 implicative 128

implies 41, 66, 128, 138, 140 imported formof a Non_object_call 121 incremental contract view 53 indirect client 50 inequality semantics 115 infix operator 10 see also Binary **inherit** 46 **inherit** 44, 45, 80 Inherit_clause 44 Inheritance 44 inheritance 16–22, 44–49, 64–72, 92–96 combined with genericity 20 multiple 46 repeated 92–96 definition 92 single 46 syntax 44 inherited as **deferred** 66 as effective 66 **feature** 35, 94 name 49 Initialization 99 initialization 9 inline C 151 C++ 152 inline 150, 151 Inline Agent Requirements 136 Inline Agent rule136 inline, see under “agent” Inline_agent 135 **inspect** expression 97 value98 **inspect** 97 INSPECT_RANGE_VIOLATION 3 instance 6, 7, 8, 9, 10, 11, 22 current 10 direct 7, 24 of a **class** 73 of a type73 of a **class** 73 of a type73 Instance principle 73 Instruction 56 instruction creation properties 111 syntax 110 unfolded form110 setter 105 syntax 56 INTEGER 3, 147 Integer 159 integer semantics 160 syntax 159 types 147 Integer rule160 INTEGER_16 3, 147 INTEGER_32 3, 147 INTEGER_64 3, 147 INTEGER_8 3, 147 Integer_base 159 Integer_base_letter 159 Integer_constant 142 INTEGER_GENERAL 147 Internal 55 interval 97 unfolded form98 Interval rule98 introducing a **feature** 35 Invariant 57 **invariant** of a **class** 11, 57 of a loop62 **invariant** 57 INVARIANT_ENTRY_VIOLATION 3 INVARIANT_EXIT_VIOLATION 3 involved types and classes involved in a type76 J Join rule71 Join Semantics rule72 Join-Sharing Reconciliation rule95 K Key_list 55 keyword 155 L Lace 24 language interfaces to other languages 148–153 registered 149 language processing tool 28 latest applicable targetand result 124 left-aligned form146 letter 153 Letter Case rule155 lexical components 153–161 like 72, 75, 76 Line sequence145 Line_wrapping_part 145 Local 102 local 55 Local Variable rule56 local variable, see under “variable” Local_declarations 55 longest break prefix 146 Loop99 **loop invariant** 62 semantics 99 syntax 99 **variant** 62 loop99 Loop_body 99 **loop**-correct 62 M machine 28 manifest character 158 semantics 159 constant 143 semantics 144 syntax 142 string semantics 146 syntax 144 tuple syntax 84 value144 Manifest Constant rule100 Manifest String rule145 Manifest_constant 142 Manifest_string 144 Manifest_tuple 84 Manifest_type 142 Manifest_value 142 Manifest-Type Qualifier rule143

matching branch 99 MEMORY_ALLOCATION_FAILURE 3 Message 35 monitoring of assertions 63 Multi_branch 97 multi-branch **inspect** value98 semantics 99 syntax 97 unfolded form97 Multi-branch rule98 Multiple Constraints rule83 multiple inheritance 46 Multiple_constraint 80 N name clash 67, 95 final, see “final name” inherited 49 NameClash rule95 NATURAL 3, 147 NATURAL_16 3, 147 NATURAL_32 3, 147 NATURAL_64 3, 147 NATURAL_8 3, 147 needed **class** 24 needs (relation between classes) 24 new **feature** list syntax 37 new line 154 New_export_item 51 New_export_list 51 New_exports 51 New_feature 37 New_feature_list 37 NO_MORE_MEMORY 113 Non_conformance 44 Non_object_call 120 non-basic **class** 102 type102 non-conforming parent 46 NONE 2, 8, 9, 12, 13, 16, 17, 44, 52, 53, 86, 119 non-generic **class** type semantics 74 non-generic type79 Non-Object Callrule121 non-object call, see under “call” non-production syntax rule27 non-terminal 25 not 62, 138 not enough memory available 114 note33, 155 Note_entry 34 Note_item 34 Note_list 34 Note_name 34 Note_values 34 Notes 33 notes semantics 34 syntax 33 null pointers, avoiding, see “void” numeric 153 O object 101–107 comparison 114 composite 102 copying and cloning 115 deep 116 creation 107–114 current 125 semantics 125 duplication, see “cloning” equality 115 generating type, generator 101 inequality 115 semantics 101 targetobject 123 test, see “object test” Object principle 101 object test see also Object-Test Local semantics 128 syntax 127 Object Test rule127 Object_call 120 Object_test 127 object-oriented design 6–7, 22 Object-Test Local 127 scope 128 semantics 128 Obsolete 35 **obsolete** semantics 35 syntax 35 **obsolete** 35 Old 58 old semantics 59 syntax 58 old 58, 59, 61, 66, 138, 142 Old Expression rule59 OLD_VIOLATION 3 Once 55 once 37, 55, 137 once, see under “routine” Once_string 137 Only 60 only syntax 60 unfolded **feature** list60 unfolded formof clause61 only 60 Only Clauserule60 open operand 136 Open_bracket 144 openness 6 operand, see under “agent”, “call”, “closed”, “expression” and “open” operation creation 108 Operator 41 operator **feature** 40 free 157 precedence levels 138 predefined 156 prefix or infix 10 see also Unary and Binary same 40 semistrict 140 standard 156 symbol 157 syntax

41, 138 Operator Expression rule139 Operator_expression 138 or 41, 66, 72, 128, 138 or else 41, 66, 128, 138, 140, 155, 156 organization of Eiffel software 24, 30–35 origin 69 P parameter, see “formal parameter” and “actual parameter”

under “generic” Parent 44 parent 16, 46 conforming 46 non-conforming 46 part for a **class** 45 for a type45 Parent rule47 Parent_list 44 Parent_qualification 67 Parenthesized 137 parenthesized expression semantics 138 parenthesized formof an expression 139 Parenthesized_target 120 path conformance 87 percent character codes 159 Placeholder 135 platform 28 POINTER 3, 147 polymorphic data structure 20 expression 119 polymorphism 18–20 portability 6 position of a calloperand 134 Postcondition 57 postcondition 11, 57 combined 66 POSTCONDITION_VIOLATION 3 precedence of operators 138 precedes 104 Precondition 57 precondition 11, 57 combined 66 statically satisfied 91 Precondition Export rule58 PRECONDITION_VIOLATION 3 precondition-free routine 91 **Precursor** 67 **Precursor** 67 relative unfolded form67 unfolded form68 precursor 95 joined features 71 semantics 68 syntax 67 **Precursor** rule67 predefined operator 156 PREDICATE 3 prefix 146 longest break 146 prefix operator 10 see also Unary prevailing immediately 97 printable 153 PROCEDURE 3 procedure 8, 38 conversion 88 creation general 110 of a **class** 109 of a type109 production 26 aggregate 26 choice 26 non-production syntax rule27 repetition 26 program, see system proper ancestor 47 descendant 47 properly set variable 105 property public 53 secret 53 public property 53 Q qualified call120 query 38 conversion query 88 Query_mark 36 R Read_only 102 read-only 102 void test 128 REAL 3, 147 Real number 160 semantics 161 REAL_32 3, 147 REAL_64 3, 147 Real_constant 142 REAL_GENERAL 147 reattachment 117 semantics 118 Reattachment principle 117 redeclaration 21, 35, 64 Redeclaration rule70 redeclare 64 Redefine 68 **redefine** 66 **redefine** 68 Redefine Subclause rule68 redefinition 18, 66 semantics 68 syntax 68 reference 101 conformance 87 formal generic parameter 82 semantics 101 type10, 74 registered language syntax 149 Registered_language 149 relative unfolded formof a **Precursor** 67 reliability 6 Rename 47 **rename** 47, 48, 80 Rename Clauserule48 Rename_list 47 Rename_pair 47 Renaming 79 renaming 18 syntax 47 Renaming principle 48 repeated ancestor 92 descendant 92 inheritance 92–96 definition 92 replication 93 sharing 93 Repeated Inheritance Consistency constraint94 Repeated Inheritance rule93 repetition production 26 replication (under repeated inheritance) 93 Replication Semantics rule93 **require** 57, 70, 91 **require** else 65, 70, 91

Rescue 131 **rescue** block 132 default 131, 132 original semantics 131 syntax 131 **rescue** 131 Rescue clauserule131 reserved word 155 **Result** 56, 59, 102, 103, 104, 105, 106, 125, 126, 136, 141 use in an inline agent 136 result inline agent 136 latest applicable (non-fresh callto once query) 124 of a call126 Retry 131 **retry** 131 Retry rule131 reusability 6 root **class** 24, 33 creation procedure 24 object 24 procedure 33 procedure name 31 type33 typename 31 Root Procedure rule32 Root Type rule32 ROUTINE 3 routine 8, 38, 54–57 body syntax 55 current 125 external syntax 148 failure 14 non-once execution semantics 125 once execution semantics 124 freshness of a call124 precondition-free 91 ROUTINE_FAILURE 3 Routine_mark 55 run time 28 S same alias 40 **feature** name 40 operator 40 scope Object-Test Local 128 read-only void test 128 secondary part97 secret **feature** 52 property 53 seed 69 Select 93 **select** syntax 93 **select** 93 Select Subclause rule94 selectively available 52 self-initializing formal parameter 79 type103 variable 104 semantics 28, 28–29 copy 101 reference 101 Semicolon Optionality rule29 semistrict 140 term 128 separator 26 setter instruction 105 sharing (under repeated inheritance) 93 short form14, 53 Sign 142 Sign Syntax rule143 signature argument signature38 conformance 85 external 149 semantics 149 syntax 149 of a **feature** 38 signature149 simple client 50 string 159 single inheritance 46 Single_constraint 79 Single-levelCall rule130 sized variants of basic types 147 of STRING 147 source of reattachment 117 special symbol 156 Special_expression 137 specimen 25 stand-alone type77 standard operator 156 statically satisfied precondition 91 strict expression semantics 139 STRING 3, 147 string 159 manifest semantics 146 syntax 144 semantics 159 simple 159 verbatim semantics 146 STRING_32 3, 147 STRING_8 3, 147 String_content 144 strong typing 6, 19 subcluster 30 subcontract 19, 58 subexpression 138 subobject 102 substitution, generic 82 supplier 50 generic 50 symbol 155 operator 157 special 156 synonym 42 syntax 25, 25–30 non-production 27 system 24, 31 execution 33 T Tag 57 Tag_mark 57 Target120 target creation 110

latest applicable (non-fresh callto once routine) 124 object 123 of a call120 of reattachment 117 type of a call121 of a callagent 135 valid 123 targetevaluation of a void-unsafe system failed 123 Targetrule123 target-converted formof a binary expression 139 term 128 terminal 25 textual conventions 27 then 57, 96, 97, 99 Then_part 96 Then_part_list 96 time call134 construction 134 token25 tool language processing 28 total correctness 61 trigger 130 **True** 57, 62, 66, 142, 147 TUPLE 3 tuple 83–84 conformance 87 manifest 84 syntax 83, 84 typesequence 84 valuesequences 84

Tuple_parameter_list 83 Tuple_parameters 84 Tuple_type 83 Type 72 type10–11, 72–92 agent expression 137 ancestor types 46 anchored 75 **attached** 77 semantics 77 base 73 basic 11, 75, 146–147 sized variants 147 character 147 client relation 49 compatibility 85 constraining 79 conversion type88 converting to and from 89 creation 110 creation procedures 109 deanchored form76 dependency 31 **detachable** 77 direct instance 73 dynamic 119 dynamic typeset 119 **expanded** 74 conformance 87 expression 141 features of a type82 floating-point 147 generating 101 generic 79 semantics 81 generic-creation-ready 81 instances 73 integer 147 involved in a type76 manifest constant 143 non-basic 102 non-generic 79 of a Call126 of an exception 133 parent part45 reference 74 conformance 87 safety 130 self-initializing 103 sequence (of a tuple type) 84 stand-alone 77 syntax 72 targettype of a call121 of a callagent 135 tuple conformance 87 syntax 83 typesequence 84 valuesequences 84 values 73 Type Semantics rule74 Type_interval 97 Type_list 78 Type_mark 36 U Unary 41 Unary_expression 138 unconstrained generic parameter 79 Undefine 68 **undefine** 68 Undefine Subclause rule68 undefinition semantics 69 syntax 68 unfolded creators partof a **class** 108 **feature** listof an Only clause60 form creation instruction 110 imported formof a Non_object_call 121 interval 98 multi-branch 97 of a possibly multiple declaration 42 of an assertion 65 local 62 **Precursor** 68 relative 67 target-converted formof a binary expression 139 inheritance partof a **class** 45 Only clause61 Universal Conformance principle 45 universe 24, 31 Unlabeled_assertion_clause 57 unqualified call120 Unqualified_call 120 Unregistered_language 148 **until** 99 Upper name, lower name 29 use 150 V valid 28 validity 28 validity constraint28 value101–107 boolean valuesemantics 147 default 9 **inspect** 98